



Media Engineering and Technology Faculty  
German University in Cairo

# Multi-Agent Systems for AI Clinical Decisioning via Automation Workflows

## Bachelor Thesis

Author: Abdelrahman Mohamed Amin  
Supervisors: Dr. Shereen Moataz Afifi  
Submission Date: 29 May, 2026

This is to certify that:

- (i) the thesis comprises only my original work toward the Bachelor Degree
- (ii) due acknowledgment has been made in the text to all other material used

---

Abdelrahman Mohamed Amin  
29 May, 2026



# Acknowledgments

I would first like to thank my supervisor, Dr. Shereen Moataz Afifi, for her oversight of this thesis at the institution. Her guidance on scope, structure, and academic framing kept the project anchored to a clear research contribution, and her feedback on the written chapters helped me sharpen the way the work is presented and defended.

I am additionally grateful to Engineer Mohamed from Bareeq Solutions, who took the time to review parts of the work and share his feedback, which helped me refine specific elements of the design.

I am grateful to Engineer Mohamed Amin for his steady technical mentorship across the architecture decisions and the implementation work, and to Engineer Islam Amin for his careful review of the thesis and his feedback on clarity and structure.

Finally, I am deeply indebted to my family, friends, and colleagues for their constant encouragement, patience, and feedback throughout this work. Their support made it possible to stay focused through the long stages of research, implementation, and writing.



# Abstract

Recent literature identifies workflow coordination, not individual agent capability, as the principal unsolved problem in clinical multi-agent AI. This thesis presents CMADS, a seven-agent LangGraph clinical decision support pipeline that targets that problem through two architectural contributions: a non-destructive Reviewer–Refiner pattern that lets a critique step rewrite a flawed primary diagnosis while preserving the original reasoning for audit, and a four-tier shared-memory subsystem instantiated from the CoALA taxonomy, combined with a three-phase anti-anchoring protocol that controls undue influence from retrieved priors.

Evaluation uses an LLM-as-judge protocol on a 160-patient Synthea-derived cohort that spans eight cardiometabolic disease families. The eight families bound the evaluation cohort, not the system: the agents emit free-text differentials and the pipeline is disease-agnostic in design — the scope of the evaluation reflects the disease coverage Synthea can produce with sufficient pre-diagnosis signal, not a fixed output vocabulary. With multi-level memory enabled, the pipeline surfaces the correct diagnosis somewhere in its top-five differential for  $\sim 95\%$  of the 160 synthetic patients (151/160). On the same 160 patients, the multi-level memory configuration lifts that found rate over the single-level baseline on a paired controlled comparison. The system is built around two engineering properties demonstrated in this work — **explainability** (every agent’s reasoning, memory operation, and retrieved guideline passage is exposed verbatim in a doctor-facing console) and **reproducibility** (every run writes a deterministic execution trace so any number reported here can be re-derived from the on-disk artefacts) — and two further architectural properties (provider-agnostic LLM adapter and pluggable converter agents for new clinical modalities) whose demonstration at scale is left to future work.



# List of Acronyms and Abbreviations

|              |                                                                           |
|--------------|---------------------------------------------------------------------------|
| <b>AKI</b>   | Acute Kidney Injury                                                       |
| <b>BMI</b>   | Body Mass Index                                                           |
| <b>BP</b>    | Blood Pressure                                                            |
| <b>CDC</b>   | Centers for Disease Control and Prevention                                |
| <b>CDM</b>   | Common Data Model (OMOP CDM v5.4)                                         |
| <b>CDS</b>   | Clinical Decision Support                                                 |
| <b>CHF</b>   | Chronic Congestive Heart Failure                                          |
| <b>CKD</b>   | Chronic Kidney Disease                                                    |
| <b>CMADS</b> | Clinical Multi-Agent Decisioning System (the system built in this thesis) |
| <b>CoALA</b> | Cognitive Architecture for Language Agents                                |
| <b>CSV</b>   | Comma-Separated Values                                                    |
| <b>DAG</b>   | Directed Acyclic Graph                                                    |
| <b>dbt</b>   | data build tool ( <b>dbt-core</b> )                                       |
| <b>eGFR</b>  | Estimated Glomerular Filtration Rate                                      |
| <b>EHR</b>   | Electronic Health Record                                                  |
| <b>ESRD</b>  | End-Stage Renal Disease                                                   |
| <b>FHIR</b>  | Fast Healthcare Interoperability Resources (HL7 standard, release 4)      |
| <b>GUC</b>   | German University in Cairo                                                |
| <b>HFpEF</b> | Heart Failure with Preserved Ejection Fraction                            |
| <b>HFrEF</b> | Heart Failure with Reduced Ejection Fraction                              |
| <b>HL7</b>   | Health Level Seven (clinical messaging standard family)                   |
| <b>HR</b>    | Heart Rate                                                                |
| <b>IHD</b>   | Ischemic Heart Disease                                                    |
| <b>LLM</b>   | Large Language Model                                                      |

**LOINC** Logical Observation Identifiers Names and Codes  
**LVEF** Left Ventricular Ejection Fraction  
**MAS** Multi-Agent System  
**MDT** Multi-Disciplinary Team  
**MRA** Mineralocorticoid Receptor Antagonist  
**NICE** National Institute for Health and Care Excellence (UK)  
**OMOP** Observational Medical Outcomes Partnership  
**QRISK** QRISK cardiovascular-risk score (UK)  
**RAG** Retrieval-Augmented Generation  
**SaMD** Software as a Medical Device  
**SDD** System Design Document  
**SNOMED** Systematised Nomenclature of Medicine – Clinical Terms  
**SRD** System Requirements Document  
**SSE** Server-Sent Events  
**T2DM** Type 2 Diabetes Mellitus  
**UUID** Universally Unique Identifier

# Contents

|                                                                             |            |
|-----------------------------------------------------------------------------|------------|
| <b>Acknowledgments</b>                                                      | <b>III</b> |
| <b>Abstract</b>                                                             | <b>V</b>   |
| <b>List of Acronyms and Abbreviations</b>                                   | <b>VII</b> |
| <b>1 Introduction</b>                                                       | <b>1</b>   |
| 1.1 Motivation . . . . .                                                    | 1          |
| 1.2 Key Terms and Definitions . . . . .                                     | 2          |
| 1.3 Research Gap and Problem Statement . . . . .                            | 3          |
| 1.4 Aim and Research Question . . . . .                                     | 4          |
| 1.5 Objectives . . . . .                                                    | 4          |
| 1.6 Scope and Assumptions . . . . .                                         | 5          |
| 1.7 Thesis Outline . . . . .                                                | 6          |
| <b>2 Background</b>                                                         | <b>7</b>   |
| 2.1 Concepts Overview . . . . .                                             | 7          |
| 2.1.1 The Limitations of Single-Model AI in Clinical Settings . . . . .     | 7          |
| 2.1.2 Multi-Agent Systems: Principles and Architecture . . . . .            | 8          |
| 2.1.3 Large Language Models as Clinical Agents . . . . .                    | 8          |
| 2.1.4 Automation Workflow Design in Clinical AI . . . . .                   | 9          |
| 2.1.5 Clinical Reasoning and Decision-Making . . . . .                      | 9          |
| 2.2 Search Strategy . . . . .                                               | 10         |
| 2.3 Literature Review . . . . .                                             | 11         |
| 2.3.1 Foundational Multi-Agent Frameworks for Clinical AI . . . . .         | 12         |
| 2.3.2 Agent Coordination Architectures and Workflow Orchestration . . . . . | 12         |
| 2.3.3 Inter-Agent Deliberation, Debate, and Conflict Resolution . . . . .   | 14         |
| 2.3.4 Memory and State in Multi-Agent Clinical Systems . . . . .            | 15         |
| 2.3.5 Domain-Specific Clinical Workflow Deployment . . . . .                | 16         |
| 2.3.6 Safety, Robustness, and Evaluation of Clinical MAS . . . . .          | 17         |
| 2.4 Analysis and Discussion . . . . .                                       | 18         |
| 2.4.1 Key Findings . . . . .                                                | 18         |
| 2.4.2 Research Gaps Identified . . . . .                                    | 20         |
| 2.4.3 Comparative Summary . . . . .                                         | 20         |

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>3</b> | <b>Methodology</b>                                        | <b>23</b> |
| 3.1      | System Architecture Overview . . . . .                    | 23        |
| 3.2      | Synthetic Data Generation and Case Filtering . . . . .    | 24        |
| 3.2.1    | Synthea Generation . . . . .                              | 24        |
| 3.2.2    | Target-Cohort Filtering . . . . .                         | 25        |
| 3.2.3    | LLM-Based Detectability Verifier . . . . .                | 26        |
| 3.2.4    | Gold Case Construction . . . . .                          | 27        |
| 3.3      | Multi-Agent System . . . . .                              | 29        |
| 3.4      | Agent Blueprint . . . . .                                 | 30        |
| 3.5      | Agent Specifications . . . . .                            | 32        |
| 3.5.1    | EHR Analyst Agent . . . . .                               | 32        |
| 3.5.2    | Lab Interpreter Agent . . . . .                           | 33        |
| 3.5.3    | Diagnostic Reasoning Agent . . . . .                      | 34        |
| 3.5.4    | Clinical Reviewer Agent . . . . .                         | 35        |
| 3.5.5    | Diagnostic Refiner Agent . . . . .                        | 36        |
| 3.5.6    | LLM Evaluator Agent . . . . .                             | 37        |
| 3.5.7    | Treatment Planning Agent . . . . .                        | 37        |
| 3.6      | Shared Memory Design . . . . .                            | 38        |
| 3.6.1    | Base layer: <code>PipelineState</code> . . . . .          | 39        |
| 3.6.2    | Multi-level memory subsystem . . . . .                    | 39        |
| 3.6.3    | Memory consolidation node . . . . .                       | 41        |
| 3.6.4    | Access pattern . . . . .                                  | 41        |
| 3.7      | Retrieval-Augmented Generation Layer . . . . .            | 41        |
| 3.8      | LLM Configuration and Provider Abstraction . . . . .      | 42        |
| 3.9      | Observability and Trace Inspection . . . . .              | 43        |
| 3.10     | Evaluation Framework . . . . .                            | 44        |
| 3.10.1   | Ground Truth Extraction . . . . .                         | 44        |
| 3.10.2   | LLM-as-Judge Methodology . . . . .                        | 44        |
| 3.10.3   | Metrics . . . . .                                         | 44        |
| 3.11     | Implementation Stack and Deployment . . . . .             | 45        |
| 3.11.1   | Storage layer and scalability . . . . .                   | 47        |
| 3.12     | End-to-End Execution Pipeline . . . . .                   | 48        |
| <b>4</b> | <b>Results</b>                                            | <b>51</b> |
| 4.1      | Overview . . . . .                                        | 51        |
| 4.2      | Experimental Setup . . . . .                              | 51        |
| 4.3      | Evaluation Metrics . . . . .                              | 52        |
| 4.4      | End-to-End Diagnostic Outcomes . . . . .                  | 53        |
| 4.4.1    | Detectability-Gate Sensitivity . . . . .                  | 55        |
| 4.5      | Per-Agent Reliability and Latency . . . . .               | 55        |
| 4.6      | Adaptive Reasoning and Reviewer/Refiner Effects . . . . . | 57        |
| 4.7      | Multi-Level Memory A/B Study . . . . .                    | 57        |
| 4.8      | Model Selection Pilot . . . . .                           | 58        |
| 4.9      | Comparison with the Literature . . . . .                  | 59        |

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| 4.9.1    | Revisiting the Optimisation Paradox . . . . .       | 61        |
| 4.10     | Qualitative Case Walkthroughs . . . . .             | 61        |
| 4.11     | Single-LLM Controlled Baseline . . . . .            | 62        |
| 4.12     | Doctor-Facing Dashboard . . . . .                   | 64        |
| 4.12.1   | Doctor (runtime) journey . . . . .                  | 64        |
| 4.12.2   | Researcher (statistics) journey . . . . .           | 66        |
| <b>5</b> | <b>Discussion</b>                                   | <b>69</b> |
| 5.1      | Interpretation of the Headline Outcomes . . . . .   | 69        |
| 5.2      | Interpretation of the Memory A/B . . . . .          | 69        |
| 5.3      | Model Comparison Read . . . . .                     | 70        |
| 5.4      | Token and Cost Accounting . . . . .                 | 70        |
| 5.5      | Clinical Workflow Integration . . . . .             | 70        |
| 5.6      | Regulatory Considerations . . . . .                 | 72        |
| 5.7      | Generalisation Beyond Synthea . . . . .             | 72        |
| 5.8      | Clinician Acceptance and Interpretability . . . . . | 73        |
| 5.9      | Limitations . . . . .                               | 74        |
| <b>6</b> | <b>Conclusion and Future Work</b>                   | <b>75</b> |
| 6.1      | Conclusion . . . . .                                | 75        |
| 6.2      | Contributions . . . . .                             | 76        |
| 6.3      | System properties beyond accuracy . . . . .         | 76        |
| 6.4      | Key Empirical Findings . . . . .                    | 78        |
| 6.5      | Limitations . . . . .                               | 78        |
| 6.6      | Future Work . . . . .                               | 79        |
| 6.7      | Closing Remarks . . . . .                           | 80        |
| <b>A</b> | <b>Appendix</b>                                     | <b>81</b> |
|          | <b>References</b>                                   | <b>87</b> |



# List of Tables

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |    |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Inclusion and Exclusion Criteria for Selected Articles . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 11 |
| 3.1 | Detectability-verifier prompt structure. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 26 |
| 3.2 | Per-patient Gold output files: contents and consumers. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | 28 |
| 3.3 | The seven specialised agents and their shared-memory dependencies. . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | 32 |
| 3.4 | Match-type rubric applied by the LLM Evaluator. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | 45 |
| 4.1 | Headline numbers at a glance. Every value below is computed on the same 160-patient paired cohort; each result has its own section in this chapter for the supporting detail. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                     | 51 |
| 4.2 | Headline diagnostic outcomes on the 160-patient paired cohort under the principal multi-level memory configuration. Wilson 95% confidence intervals are shown in brackets. The same 160 UUIDs anchor the paired memory A/B in Section 4.7, where the single-level arm scores 54.4% DIRECT (87/160) on the identical patient set — a +23.1 percentage-point gap in favour of the multi-level configuration on the paired comparison. .                                                                                                                                                                                     | 53 |
| 4.3 | Diagnostic outcomes by target disease family on the 160-patient paired cohort under the principal multi-level memory configuration. The renal families (ESRD, CKD stage 3, CKD stage 2) are reported separately and as a combined subtotal because they share a single NICE management pathway. Rows marked † have $n \leq 8$ and are not powered for per-disease accuracy claims. . . . .                                                                                                                                                                                                                                | 54 |
| 4.4 | Per-agent reliability on the 160-patient cohort. <i>Output file present</i> = the agent produced its persisted JSON output (best-case view, evaluated over the full $n = 160$ ); <i>Success (trace)</i> = the status string written into the run trace (worst-case view: 49 of 160 runs predate a telemetry change and report <i>trace missing</i> on the LLM Evaluator and Treatment Planning rows). Best-case = the output file confirms success on those 49 runs; worst-case = those 49 runs are counted as failures. The two views diverge only for the last two rows and bound the agent’s true reliability. . . . . | 56 |
| 4.5 | Per-agent wall-clock latency on the 160-patient cohort, computed from cases with a recorded trace entry. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 56 |

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |    |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.6 | Paired model comparison on a fixed twenty-patient subset. Identical patients, prompts, evaluator, and memory configuration; only the diagnostic model differs. Wall-clock is the lead column — roughly $13\times$ faster for the cloud-served GPT-OSS-120B than for the locally served Med42-70B. . . .                                                                                                                                                                                                                                                                                                                                              | 59 |
| 4.7 | Position of the proposed system relative to the most relevant direct comparators in the multi-agent clinical AI literature. Headline metrics are reproduced as reported in the original papers and are not directly substitutable for the DIRECT / INDIRECT / MISS metric used in this work.                                                                                                                                                                                                                                                                                                                                                         | 60 |
| 4.8 | Paired single-LLM vs. multi-agent comparison on the same $n = 160$ patients. Both arms use the identical reasoning model (GPT-OSS-120B) and judge model (Qwen3-32B). The difference between arms is the multi-agent orchestration — parallel Stage 1, adaptive Diagnostic loop, non-destructive Reviewer-Refiner, and treatment gating. The multi-agent column is reported on the 160 patients used for the single-LLM baseline; the principal headline in Section 4.4 reports the multi-level memory configuration on a different paired-160 cohort, hence the slight numerical difference between the two reports of the 7-agent pipeline. . . . . | 63 |

# Chapter 1

## Introduction

### 1.1 Motivation

Clinical diagnostic error remains a major source of preventable patient harm. A large outpatient-care review estimated that around five per cent of United States adults experience a diagnostic error in any given year [1], and a 2024 analysis attributed approximately 795,000 deaths or permanent disabilities each year in the United States alone to diagnostic error [2]. At the same time, the clinical workload around the electronic health record has grown: a time and motion study found that physicians spend nearly two hours on the EHR for every hour of direct patient contact [3]. This leaves less time for the difficult part of diagnosis: connecting symptoms, previous conditions, medications, laboratory trends, risk factors, and clinical guidelines into one coherent judgement.

This is why clinical decision support remains an important research area. Diagnosis is not a single-step task. It requires the clinician to compare competing explanations, notice missing data, avoid premature closure, and decide what evidence should be trusted. A patient record may contain clear signals, but it may also contain old diagnoses, incomplete medication lists, abnormal values that are not clinically meaningful, and findings that point in different directions. Any useful clinical AI system therefore needs to support the reasoning process, not only produce a final answer.

Large language models (LLMs) have renewed interest in this problem because they can read long clinical text, summarise evidence, and explain a line of reasoning in natural language, sitting within the broader convergence of human and artificial intelligence in medicine described by Topol [4]. Recent studies show strong results on medical question-answering tasks and controlled clinical benchmarks [5, 6, 7]. However, benchmark performance does not automatically translate into a safe clinical workflow. A single LLM call is still a monolithic process: the same model reads all inputs, forms the diagnosis, and presents the result without an independent reviewer inside the workflow. Prior work shows that medical LLM errors often come from domain knowledge gaps and from cognitive biases such as anchoring and confirmation bias [6, 8]. These are the same types of failure that clinical decision support is meant to reduce.

Multi-agent systems provide a more suitable architecture for this setting. Instead of asking one model to perform the whole clinical task, the work is distributed across agents with separate responsibilities. One agent can analyse the EHR, another can interpret laboratory trends, another can construct the differential diagnosis, and a reviewer can challenge the result before a final answer is saved. This resembles the way complex cases are often discussed by clinical teams, where the value comes not only from one expert opinion but from structured comparison and review. Recent systems show that multi-agent LLM workflows can improve medical reasoning in specific settings: MDAgents routes cases by complexity across solo and group consultation modes [7]; ZODIAC reaches cardiologist-level performance on cardiac data with a human-review stage [9]; ClinicalLab deploys department-specific agents across eleven hospital departments on real cases [10]; the MAC framework structures rare-disease consultation as multi-doctor debate under a supervisor [11]; RareAgents combines persistent memory and tool use for rare-disease diagnosis [12]; and MDTeamGPT models a self-evolving multidisciplinary team that debates before synthesising the final answer [13].

The remaining challenge is not simply choosing a larger or more medical model. It is designing the workflow that connects the agents. If agents pass weak outputs to one another, silently agree with an early mistake, or overwrite useful intermediate reasoning, the final result may look polished while still being clinically wrong. Bedi et al. articulate this tension between individually strong components and their coordination [14]; their framing motivates the design choices made here — explicit shared state, a non-destructive reviewer step, and a bounded diagnostic critique loop — but this thesis does not empirically reproduce their best-of-breed versus coordinated comparison. Starting from that motivation, the thesis designs and evaluates the Clinical Multi-Agent Decisioning System (CMADS), a seven-agent clinical decision support pipeline built on synthetic patient records, multi-level shared memory, NICE guideline retrieval, automated evaluation, and a doctor-facing dashboard for inspecting how the result was produced.

## 1.2 Key Terms and Definitions

The thesis uses several project-specific terms that are worth defining once up front. Generic terms (LLM, agent, multi-agent system, orchestrator, differential diagnosis, RAG, LLM-as-judge) are used in their standard meanings; the data-stack terms (FHIR R4, OMOP CDM, Bronze/Silver/Gold layers) are introduced where they are first needed in Chapter 3.

**Synthea.** The open-source generator used as the single source of patient data and hidden ground-truth disease labels.

**DIRECT / INDIRECT / MISS.** The three evaluation labels. DIRECT = the target disease is found clearly; INDIRECT = a clinically related condition is found (cause, consequence, precursor, or subtype); MISS = neither is found.

**Rank-1-when-found.** When the system finds the target or a related disease, how often it ranks first in the returned differential.

**Cohort leakage.** A risk in memory-based evaluation where the system looks better because it has already seen similar cases from the same test cohort. Controlled in this thesis by a paired 160-patient memory A/B in which patient identity is held constant across arms.

## 1.3 Research Gap and Problem Statement

The literature reviewed in Chapter 2 shows that multi-agent clinical systems are promising, but it also leaves a practical gap: few systems join data preparation, diagnostic reasoning, memory, review, treatment planning, evaluation, and clinician inspection into one reproducible pipeline. Four problems are especially important for this thesis.

The first problem is **end-to-end coordination**. Many studies evaluate one benchmark or one reasoning stage. A clinical decision support workflow has to do more than that. It must move from patient data preparation to EHR analysis, lab interpretation, differential diagnosis, review, refinement, evaluation, and treatment planning. This requires an execution graph that can run agents in the right order, allow parallel work where it is useful, and continue safely when an agent produces a partial or invalid output.

The second problem is **meaningful disagreement between agents**. A multi-agent system is weak if all agents simply confirm the first diagnosis, and prior work shows that structured conflict resolution improves diagnostic accuracy [15, 16]. The Reviewer-Refiner pattern adopted in this thesis is described in Chapter 3.

The third problem is **shared memory**. Many clinical agent systems treat each patient as a new case and do not test whether useful patterns can be reused across runs. This thesis implements multi-level memory and compares it against a memory-disabled setting; the cohort-leakage risk that any such comparison must control for is discussed in Section 2.3.4.

The fourth problem is **evaluation**. Medical multiple-choice benchmarks are useful, but they are not the same as generating a ranked differential diagnosis from a longitudinal patient record. The system must be judged on whether it identifies the target disease, whether it finds a clinically related condition, and where that condition appears in the ranked differential. This thesis therefore uses a separate LLM-as-Judge evaluator that compares the generated differential with the hidden Synthea disease label and assigns a DIRECT, INDIRECT, or MISS verdict.

The research problem can therefore be stated as follows:

How should a team of specialised AI agents be coordinated so that their combined clinical reasoning produces a diagnosis a physician could trust and audit, rather than a polished but unverifiable answer from a single black-box model?

## 1.4 Aim and Research Question

The aim of this thesis is to design, implement, and evaluate a multi-agent clinical decision support system for synthetic patient records. The system is intended to produce a ranked differential diagnosis, allow agent outputs to be reviewed and refined, use shared memory in a controlled way, ground treatment planning in guideline retrieval, and present the reasoning to a doctor through an inspection dashboard.

The main research question is:

When clinical reasoning is split across a small team of specialised AI agents that can review, critique, and remember each other’s work, can they reach a clinically plausible diagnosis — and does giving them memory of past patients improve, leave unchanged, or degrade diagnostic accuracy?

The corresponding hypothesis is that a coordinated team of specialised agents, with an explicit review step that can rewrite a flawed primary diagnosis without erasing the earlier reasoning, produces clinically plausible diagnoses more reliably than a single model alone; and that shared memory of past patients broadens the differential the system considers while not statistically improving top-diagnosis accuracy at the cohort size tested.

The detailed agent graph, shared-memory subsystem, data pipeline, and dashboard are unpacked in Chapter 3; the evaluation cohort and headline results are reported in Chapter 4.

## 1.5 Objectives

The objectives of the thesis are:

- **Build the clinical data pipeline.** Convert Synthea FHIR R4 records into Bronze, Silver, and Gold layers, with Gold EHR and laboratory case files for each selected patient.
- **Construct the evaluation cohort.** Filter a Synthea-generated patient pool with deterministic eligibility rules and an LLM-based detectability verifier to yield a 160-patient evaluation cohort across the implemented disease groups.
- **Implement the multi-agent workflow.** Build a LangGraph pipeline of seven specialised agents, including the Reviewer–Refiner step that challenges the primary diagnosis while preserving the earlier reasoning for audit.
- **Implement multi-level shared memory.** Store working state, episodic traces, semantic disease summaries, guideline knowledge, and case-based retrieval over prior patients.

- **Ground treatment planning in guidelines.** Retrieve NICE guideline material from Qdrant and run treatment planning only when the diagnostic evaluator returns a DIRECT match.
- **Evaluate free-text diagnoses.** Use a separate LLM judge with DIRECT, INDIRECT, and MISS labels, plus rank-1-when-found.
- **Measure the effect of memory.** Compare multi-level memory with a memory-disabled setting under the leakage-aware protocol introduced in Section 2.3.4.
- **Build the reviewer-facing doctor console.** A single React + FastAPI front end with two journeys: *Doctor (runtime)*, which streams a live patient run and presents the differential and NICE-grounded plan; and *Researcher (statistics)*, which exposes the cohort Overview, Memory A/B, Model, and MAS-vs-single-LLM tabs plus a per-patient drill-down.

## 1.6 Scope and Assumptions

This thesis has several limits that are important for reading the results.

**Synthetic data only.** All patient records come from Synthea. No real patient data or protected health information is used. This makes the work easier to reproduce, but it also means the results depend on the quality and limits of Synthea’s disease models.

**Decision support only.** The system produces differential diagnoses, evidence summaries, and treatment suggestions for human review. It is not designed to make autonomous clinical decisions.

**Not a regulated medical device.** The implementation is a research prototype. It has not been clinically validated, cleared by a regulator, or tested prospectively in patient care.

**Bounded clinical coverage.** The implemented cohort focuses on these target diagnoses and related disease groups: ischemic heart disease, chronic congestive heart failure, essential hypertension, type-2 diabetes mellitus, chronic kidney disease stage 2, chronic kidney disease stage 3, end-stage renal disease, and metabolic syndrome X. The results should not be read as evidence that the system works for diseases outside this set.

**Limited evaluation size.** The empirical results in Chapter 4 are reported on 160 Gold-layer pipeline runs. The smaller memory A/B studies are used to study memory behaviour, not to claim final clinical performance.

**English-only reasoning.** The prompts, retrieved guidelines, and dashboard text are in English. Non-English clinical text is outside the scope.

## 1.7 Thesis Outline

This thesis is divided into six chapters.

**Chapter 2: Background** reviews multi-agent LLM work in clinical AI. It groups the literature by topic, including foundation frameworks, coordination patterns, debate and review mechanisms, clinical deployments, safety, and evaluation. The chapter is used to justify the design choices made later in the system.

**Chapter 3: Methodology** explains how CMADS was built. It covers the Synthea data pipeline, Bronze/Silver/Gold data layers, Gold case construction, LangGraph workflow, agent blueprint, seven specialised agents, multi-level shared memory, NICE guideline retrieval, model configuration, evaluation method, and implementation stack.

**Chapter 4: Results** reports the empirical findings. It presents the 160-patient completed cohort, end-to-end diagnostic outcomes, per-disease results, per-agent reliability, adaptive reasoning behaviour, memory comparison, model comparison, literature comparison, qualitative case walkthroughs, and the doctor-facing dashboard.

**Chapter 5: Discussion** interprets the empirical results: how the headline outcomes fit the design intent, how the memory and orchestration comparisons should be read, the token and cost accounting, and a consolidated limitations list.

**Chapter 6: Conclusion and Future Work** summarises the thesis contributions, explains the limits of the current prototype, and outlines future steps such as real EHR testing, broader disease coverage, prospective clinical review, and larger memory-controlled experiments.

# Chapter 2

## Background

### 2.1 Concepts Overview

This chapter gives the background needed to understand the system built in this thesis. It first explains why a single LLM is not enough for clinical decision support, then introduces multi-agent systems, clinical LLM agents, retrieval, and workflow design. The second half of the chapter reviews the papers that shaped the implementation choices used later in Chapter 3.

#### 2.1.1 The Limitations of Single-Model AI in Clinical Settings

Single LLM systems have been used for diagnosis support, drug recommendation, and clinical documentation. They can produce useful medical text and often perform well on question-answering benchmarks [5]. However, a clinical workflow is harder than a benchmark. The model has to work with partial evidence, uncertainty, and information that may be spread across many parts of a patient record.

The first problem is missing knowledge. A model only has the knowledge learned during training unless it is connected to external data or tools. It cannot automatically query a patient database, check a current guideline, or calculate a clinical score. Tang et al. found that 77% of LLM medical-task errors were linked to domain knowledge gaps rather than pure reasoning failure [6]. This supports the idea of using specialist agents and retrieval, instead of asking one model to carry the whole task.

The second problem is single-pass reasoning. A normal LLM call produces one answer from one prompt. Clinical reasoning often needs several rounds: collect evidence, form a differential, check missing information, revise the answer, and only then decide. Shang et al. showed that dynamic multi-round decision-making performed better than single-pass approaches on complex patient cases [17].

The third problem is bias. LLMs can anchor on an early diagnosis, look mainly for evidence that supports it, or rely on familiar patterns. These are similar to known

diagnostic biases in human medicine. Ke et al. showed that multi-agent conversation with a critical evaluator reduced anchoring, availability, and confirmation bias compared with a single-agent setting [8].

### 2.1.2 Multi-Agent Systems: Principles and Architecture

A multi-agent system (MAS) divides a task between several agents. In clinical AI, each agent usually has a role, such as retrieving information, reading the EHR, interpreting labs, reasoning about diagnoses, or reviewing the answer [7]. The agents exchange structured messages and can use different prompts, tools, or knowledge sources.

The way agents are connected matters. In a centralised design, one orchestrator assigns work and combines the outputs. In a decentralised design, agents exchange messages more directly. In a hierarchical design, simple cases can be handled at a lower level and difficult or risky cases can be escalated. Chen et al. showed that topology affects safety: centralised systems were more vulnerable to a compromised agent, while decentralised systems were more resilient [18]. Kim et al. also used a tiered design in TAO to improve safety handling [19].

Specialisation is another important idea. A clinical MAS should not use several copies of the same general agent and expect them to add much value. The stronger pattern is to give agents different responsibilities, such as a Retrieval Agent, Reasoning Agent, Synthesis Agent, or Critical Evaluator. This pattern appears in many of the systems reviewed in this chapter [9, 10].

### 2.1.3 Large Language Models as Clinical Agents

In this thesis, an LLM agent is a model call wrapped in a defined role, a prompt, an input format, and an output schema. The LLM provides the language reasoning, but the surrounding workflow controls what evidence the model sees and how its answer is checked. This shape mirrors the ReAct pattern of Yao et al. [20], in which a language model interleaves explicit reasoning steps with actions (such as retrieval or tool calls); each agent in the pipeline produces reasoning, then a structured action — writing a finding, retrieving a guideline, or emitting a critique — whose effect is visible to downstream agents.

Different LLMs vary in size, training data, instruction following, and medical accuracy. The literature shows that the largest model is not always the best choice if the workflow is weak. Chen et al. reported that a Llama-3.1-70B multi-agent system outperformed GPT-4o on rare-disease diagnosis when the smaller model was supported by memory and tools [12]. This is important for this thesis because it shifts attention from only choosing a model to designing the workflow around the model.

Retrieval-Augmented Generation (RAG) is one way to reduce missing knowledge. With RAG, an agent receives retrieved documents, such as clinical guidelines, before it

writes its answer. Choi et al. showed that adding RAG to a multi-agent pharmacovigilance system reduced hallucinated drug-event associations by 64% [21]. This motivates the guideline retrieval used later by the Treatment Planning agent.

### 2.1.4 Automation Workflow Design in Clinical AI

Automation workflow design means breaking a clinical task into clear steps and deciding which agent or tool handles each step. It also defines how outputs move between agents and where quality checks are placed [22].

Most clinical MAS workflows follow a simple pattern: gather evidence, analyse it, then produce a final answer. Later systems extend this pattern with parallel agent work, adaptive loops, retrieval, review, and error recovery [6, 22, 23, 24]. Bedi et al. add an important warning: better individual agents do not guarantee a better system if the handoffs between agents are poor [14].

For this thesis, the workflow must handle four practical needs. It must divide the clinical task into agent roles, coordinate the order of execution, allow agents to challenge each other, and prevent one weak output from silently corrupting later stages. The literature review below explains how earlier work handled these needs and where gaps remain.

### 2.1.5 Clinical Reasoning and Decision-Making

The systems reviewed in this chapter all attempt to support a process that clinicians normally carry out themselves. A short primer on that process helps to read the rest of the chapter, especially for readers from a computing background.

A *differential diagnosis* is a ranked list of conditions that could plausibly explain a patient’s presentation. Clinicians build it by combining the patient’s history, current symptoms, vital signs, prior conditions, medications, and laboratory and imaging findings, and they update it as new information arrives. The ranking is not just a list of possibilities; each candidate carries an implicit probability and an implicit cost of missing it, and treatment decisions are made against the top-ranked candidate while remaining alert to the alternatives. A clinical decision support (CDS) system is any tool intended to help with this process, from rule-based alerts in an electronic health record through to the multi-agent LLM systems reviewed below.

Clinical reasoning is well documented to be cognitive-bias-prone. Three biases matter most in the literature reviewed here. *Anchoring* occurs when an early hypothesis disproportionately influences later reasoning: a clinician who suspects pulmonary embolism early may interpret a borderline D-dimer as supportive even when the broader history points elsewhere. *Availability bias* occurs when recently seen or memorable cases are weighted too heavily: a doctor who recently diagnosed a rare condition may over-suspect

it in the next patient. *Confirmation bias* is the tendency to look for evidence that supports an initial hypothesis and to discount evidence that contradicts it. Ke et al. show that LLM agents reproduce these biases when used as single models and that structured multi-agent configurations can attenuate them [8].

Two further properties of clinical reasoning shape the design of any CDS system. First, the evidence is *longitudinal*: laboratory trends over months matter, not just the most recent value. Second, the reasoning is *multi-source*: a single answer typically rests on data from the history, the examination, the chart, and any retrieved guideline or reference. A CDS system that processes only the current visit, or only one type of evidence, will miss precisely the cases where reasoning is hardest. The pipeline built in this thesis is designed to combine these sources explicitly, and the review below selects work that does the same.

## 2.2 Search Strategy

The literature search focused on recent multi-agent and agentic LLM work in clinical AI. Searches were conducted between January and May 2026 in arXiv, PubMed, ACL Anthology, IEEE Xplore, and Semantic Scholar. The main search terms were “multi-agent system AND clinical decision”, “LLM agent AND medical reasoning”, “automation workflow AND clinical AI”, “multi-agent AND electronic health records”, and “agent collaboration AND healthcare”. The five searches together returned approximately 410 candidate articles across the databases, with substantial overlap between sources.

The candidate set was filtered in three stages. First, duplicate records and non-research items (editorials, course notes, blog posts) were removed, reducing the set to about 180 distinct articles. Second, titles and abstracts were screened against the inclusion and exclusion criteria of Table 2.1; this left approximately 55 articles for full-text review. Third, full-text review removed papers that did not propose or evaluate a working multi-agent system, that focused only on medical image analysis without a clinical decision component, or that were review articles without an original empirical contribution. After this process, 22 papers were retained for the multi-agent clinical AI literature review and grouped into the five themes used below. The wider thesis bibliography additionally cites the CoALA cognitive-architecture taxonomy, the BioLORD embedding model [25] used by the case-based memory and retrieval layers, the epidemiology / EHR-workload sources cited in Chapter 1, and the AMIE and AgentClinic contemporary works discussed below as the closest published evaluation shapes to this thesis’s protocol; these sit outside the MAS review count and bring the total bibliography to 29 entries.

**Preprint reliance.** Approximately 40% of the 22 retained multi-agent clinical AI papers are arXiv preprints rather than peer-reviewed journal or conference publications. This reflects the velocity of the subfield: the relevant techniques (LangGraph orchestration, multi-agent debate patterns, clinical LLM benchmarks) were published in

2024–2025 and many results were still in preprint at the time of writing. Where peer-reviewed equivalents exist for foundational concepts — LLM-as-judge methodology [26], retrieval-augmented generation [27], Synthea [28], OMOP CDM [29], sentence-BERT [30], chain-of-thought prompting [31], self-consistency [32], ReAct [20], and the foundational framing of AI in medicine [4] — this thesis prefers the peer-reviewed source. Multi-agent clinical-AI claims that are still preprint-only are flagged in the text where they are first introduced (e.g. MAC Framework, MedSentry, MDAgents) so that the reader can weigh the strength of the underlying evidence appropriately.

A paper was included when it proposed or evaluated a multi-agent or agentic LLM system, applied it to a medical task, and reported either quantitative results or clinician-reviewed outcomes. Single-model systems, image-only systems without clinical decision support, and review papers without new experiments were excluded. Table 2.1 summarises the criteria.

Table 2.1: Inclusion and Exclusion Criteria for Selected Articles

| Criteria           | Inclusion                                                                                | Exclusion                                                                  |
|--------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Publication Period | 2024–2025 (2023 for foundational works)                                                  | Articles published before 2023                                             |
| System Type        | Multi-agent or agentic LLM systems for clinical/medical tasks                            | Single-model AI systems without agent coordination                         |
| Task Domain        | Clinical decision support, medical diagnosis, EHR analysis, clinical workflow automation | Medical image analysis without clinical decisioning component              |
| Evidence Type      | Quantitative metrics or clinician-validated outcomes with full methodological disclosure | Review articles or opinion papers without original empirical contributions |
| Language           | English-language publications                                                            | Non-English publications                                                   |

## 2.3 Literature Review

The 22 selected papers are grouped into five themes. The first group covers early multi-agent medical reasoning systems. The second group focuses on agent coordination and workflow design. The third group studies debate, disagreement, and conflict resolution between agents. The fourth group reviews systems built for specific clinical domains. The final group covers safety, reliability, and evaluation.

### 2.3.1 Foundational Multi-Agent Frameworks for Clinical AI

Four early systems shaped the field. They introduced role-based agents, discussion loops, final synthesis, and supervisor-style review. These ideas are reused in many later systems, including the system built in this thesis; per-paper rows for these four systems appear in the comparative summary in Section 2.4.3.

MedAgents by Tang et al. is one of the main starting points for this area [6]. It uses role-assigned medical agents that first analyse a query separately, then discuss it, and finally produce a shared answer. It was tested on nine medical benchmarks, including MedQA, MedMCQA, PubMedQA, and MMLU medical tasks. The system performed better than the baselines in a zero-shot setting, and the discussion step was the most useful part in the ablation study. Its error analysis is also important for this thesis: 77% of the reviewed LLM medical errors were linked to knowledge gaps. The main limitation is that the evaluation was still based on benchmark questions, not a full clinical workflow with patient records, memory, or treatment planning.

openCHA by Abbasian et al. adds a stronger connection to external data and tools [5]. It uses an interface layer, an orchestrator, and external sources. The orchestrator plans tasks, executes them, stores intermediate results, builds prompts, and generates the final response. The system was shown through two live demonstrations and four use cases, including dietary guidance and diabetes management. For this thesis, the useful part is the separation between planning and execution. However, openCHA does not focus on parallel clinical agents or structured disagreement between agents.

AI Hospital by Fan et al. studies what happens when agents interact over several turns [15]. It includes a Doctor agent, Patient agent, Examiner agent, and Chief Physician agent. The Patient reveals symptoms over time, while the Doctor asks questions and requests tests. The result showed that multi-turn diagnosis is much harder than answering with all information given at once. It also showed that removing dispute resolution lowered diagnostic accuracy. This supports the Reviewer-Refiner pattern used in this thesis.

The MAC framework by Shen et al. models diagnosis as a group consultation [11]. Doctor agents discuss, critique, and refine their answers under a Supervisor agent. On 302 rare-disease cases, the best setup was four Doctor agents and one Supervisor. Adding too many agents made the discussion redundant. MAC also performed better than the Self-Consistency baseline of Wang et al. [32] — which improves chain-of-thought reasoning by sampling multiple reasoning paths and majority-voting the answer — even though Self-Consistency used more model calls. This is useful evidence that organised discussion can be better than simply sampling many independent answers.

### 2.3.2 Agent Coordination Architectures and Workflow Orchestration

The first group of papers shows the value of multiple agents, but many of those systems use a fixed structure for every case. The next group focuses on how agents should be

coordinated — adaptive routing, escalation, workflow decomposition, and dynamic loops — and is summarised together with the rest of the reviewed literature in Section 2.4.3.

MDAgents by Kim et al. is important because it does not use the same agent setup for every query [7]. It first classifies the case as low, moderate, or high complexity. Low-complexity cases can be handled by one agent, while harder cases use a group or a multidisciplinary team. Across ten medical benchmarks, MDAgents achieved the best result on seven. The adaptive setup also performed better than fixed solo and fixed group settings by up to 4.2%, and a Moderator agent with retrieval improved group accuracy further. The trade-off was latency: high-complexity cases took much longer than low-complexity cases.

TAO, also by Kim et al., applies a similar idea to safety [19]. Instead of only routing by difficulty, it routes by risk. Tier 1 handles routine low-risk work, Tier 2 uses specialist agents for moderate cases, and Tier 3 is used for high-risk decisions. An Agent Recruiter chooses the starting tier, while an Agent Router can escalate a case when uncertainty appears. The result is useful for this thesis because it shows that orchestration can be designed around safety, not only accuracy.

Bedi et al. give one of the most relevant warnings for this thesis [14]. Their Optimisation Paradox study compared three systems on 2,400 real MIMIC-CDM patient cases: a single agent, a coordinated homogeneous multi-agent system, and a Best-of-Breed system that used the strongest individual component for each stage. The Best-of-Breed system performed worse, with 67.7% diagnostic accuracy and a 13.87% test-result hallucination rate. This shows that strong parts do not automatically make a strong system. The handoffs and coordination between agents can be more important than the isolated quality of each agent. Figure 2.1 shows the workflow used in that study.

Barra et al. provide a practical workflow example rather than only a benchmark study [22]. Their n8n pipeline uses seven agents and combines task decomposition, prompt chaining, parallel work, retrieval, and refinement. A Reviewer Agent checks the output against INACSL and ASPiH standards before an Editor Agent formats it. Across 50 runs, the pipeline reduced development time by 70–80%. The main idea used in this thesis is the separation between generation, review, and final formatting.

DynamiCare by Shang et al. focuses on multi-round clinical interaction [17]. Instead of giving the full case to the agents at the start, the system has to ask a Patient Agent for more information over time. This is closer to a real consultation, where the doctor gathers history, examination findings, and investigations step by step. The paper also introduced the MIMIC-Patient benchmark for dynamic agent-based clinical decision-making.

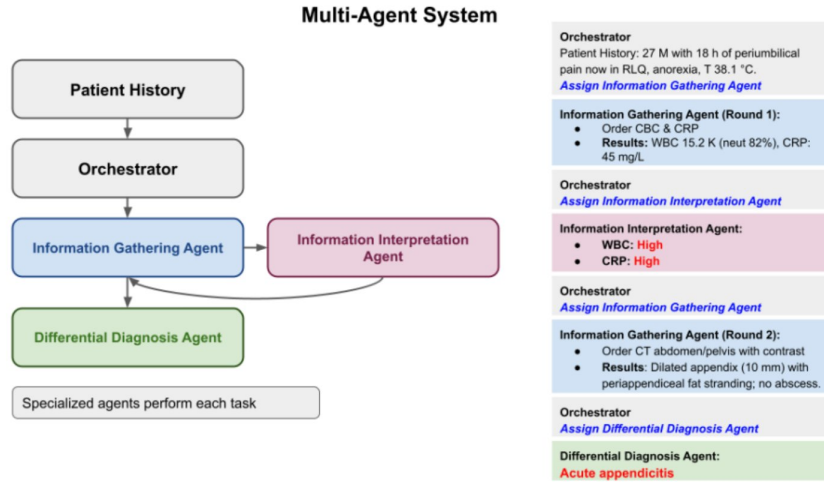


Figure 2.1: Multi-agent system (left) and example workflow (right) from Bedi et al. [14]. The figure shows an orchestrator coordinating specialised agents for information gathering, interpretation, and differential diagnosis in an acute appendicitis case. The study showed that a coordinated homogeneous system performed better than a Best-of-Breed system built from individually stronger components.

### 2.3.3 Inter-Agent Deliberation, Debate, and Conflict Resolution

Coordination decides which agents run and in what order. A separate question is what the system should do when agents disagree. The papers in this group study debate, dissent, and bias reduction; they matter because a clinical MAS should not simply accept the first answer without challenge. Per-paper rows are included in the comparative summary in Section 2.4.3.

MDTeamGPT by Chen et al. models a multidisciplinary team consultation [13]. The workflow has four phases: assign roles, let agents analyse independently, debate, and synthesise the final answer. On MedQA and MedMCQA, it performed better than single-agent and fixed-team baselines. The debate phase was the most useful part. The paper also found that performance was better when agents started with different initial diagnoses. This supports the idea that disagreement can be useful when it is structured.

Layered Debate by Zhao et al. focuses on diseases that look similar [33]. The first layer asks specialist agents to produce initial diagnoses. The second layer makes them cross-examine each other. The third layer uses a Judge Agent to read the debate and make the final decision. The system performed better than single-agent, standard multi-agent, and chain-of-thought [31] baselines. For this thesis, the useful idea is that review should examine the reasoning path, not only the final diagnosis.

Wang et al. identify a common failure called *silent agreement* [16]. This happens when agents appear to discuss a case but quickly agree with the first diagnosis without real criticism. Their Catfish Agent is designed to inject controlled dissent into the

discussion. It changes the strength of the disagreement based on case difficulty and keeps the tone collaborative enough to avoid harming performance. Across twelve benchmarks, it outperformed strong baselines, especially on difficult cases. This supports the need for a reviewer agent that can challenge the diagnostic agent.

Ke et al. connect disagreement to safety [8]. They tested whether multi-agent LLM conversations can reduce anchoring, availability, and confirmation bias. Their three-role setup used a primary decision agent, expert consultants, and a critical evaluator. Across ten clinical vignettes, the multi-agent setting produced fewer bias events and better diagnostic accuracy. The critical evaluator was the most useful part because it stopped the primary agent from staying with a wrong early diagnosis when later evidence contradicted it.

### 2.3.4 Memory and State in Multi-Agent Clinical Systems

The systems reviewed so far treat each clinical case as a fresh problem. Useful information from one case is not carried into the next: there is no record of which evidence patterns supported which conclusions, no record of which agents tended to disagree on which kinds of cases, and no record of how the system handled similar patients before. This is a known limitation of the foundational and coordination work and shapes the memory subsystem that the thesis builds in Chapter 3.

The conceptual taxonomy used for the memory design in this thesis is drawn from the Cognitive Architecture for Language Agents (CoALA) family of work. CoALA distinguishes four memory categories — *working*, *episodic*, *semantic*, and *procedural* [34] — and is a categorisation framework rather than an empirical claim that every tier adds value on every task. The thesis adopts the first three categories directly and substitutes *case-based* memory for the procedural tier: procedural memory in CoALA encodes reusable skills or routines, which has no clean analogue in a fixed seven-agent clinical pipeline whose orchestration is wired in code, whereas a case-based store of prior patients retrieved by clinical similarity has a direct analogue in how clinicians draw on past cases at the bedside. With that substitution made: *Working* memory holds the state of the current task — the prompt, the intermediate outputs, the scratchpad. *Episodic* memory is an append-only timeline of events from prior runs, useful for replaying what happened in a particular case. *Semantic* memory stores summarised, condition-level aggregates — for example, the typical evidence patterns that supported a confirmed match for a given disease across many runs. *Case-based* memory stores prior patients as searchable records that can be retrieved by similarity to the current patient. The thesis instantiates all four tiers in Section 3.6; a per-tier ablation that would isolate the contribution of each tier separately is identified as future work in Chapter 6.

Three papers from the broader review intersect this design. RareAgents combines persistent memory with tool use and shows that an open-source backbone with memory beats a larger general model on rare-disease diagnosis [12]; this is the closest published example

of case-based retrieval changing diagnostic behaviour. MDTeamGPT introduces a self-evolving memory that updates which discussion patterns work for which case types [13]; the mechanism is closer in spirit to semantic memory than to case-based memory. The MALADE pharmacovigilance pipeline is the most direct example of retrieval-augmented generation in a multi-agent clinical setting, and reports a 64% reduction in hallucinated drug–event associations when retrieval is added [21]; this result motivates the NICE-guideline retrieval used by the Treatment Planning agent in this thesis.

Most of these systems do not report how they prevent the case store from retrieving near-duplicate prior runs of the same cases used for evaluation. That risk can inflate apparent accuracy without genuine algorithm improvement. The memory comparison in Chapter 4 is therefore reported cautiously across all available runs for each memory setting, with the cohort-leakage risk stated explicitly instead of being hidden behind a single headline accuracy number.

### 2.3.5 Domain-Specific Clinical Workflow Deployment

The previous sections focused on general design ideas. The papers in this group look at systems built for specific clinical domains, including cardiology, inpatient pathways, pharmacovigilance, emergency triage, and rare-disease diagnosis. They are useful because they show how multi-agent ideas behave when applied to real or realistic clinical tasks; per-paper rows are included in the comparative summary in Section 2.4.3.

ZODIAC by Zhou et al. applies multi-agent design to cardiology [9]. It uses five agents for cardiac data extraction, arrhythmia detection, report generation, clinical reasoning, and human review. The agents were fine-tuned on real patient data reviewed by cardiologists. The system reached cardiologist-level performance on 7 of 8 metrics and outperformed general-purpose GPT-4 on cardiac tasks. It is also relevant because it includes human cardiologist review as part of the workflow.

MAP by Chen et al. studies inpatient pathway automation [35]. It covers admission assessment, daily monitoring, medication planning, procedure recommendation, and discharge summary generation. The authors also introduced MAP-Bench, built from real EHR data across four departments. Splitting the work by pathway stage improved workflow adherence by 23% over single-agent baselines. Department-specific agents also performed better than general agents.

ClinicalLab by Yan et al. is the broadest deployment in this review [10]. It covers 11 departments and 1,500 real patient cases. The system uses department-specific specialist agents, a Coordinator for routing, and a Chief Physician for final synthesis. It achieved diagnostic accuracy within 5% of senior physicians and received a clinician satisfaction score of 4.1/5. The Coordinator and Chief Physician roles are especially relevant to this thesis because they resemble the orchestrator and reviewer roles.

MALADE by Choi et al. applies a RAG-based multi-agent pipeline to pharmacovigilance [21]. The system improved adverse drug event identification precision by 18% and

recall by 12% compared with single-agent GPT-4. RAG reduced hallucinated drug-event associations by 64%. The important design point is that the orchestrator can reroute the task when retrieval quality is weak, instead of passing poor evidence to later stages.

CDR-Agent by Xiang et al. automates clinical decision rules such as the HEART Score, Wells Criteria, and Ottawa Rules [36]. Its Select–Extract–Calculate–Recommend pipeline reached 94.2% rule selection accuracy and 89.1% variable extraction F1 on 500 de-identified UCSF notes. UCSF clinicians rated 91% of recommendations as clinically acceptable or better. The system also correctly declined to apply a rule in 88% of out-of-scope cases, which is useful evidence for deferral as a safety behaviour.

TriageAgent by Lu et al. focuses on emergency triage [23]. It uses a five-stage pipeline with confidence scoring and early stopping. If agent confidence is high enough, the system stops further deliberation. It also uses RAG over the Emergency Severity Index handbook. On GPT-3.5-turbo, it outperformed MedAgents by 5.55%. This supports confidence-aware stopping in agent workflows.

RareAgents by Chen et al. addresses rare-disease diagnosis [12]. It uses Llama-3.1-70B with persistent memory, medical tool use, and five specialist MDT agents. On RareBench and MIMIC-IV-Ext-Rare, RareAgents outperformed GPT-4o and domain-specific fine-tuned models. Tool use also reduced hallucinated medication recommendations by 41%. This is important because it shows again that workflow, memory, and tools can matter more than model size alone.

### 2.3.6 Safety, Robustness, and Evaluation of Clinical MAS

The domain papers show promising accuracy, but accuracy alone is not enough for a clinical system. The system also has to be safe, reliable, and able to recover from errors. Only a small number of reviewed papers focus directly on these issues; the two most relevant appear as rows in the comparative summary in Section 2.4.3 and are discussed below.

MedSentry by Chen et al. studies adversarial safety in clinical multi-agent systems [18]. The authors tested centralised, decentralised, layered, and shared-pool topologies by adding one compromised agent and then running 5,000 adversarial medical prompts. The centralised topology was the most vulnerable because one compromised component could influence the whole system. The decentralised topology was the most resilient. The study also showed that simple safety filters could be bypassed, so agent isolation and prompt purification are needed. Figure 2.2 shows the defence results.

EHRFlow by Wu et al. focuses on operational reliability instead of adversarial attack [24]. It uses four agents: a Coder Agent that writes analysis code, an Executor Agent that runs it, a Debugger Agent that fixes errors, and a Summariser Agent that turns outputs into readable findings. It achieved higher task completion than single-agent systems on EHR analysis tasks such as cohort selection, lab trend analysis, and outcome prediction. The main lesson for this thesis is that failures should be handled inside the workflow before they propagate to later agents.

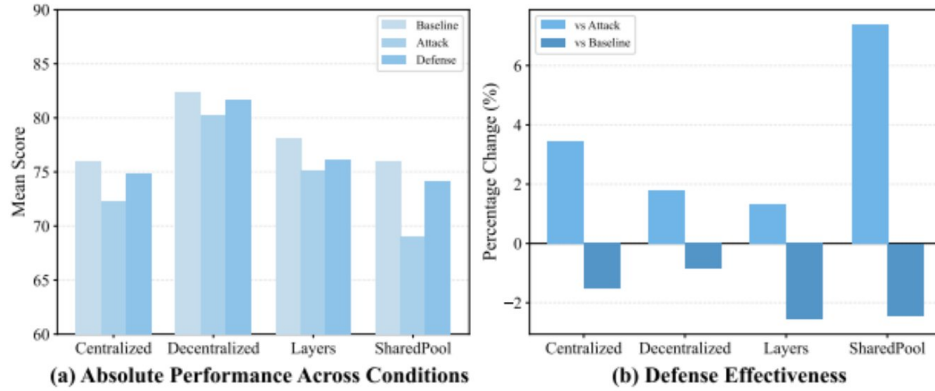


Figure 2.2: Multi-agent system defence evaluation from Chen et al. [18]. The decentralised topology shows the strongest resilience, while the centralised topology is most vulnerable to adversarial manipulation.

## 2.4 Analysis and Discussion

The reviewed papers point to a small number of common lessons. This final section summarises those lessons, explains the gaps that remain, and gives a comparative table of the 22 papers.

### 2.4.1 Key Findings

Four findings appear repeatedly in the reviewed papers. These findings explain why this thesis uses a multi-agent workflow with review, memory, and explicit evaluation.

#### Multi-Agent Systems Usually Perform Better Than Single Models

Most reviewed papers report that multi-agent systems outperform single-model baselines on their tested tasks. This appears across medical reasoning benchmarks [6], clinical diagnosis [7, 9, 10], pharmacovigilance [21], emergency triage [23], rare-disease diagnosis [12], and inpatient pathway management [35]. This does not mean every multi-agent design is good, but it does show that splitting clinical work across roles can be useful.

#### Coordination Is as Important as Model Choice

Bedi et al. show that choosing the best individual component for each stage does not guarantee the best final system [14]. Chen et al. and Wang et al. support the same point from safety and debate perspectives [18, 16]. The practical lesson is that model choice matters, but agent handoffs, routing, and review structure matter as much or more.

### Conflict Resolution Must Be Designed

Clinical agents should not automatically agree with each other. Fan et al. found that removing dispute resolution reduced diagnostic accuracy [15]. Wang et al. describe silent agreement, where agents appear to collaborate but do not really challenge the first answer [16]. Zhao et al. and Chen et al. show that structured debate can improve difficult diagnostic tasks [33, 13].

### Safety Depends on the Workflow

Safety is not only a property of the base model. MedSentry shows that the communication topology changes how vulnerable the system is to a compromised agent [18]. TAO shows that a workflow designed around risk and escalation can improve safety outcomes [19]. This supports the use of clear stage boundaries and review points in the thesis system.

**Closest contemporary evaluation shapes.** Two recent works deserve mention here because their evaluation protocols are the closest published comparators to the present thesis. **AMIE** [37] is a conversational diagnostic system evaluated through OSCE-style consultations against primary-care physicians, reporting differential-diagnosis accuracy and history-taking quality across patient-actor scenarios. AMIE’s evaluation maps onto this thesis’s DIRECT and Found metrics; the present work differs in running on full Synthea-derived EHR records rather than scripted OSCE encounters, and in evaluating an end-to-end multi-agent pipeline with persistent memory and treatment planning rather than a single-agent conversational model. **AgentClinic** [38] is a multimodal benchmark of LLM agents in simulated clinical environments, reporting diagnostic accuracy across several LLM backbones with a clinician baseline. AgentClinic provides a structured benchmark rather than a deployed pipeline; the present work is closer to a deployment shape, running the same model under controlled patient identity and reporting paired controlled comparisons on both memory and orchestration A/B contrasts. Together AMIE and AgentClinic are the closest published evaluation shapes to the protocol used in Chapter 4; neither ships the combination of end-to-end EHR ingestion, non-destructive Reviewer-Refiner, CoALA-style four-tier memory, and a doctor-facing inspection interface that this thesis evaluates as one system.

**Synthea as a data substrate for diagnostic AI.** The data source itself deserves a brief comment. The most directly comparable contemporary work that evaluates a diagnostic AI system on Synthea-generated patients is the neural-network telemedicine study of Rahman and Emdad [39], which generates 100,000 synthetic encounters with Synthea and trains a family of tabular classifiers (MLP, deep MLP, TabNet, and a custom PyTorch network) to detect metabolic, oncology, and allergy/immunology conditions; their reported F1 drops to 0.581 when telemedicine context is limited. The two designs operate at opposite ends of the design space: their work treats Synthea as a large

statistical training cohort for tabular classifiers under feature drop-out, while the present work treats a smaller, deeply-annotated Synthea cohort as a substrate for multi-agent LLM differential diagnosis with persistent memory and treatment planning. To the best of the literature search reported in Section 2.2, no prior multi-agent LLM diagnostic pipeline has been evaluated end-to-end on Synthea-derived patient records; the present thesis is the first to do so in this configuration.

### 2.4.2 Research Gaps Identified

The reviewed papers leave four gaps that are relevant to this thesis. First, the Optimisation Paradox is clearly identified, but there is still a need to design workflows that handle this problem in practice [14]. Second, many systems are still evaluated on structured benchmarks rather than full patient workflows, although ClinicalLab and ZODIAC move closer to real deployment [10, 9]. Third, conflict resolution is often missing, narrow, or tested only on a special task [16, 33]. Fourth, safety and reliability are still under-studied; only a few reviewed papers focus on them directly [19, 24, 18]. A fifth gap, identified in Section 2.3.4, is that memory-augmented clinical multi-agent systems are not currently evaluated under a protocol that separates the cohort used to populate the memory store from the cohort used to test the with-memory configuration, so apparent gains can be confounded with same-cohort contamination.

Chapter 3 responds to each of these gaps directly. The end-to-end coordination problem is addressed by a LangGraph **StateGraph** with six clinical stages plus one memory-consolidation stage that wires the full pipeline from EHR analysis through treatment planning and Stage 7 memory consolidation into a single typed graph (Section 3.3). The conflict-resolution gap is addressed by the non-destructive Reviewer-Refiner pattern that lets the Clinical Reviewer challenge the Diagnostic Reasoning agent while preserving the original output for audit (Section 3.5.5). The memory gap is addressed by the four-tier shared-memory subsystem of Section 3.6, which instantiates the CoALA taxonomy of Section 2.3.4. The evaluation gap is addressed by the LLM-as-judge framework of Section 3.10, which compares free-text differentials against hidden Synthea ground truth on a three-label scale. The leakage-control gap is addressed more cautiously in Chapter 4 by reporting the memory comparison across all available runs for each memory setting and making the cohort-leakage risk explicit. This does not fully solve leakage, but it prevents the thesis from claiming that memory improves the system when the observed pattern is instead broader differentials and fewer misses. The remaining gap — safety and reliability of clinical MAS under adversarial conditions — is acknowledged but is outside the scope of this thesis and is returned to as future work in Chapter 6.

### 2.4.3 Comparative Summary

The 22 reviewed papers are best read thematically rather than as a row-wise matrix. Five themes recur across the corpus, and each is the precedent for a specific architectural choice taken later in this thesis.

**Knowledge pooling and role decomposition.** MedAgents [6], openCHA [5], and the npj Digital Medicine MAC framework [11] establish that role-decomposed pipelines (gather–discuss–decide; planner / executor / responder; proposal–critique–refine) outperform single LLMs on medical reasoning, and that 77% of single-LLM errors are knowledge gaps closable by structured collaboration — the direct justification for the seven-agent decomposition in Chapter 3.

**Adaptive complexity routing.** MDAgents [7], TAO [19], and DynamiCare [17] show that routing easy cases to a short loop and hard cases to a longer deliberative loop beats a fixed pipeline, and that hierarchical escalation (junior → senior → specialist) maps cleanly onto clinical practice. This is the orchestration precedent behind the adaptive diagnostic rounds and the Reviewer–Refiner pattern.

**Non-destructive deliberation and dissent.** AI Hospital [15], MDTeamGPT [13], the layered-debate framework [33], the Catfish Agent [16], and the cognitive-bias study by Ke et al. [8] converge on a single point: structured disagreement (judge agents, dissent agents, critical evaluators) reduces anchoring and confirmation bias and beats silent consensus. This is the precedent for the Reviewer-then-Refiner pattern that preserves both critique and original output for audit.

**Memory, retrieval, and grounding.** RareAgents [12], MALADE [21], CDR-Agent [36], TriageAgent [23], and the Optimisation Paradox study [14] demonstrate that memory and retrieval-augmented grounding (longitudinal patient modelling, RAG over guidelines, confidence-aware deferral) are the dominant drivers of factual accuracy and hallucination reduction (up to −64%). These papers motivate both the four-tier shared-memory subsystem and the NICE-guideline-grounded Treatment Planning stage.

**Real-world validation, safety, and topology.** ZODIAC [9], ClinicalLab [10], MAP [35], EHRFlow [24], the Barra et al. workflow study [22], and MedSentry [18] represent the strongest real-data evidence in the corpus: cardiologist-level performance on real patient data, full admission-to-discharge pipelines, and the finding that decentralised topologies survive adversarial injection better than centralised ones. These papers anchor the discussion of clinical validity and safety in Chapter 5.

The “22” here refers to papers retained by the multi-agent-system literature review process of Section 2.2; CoALA, BioLORD, and the epidemiology and workload sources cited elsewhere in the thesis sit outside that count and bring the total bibliography to 29 entries.



# Chapter 3

## Methodology

### 3.1 System Architecture Overview

This chapter explains how the Clinical Multi-Agent Decisioning System (CMADS) was built. The system is first shown as a set of simple black boxes, then each box is opened in the same order. Figure 3.1 shows the top-level flow: Synthea patient generation, case filtering, Gold case construction, the LangGraph multi-agent system, and the doctor-facing dashboard. Inside the multi-agent system are the common agent blueprint, shared memory, NICE guideline retrieval, and saved run outputs.

**Reproducibility.** All source code, agent prompts, and saved run artefacts are released as the open-source CMADS implementation. Throughout this chapter and Chapter 4, components are referred to by name (e.g. “the detectability verifier”, “the persisted execution trace”) rather than by file path; the exact locations live in the repository’s `README.md` for readers who want to inspect or re-run a step. Pinned versions for the empirical results reported in Chapter 4: LangGraph 0.6.x, `dbt-core` 1.8.x, Synthea release 3.4, BioLORD-2023 (HuggingFace `FreemyCompany/BioLORD-2023`), Qdrant 1.13.x.

The rest of the chapter follows the same order. Section 3.2 explains data generation, filtering, and Gold case construction. Section 3.3 describes the LangGraph workflow. Section 3.4 gives the common agent pattern, and Section 3.5 describes each agent. Section 3.6 then explains shared memory. The final sections cover NICE retrieval, model configuration, evaluation, deployment, and the end-to-end algorithm.

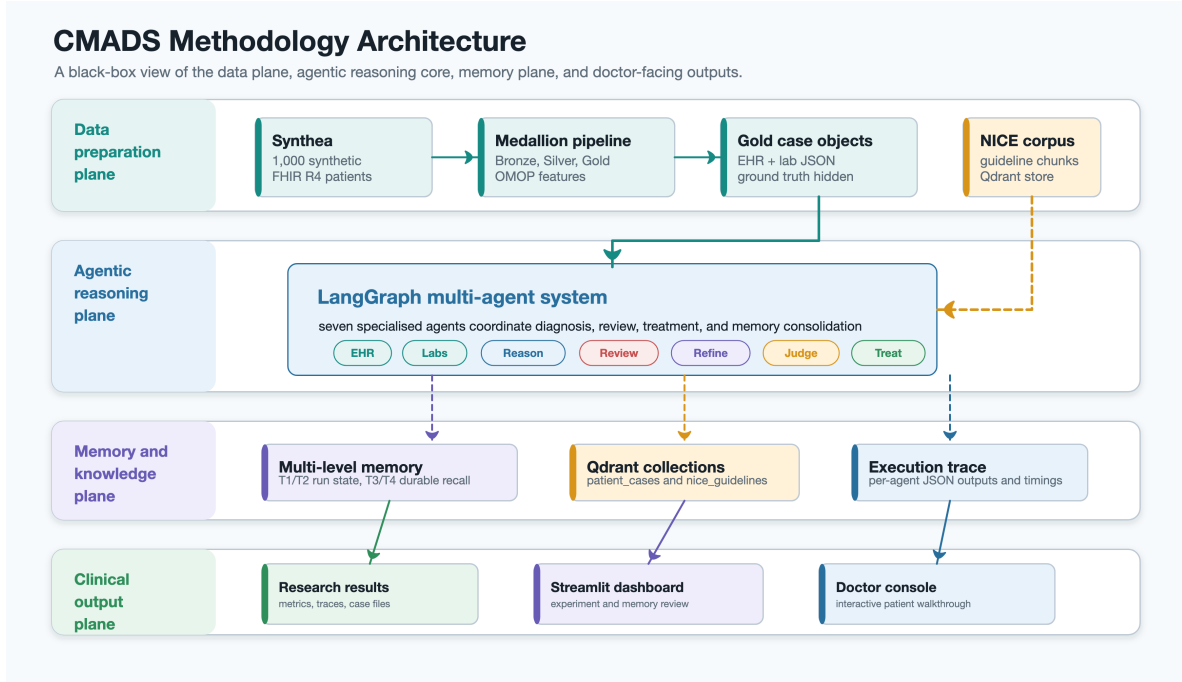


Figure 3.1: CMADS technical data-flow diagram. Solid arrows mark patient-data flow from Synthea through the Bronze/Silver/Gold medallion pipeline into the seven-agent reasoning system; dashed arrows mark control and side-output flows into the per-patient JSON store and the doctor console (which serves both the Doctor runtime and Researcher statistics journeys from a single React + FastAPI application). The NICE corpus is queried only during Stage 6 treatment planning. The reasoning system is shown collapsed here and is expanded as an explicit LangGraph in Figure 3.3.

## 3.2 Synthetic Data Generation and Case Filtering

This section explains the first three boxes in Figure 3.1: Synthea generation, deterministic case filtering, and Gold case construction. Figure 3.2 summarises how raw records become the JSON files passed to the agents.

### 3.2.1 Synthea Generation

The patient records are produced by *Synthea*, an open-source rule-based patient simulator from MITRE [28]. Synthea uses state-machine disease models informed by public health data and exports records in FHIR R4 format. It was chosen for two reasons. First, it gives a ground-truth label for the conditions it creates, which is needed for the LLM-as-Judge evaluation in Section 3.10. Second, the data is fully synthetic, so the project can be reproduced without using protected patient information.

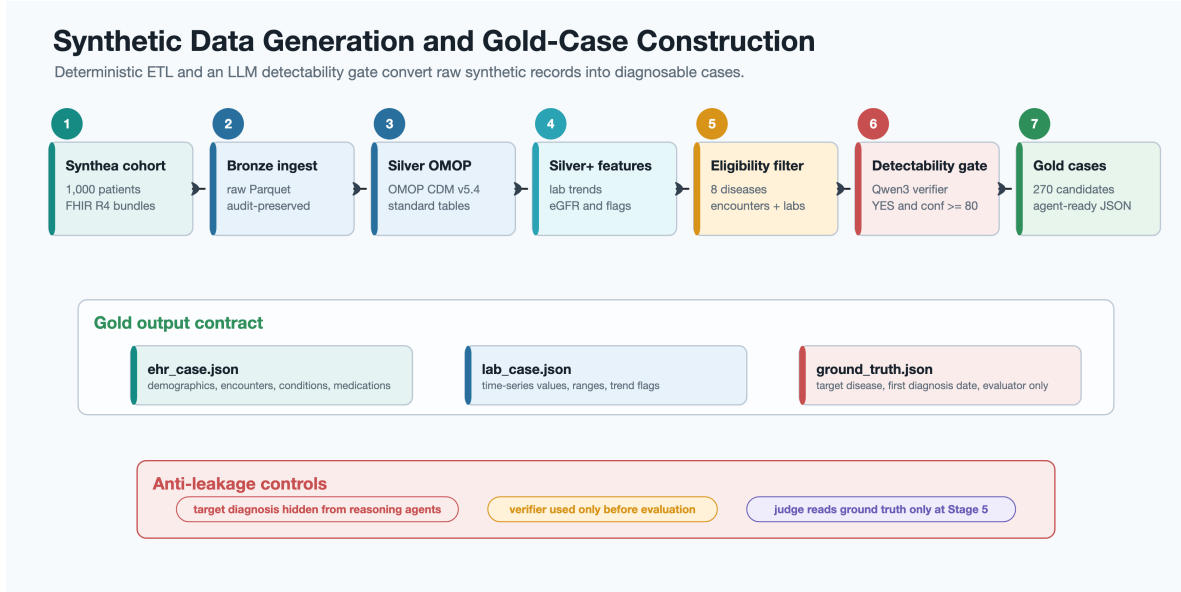


Figure 3.2: Synthetic data generation and filtering pipeline. Synthea produces raw FHIR R4 patient records; the medallion pipeline ingests and transforms them into OMOP-derived features; deterministic eligibility rules select the target cohort; and the Gold layer emits the EHR, laboratory, and hidden ground-truth JSON files consumed by the multi-agent system.

### 3.2.2 Target-Cohort Filtering

A cohort of 1,000 patients was generated by Synthea. A deterministic filter then selected patients from the target disease groups: ischemic heart disease, congestive heart failure, essential hypertension, diabetes mellitus type 2, chronic kidney disease stages 2 and 3, end-stage renal disease, and metabolic syndrome X. A patient was kept only if there were at least three encounters and five laboratory observations before the target diagnosis date. This gives the agents enough earlier evidence to reason from while keeping the target diagnosis itself hidden. From the 1,000 patients carried forward into the verification stage, the LLM-based detectability gate (Section 3.2.3) returned **VERDICT = YES** for 428 cases and a confidence score from 0 to 100 for each; the  $\geq 80$  confidence threshold restricts the verified cohort to **270 patients**, persisted as `data/gold/cohort_verified.json`. The principal paired evaluation reported in Chapter 4 draws a balanced 160-patient subset from this verified cohort to support the paired multi-level memory A/B; the remaining patients stay available for the detectability sensitivity analysis and the single-LLM controlled baseline reported in Sections 4.4.1 and 4.11 respectively.

**Scope of the eight families.** The eight disease families above are the scope of the evaluation *cohort*, not of the system. The pipeline is disease-agnostic by design: the agents read free-text chart and lab content, emit free-text differentials, and the LLM judge scores any candidate phrasing against any ground-truth diagnosis. The eight families

were chosen because Synthea’s generation module produces longitudinal records with consistent pre-diagnosis signal for these cardiometabolic conditions; rarer presentations (e.g. malignancies, autoimmune diseases, paediatric and obstetric cases) either do not appear in Synthea or appear without enough pre-diagnosis evidence for the detectability gate to admit them. Extending the cohort to a new disease family is a data-availability question — supply Gold-layer patients carrying that family’s typical evidence pattern — not a re-architecture: the agents, prompts, memory subsystem, and judge require no changes. The remaining patients were saved for audit but not run through the agent pipeline; a subset of the immediately rejected band (confidence 60–79) is used in the detectability-gate sensitivity analysis reported in Section 4.4.1.

### 3.2.3 LLM-Based Detectability Verifier

The encounter and lab-count filters are useful but not sufficient. Synthea can label a patient with a disease even when the pre-diagnosis record contains weak evidence: an end-stage renal disease patient may have normal creatinine, or a diabetes patient may have glucose values inside the reference range. The label is still valid ground truth, but the case may not be diagnosable from the data the agents are allowed to see.

A second LLM-based gate is therefore applied before a patient enters the multi-agent pipeline. The verifier (`pipeline/lab_verifier_llm.py`) builds a short clinical summary — demographics, conditions, medications, risk scores, recent labs, critical flags, vital signs — and asks the four questions in Table 3.1 using the Synthea target disease as the reference. It returns four fields: **VERDICT** (YES or NO), **CONFIDENCE** from 0 to 100, an **EVIDENCE** list, and a short **REASONING** sentence. Only patients with **VERDICT** = YES and **CONFIDENCE**  $\geq 80$  are admitted; rejected patients are saved for audit but are not run through the agent pipeline.

The verifier and the LLM-as-Judge evaluator (Section 3.10) both default to Qwen3-32B via `get_evaluator_llm()`; the reasoning agents run on a separate model (GPT-OSS-120B). The detectability-gate sensitivity analysis in Section 4.4.1 measures how much of the headline DIRECT rate depends on this gate.

Table 3.1: Detectability-verifier prompt structure.

| Verifier question       | What the LLM is asked to check                                                                                                          |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Q1 – Lab consistency    | Are any lab values abnormal in a way consistent with the target disease (e.g. raised HbA1c for diabetes, elevated creatinine for ESRD)? |
| Q2 – Medication signal  | Do any active medications suggest the target disease (e.g. furosemide for CHF, metformin for diabetes)?                                 |
| Q3 – Condition / risk   | Do any other recorded conditions or risk factors strongly point to the target disease?                                                  |
| Q4 – Clinical suspicion | Could a competent physician suspect the target disease from this data alone?                                                            |

This two-stage filter (deterministic eligibility + LLM detectability) produces the 160-patient cohort used throughout Chapter 4. The verifier is a data-quality gate only; it does not see any agent output and does not judge the system’s diagnosis.

### 3.2.4 Gold Case Construction

The raw FHIR output passes through a four-layer medallion data lake (also visible in Figure 3.2, steps 2–7). Each layer is responsible for a single transformation, which keeps the pipeline auditable and lets each step be re-run independently.

**Bronze.** Stores the raw FHIR resources as Parquet without modification. This layer exists so that downstream changes can always be re-derived from the original Synthea output without re-running the generator.

**Silver.** Converts the FHIR resources to OMOP Common Data Model v5.4 [40] using `dbt-core` on top of DuckDB. The OMOP CDM is the canonical schema curated by the OHDSI collaborative [29] for observational health research; it normalises codes (RxNorm for drugs, SNOMED for conditions, and LOINC for labs), so the rest of the pipeline operates on a stable vocabulary regardless of source format.

**Silver<sup>+</sup>.** Adds clinically derived features on top of OMOP: lab trends (increasing, decreasing, stable), estimated glomerular filtration rate from creatinine, BP categorical labels, glycemic-control classes, and a Charlson comorbidity index. These features are precomputed because every reasoning agent uses them, and computing them at agent call time would inflate latency.

**Gold.** Assembles three JSON files per patient: `ehr_case.json`, `lab_case.json`, and `ground_truth.json`. The first two are what the agents see; the third is reserved for the LLM Evaluator (Section 3.10). Table 3.2 summarises the contents and consumers of each file.

**Point-in-time construction.** A single methodology decision shapes every Gold file: *all data shown to the agents must predate the target disease’s first diagnosis date*. The cutoff is the `condition_start_date` of the target disease, and any condition, medication, visit, imaging study, or lab dated on or after that cutoff is excluded from `ehr_case.json` and `lab_case.json`. Without this rule the agents would have the answer in their input and the task would degenerate into a lookup; with it, diagnosis is a genuine inference task on the same kind of incomplete information a clinician sees in practice.

**Target disease selection.** A single patient may carry many synthetic conditions; the Gold assembler picks the most clinically significant one as the target. Diseases are ranked into four priority tiers (priority 1: heart failure, neoplasms, ischemic heart disease, ESRD; priority 2: diabetes, CKD stage 3, aortic valve disease; priority 3: epilepsy, other CKD, metabolic syndrome; priority 4: everything else). Within a tier the most recent diagnosis is selected, which maximises the length of the pre-diagnosis history available to the agents.

**Leakage prevention.** The target disease itself is excluded from the patient’s condition list using its SNOMED code, and a post-build verification confirms that the target disease name does not appear anywhere in any `ehr_case.json`. Comorbidity flags that name the target disease (`has_chf` for a heart-failure target, etc.) are masked the same way.

Table 3.2: Per-patient Gold output files: contents and consumers.

| File                           | Contents                                                                                                                                                                                                                                    | Consumed by           |             |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|-------------|
| <code>ehr_case.json</code>     | Demographics; active and resolved conditions before cutoff (target excluded); active medications with drug-condition links; visit summary by class; risk scores (BP, glycemic, Charlson); comorbidity flags; imaging studies before cutoff. | EHR<br>(Stage 1)      | Analyst     |
| <code>lab_case.json</code>     | Latest lab values before cutoff; lab trends with min/max/median and direction; critical lab flags with severity levels; recent vitals (BP, HR, BMI).                                                                                        | Lab<br>(Stage 1)      | Interpreter |
| <code>ground_truth.json</code> | Target disease name and SNOMED code; diagnosis date; cutoff date; all conditions at time of diagnosis; post-diagnosis medications; labs at cutoff.                                                                                          | LLM<br>(Stage 5) only | Evaluator   |

**What the system reads per patient.** Every patient case is structured — the agents never see free-text notes, images, or identifiers. The case file splits into two groups. The **chart-as-recorded** fields, ingested verbatim from Synthea / FHIR, are demographics (age, sex, race, ethnicity), the open problem list with SNOMED codes and onset dates (target excluded), the active medication list with class, dose, and start date, the most recent vitals (BP, HR, weight, SpO<sub>2</sub>) before the cutoff date, and the latest values for the key analytes (eGFR, HbA1c, lipid panel, electrolytes, troponin, etc.) with units and dates. The **derived** features, computed during Silver+ / Gold-layer assembly so the diagnostic agents do not re-derive them at inference time, are critical flags (e.g. `eGFR < 60` → CKD; `HbA1c > 6.5%` → diabetes), drug-condition links anchoring each medication to a plausible indication, Boolean comorbidity flags (`has_dm`, `has_htn`, `has_ckd`, `has_ihd`, `has_chf`, `has_obesity`), and standard risk scores where the data permits (ASCVD 10-year, CHA<sub>2</sub>DS<sub>2</sub>-VASc, eGFR CKD-EPI). The same categories are surfaced verbatim in the doctor console’s *Patient Overview* panel so a clinician can audit exactly what the agents read for any given consultation.

### 3.3 Multi-Agent System

The multi-agent pipeline is run with *LangGraph* [41]. Each agent is a node in a *StateGraph*, and edges define the order of execution. A shared *PipelineState* dictionary moves through the graph and carries patient data, agent outputs, traces, and memory fields. *LangGraph* was chosen because it supports parallel fan-out and fan-in, checkpointing after agent steps, and reducer-based state updates. These reducers are important because Stage 1 runs two agents in parallel, so one agent must not overwrite the other agent’s output.

Figure 3.3 opens the Multi-Agent System box from Figure 3.1. The graph has six clinical stages plus one memory-consolidation stage. Stage 1 runs the EHR Analyst and Lab Interpreter in parallel. Stage 2 performs diagnostic reasoning. Stages 3 and 4 review and refine the diagnosis. Stage 5 evaluates it. Stage 6 plans treatment only when the diagnosis is a DIRECT match. Stage 7 consolidates the run into the cross-session memory tiers.

The pipeline comprises six clinical stages plus one memory-consolidation stage that folds each run’s outcome into the cross-session memory tiers (Section 3.6.3). The seven stages are summarised below.

- **Stage 1 — Parallel context extraction:** the EHR Analyst and Lab Interpreter run concurrently, each consuming its Gold case file and producing a structured clinical summary.
- **Stage 2 — Diagnostic reasoning:** an adaptive loop generates a ranked differential, self-critiques it, and re-ranks for up to three rounds or until a confidence threshold of 75% is reached. The Pydantic schema (*DiagnosticOutput*) requires the differential to be non-empty; the prompt requests five candidates and the reasoning agent typically returns five, but no schema-level lower bound above one is enforced.
- **Stage 3 — Clinical review:** an adversarial reviewer agent independently re-examines the evidence and issues per-diagnosis verification flags.
- **Stage 4 — Diagnostic refinement:** the outputs of Stages 2 and 3 are merged into a single final differential.
- **Stage 5 — Evaluation:** the final differential is compared against the Synthea ground truth by an LLM-as-Judge evaluator that assigns DIRECT, INDIRECT, or MISS classifications.
- **Stage 6 — Treatment planning:** the treatment node is invoked after evaluation. For DIRECT matches it retrieves relevant NICE guideline passages from a Qdrant vector store and assembles a guideline-grounded treatment plan; for INDIRECT or MISS outcomes it records a structured skip.

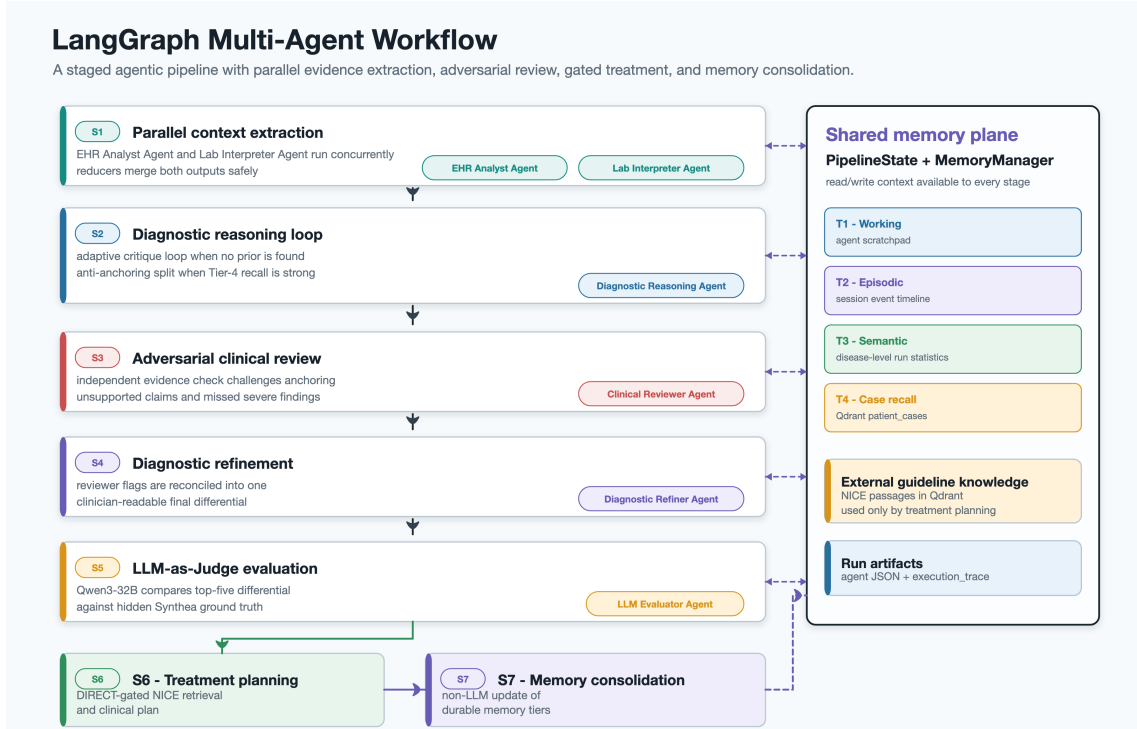


Figure 3.3: Multi-agent system execution order. Each stage card lists the inference back-end on the right and the agents executing inside the stage on the bottom. Solid arrows between stages name the output that flows downstream. The black box on the right represents the shared memory — a single `PipelineState TypedDict` backed by four memory tiers (working, episodic, semantic, case-based); every stage reads from and writes to it through dashed bidirectional arrows, as detailed in Section 3.6. Stage 6 is invoked in every run but the Treatment Planning agent emits a clinical plan only for DIRECT evaluations; the Memory Consolidator runs unconditionally and writes Tier 2–3 plus, when the run is informative enough, Tier 4.

- **Stage 7 — Memory consolidation:** a non-LLM maintenance node folds the run’s outcome into the cross-session semantic store (Tier 3) and eligible cases into the case-based vector store (Tier 4); see Section 3.6.3.

## 3.4 Agent Blueprint

Every agent follows the same internal pattern, shown in Figure 3.4. The pattern is implemented as a Python base class. Each subclass supplies its own prompt, schema, and post-processing logic. This keeps the pipeline easier to debug because every agent uses the same retry rules, trace format, validation steps, and failure handling.

The five stages of the blueprint are as follows.

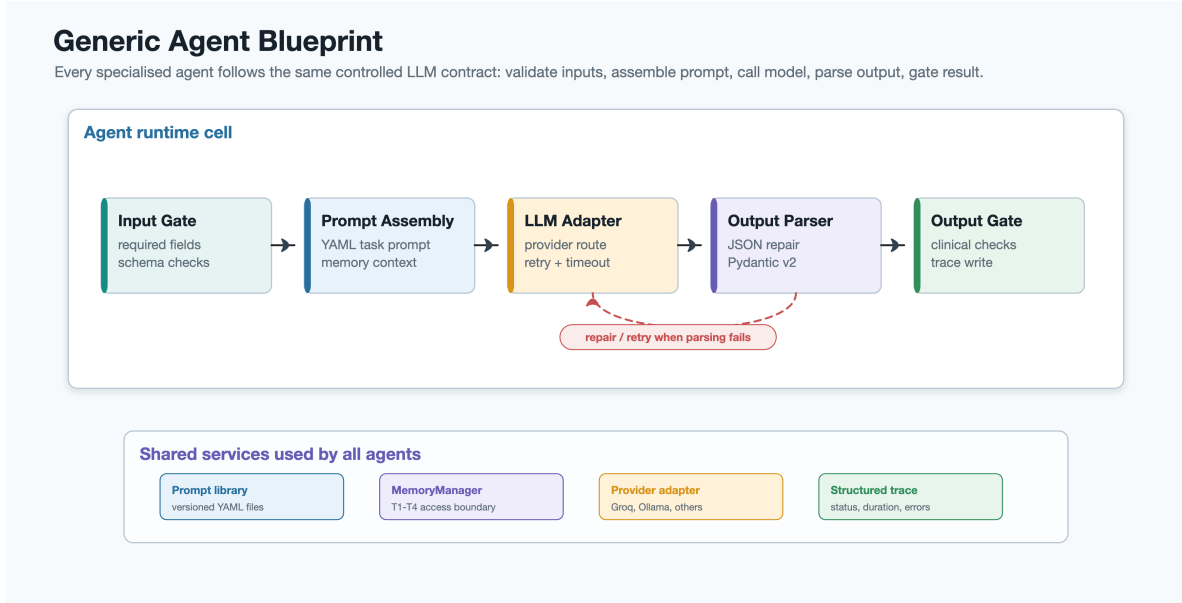


Figure 3.4: Generic agent blueprint. Every agent follows the same path: Input Gate → Prompt Assembler → LLM Invocation → Output Parser → Output Gate. The Input Gate reads upstream data, the Prompt Assembler adds any relevant memory, the LLM returns the raw answer, and the Output Gate writes the validated result. Memory access goes through one **MemoryManager** interface.

1. The **Input Gate** checks that required upstream data exists and matches its Pydantic schema. If validation fails, the agent records a structured error in `execution_trace` and stops cleanly.
2. The **Prompt Assembler** constructs the LLM prompt from a versioned YAML template, substituting the validated upstream outputs. Keeping prompts in YAML instead of hard-coded Python makes prompt changes easier to track.
3. The **LLM Invocation** layer dispatches the prompt to the configured language model through the provider-agnostic adapter (Section 3.8). Every call is wrapped in an exponential back-off retry policy (up to three attempts); a slow-call warning is logged when an individual call exceeds the configured threshold, and the outer per-agent budget enforces termination at the agent level.
4. The **Output Parser** converts the raw LLM response into a Pydantic v2 object and applies JSON repair rules for common formatting failures.
5. The **Output Gate** applies final validation before writing to shared memory. For example, the Diagnostic Reasoning gate requires a non-empty differential and is paired with a prompt that instructs the agent to return at least three candidates — the Pydantic schema does not enforce a minimum above one, but the agent’s prompt and the downstream Reviewer-Refiner stages assume a populated top-five list. The Treatment Planning gate is, at the level of code, a Pydantic schema

validator: it enforces that a structured plan is returned and that downstream fields parse correctly. The grounding requirement — that medications and procedures cite the retrieved NICE passages — is enforced prompt-side by the Treatment Planning prompt, not by a post-hoc retrieval-support check; a programmatic support check is identified as future work in Chapter 6.

## 3.5 Agent Specifications

This section describes the seven agents. For each agent, the important points are its inputs, outputs, and special behaviour. Table 3.3 summarises the agents and their shared-memory dependencies.

Table 3.3: The seven specialised agents and their shared-memory dependencies.

| Agent                | Reads                          | Writes               |
|----------------------|--------------------------------|----------------------|
| EHR Analyst          | ehr_case                       | ehr_analyst          |
| Lab Interpreter      | lab_case                       | lab_interpreter      |
| Diagnostic Reasoning | EHR + Lab outputs              | diagnostic_reasoning |
| Clinical Reviewer    | Stages 1 & 2 outputs           | clinical_reviewer    |
| Diagnostic Refiner   | Diagnostic + Reviewer          | final_diagnosis      |
| LLM Evaluator        | Final diagnosis + ground truth | evaluation           |
| Treatment Planning   | Final diagnosis + evaluation   | treatment_planning   |

### 3.5.1 EHR Analyst Agent

The **EHR Analyst** is the first Stage-1 agent. It reads `ehr_case.json` and produces a structured summary of demographics, active conditions, medications, history, and prior encounters. The prompt asks the agent to cite the record behind each claim so later agents and reviewers can check where the information came from. Internally, the agent uses three LLM calls (Figure 3.5): an *analysis* call, a *structure* call that fills JSON fields, and a *review* call that checks the structured output.

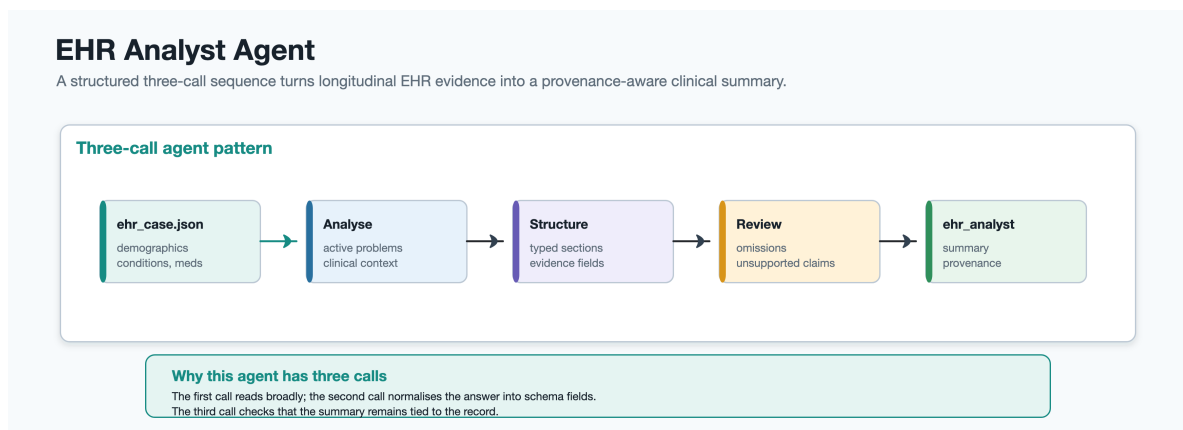


Figure 3.5: EHR Analyst Agent — the analyse/structure/review three-call sequence applied within the generic agent blueprint. Each call is a separate LLM invocation with its own system prompt loaded from `prompts/ehr_analyst.yaml`.

### 3.5.2 Lab Interpreter Agent

The **Lab Interpreter** is the second Stage-1 agent and runs in parallel with the EHR Analyst. It reads `lab_case.json`, compares each lab to its reference range, checks trends over time, and ranks abnormal findings by severity. Critical values receive a **CRITICAL** marker. It uses the same analyse–structure–review sequence as the EHR Analyst (Figure 3.6), with an added deterministic severity sort after the structure step.

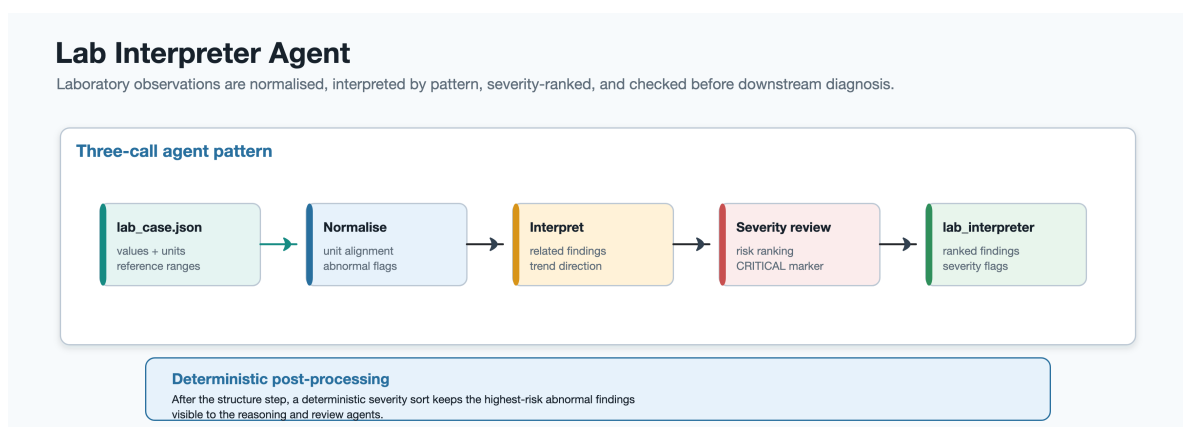


Figure 3.6: Lab Interpreter Agent — same three-call blueprint as the EHR Analyst, with an additional deterministic severity-ranking step injected between the structure and review calls. The ranking rule orders findings by their deviation from reference range and propagates a **CRITICAL** marker to downstream agents.

### 3.5.3 Diagnostic Reasoning Agent

The **Diagnostic Reasoning** agent is the Stage-2 agent that creates the ranked differential diagnosis.

Throughout this subsection a *prior* refers to a past patient case retrieved from Tier-4 case-based memory whose clinical features are similar to the current patient's. The diagnosis recorded for that retrieved case acts as a Bayesian-style starting point for the current reasoning, in the way a clinician's recall of a similar patient anchors their first impression. A prior is treated as *strong* only when the retrieved case clears both a similarity threshold and a quality threshold (the past case must itself have been a DIRECT match with high evaluator confidence); otherwise the agent acts as if no prior were available. The full case-based memory mechanism is described later in Section 3.6.2.

The agent has two modes. If memory is disabled, or if no strong prior is found, it uses an *adaptive critique loop*. If Tier-4 case-based memory returns a strong prior case, it uses a *three-phase split* to use the prior without blindly following it. Both modes are limited to three rounds.

In the adaptive loop, the agent first combines the EHR and lab summaries, then proposes at least three diagnoses, ranks them, and critiques the result. The loop stops when confidence reaches 75 or after three rounds. This design follows the idea that difficult cases may need more than one reasoning pass [17, 7].

**Three-phase split.** When memory returns a strong prior, the agent runs three rounds (Figure 3.7). *Round A* shows the prior case and asks the agent to use it only when the current evidence supports it. *Round B* hides the prior and asks the agent to reason from the current patient only. *Round C* compares both rounds. If they agree, the diagnosis is kept. If they disagree, Round B is preferred unless Round A has clear supporting evidence in the current patient. This clean-room round is included to reduce anchoring on retrieved cases [16].

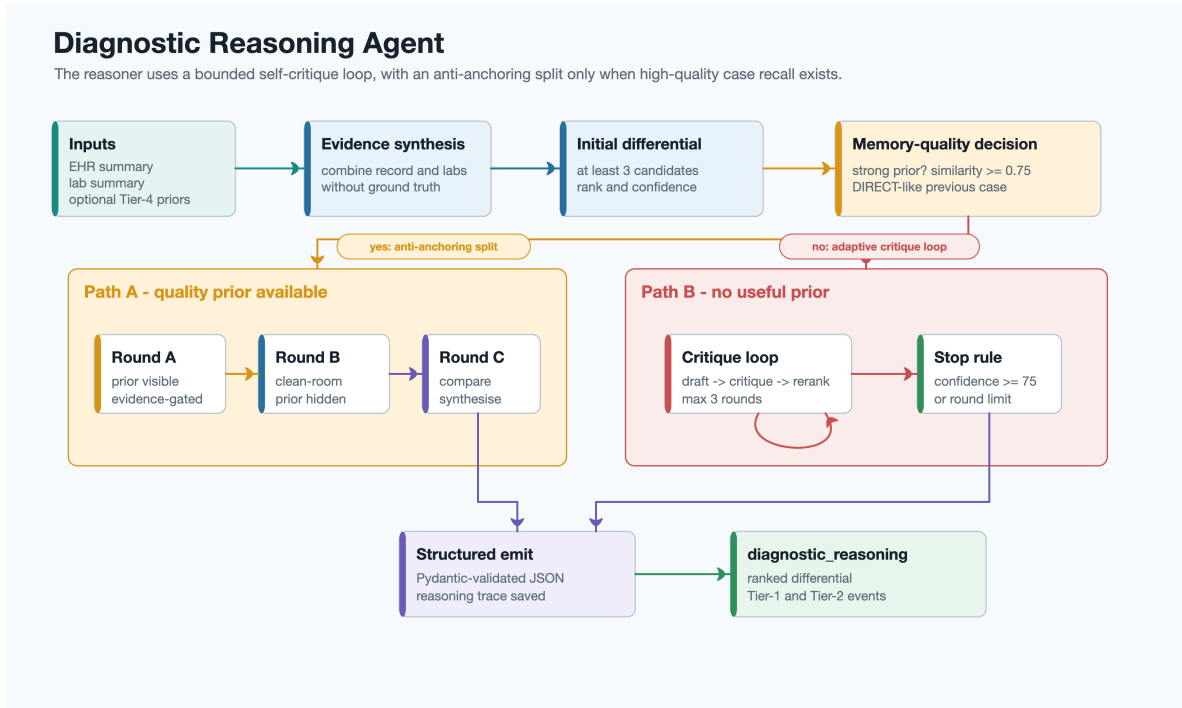


Figure 3.7: Diagnostic Reasoning Agent. EHR and lab summaries are combined with Tier 4 case recall; if a quality prior exists, the agent runs the three-phase anti-anchoring split (prior-visible, clean-room, synthesis). Without a prior, it uses the bounded adaptive critique loop with the confidence stop rule. Both paths emit `diagnostic_reasoning` and write the relevant Tier 1/Tier 2 memory events.

### 3.5.4 Clinical Reviewer Agent

The **Clinical Reviewer** is the Stage-3 agent. It re-checks the evidence and the diagnostic differential. Its prompt asks it to challenge the primary diagnosis, suggest missed alternatives, and identify possible bias. This is meant to avoid *silent agreement*, where agents appear to review each other but simply accept the first answer [16].

Figure 3.8 shows the review flow. When multi-level memory is enabled, the Reviewer also reads the Diagnostic Reasoning agent’s Tier-2 critique trail. This allows it to challenge how the diagnosis was reached, not only the final diagnosis name.

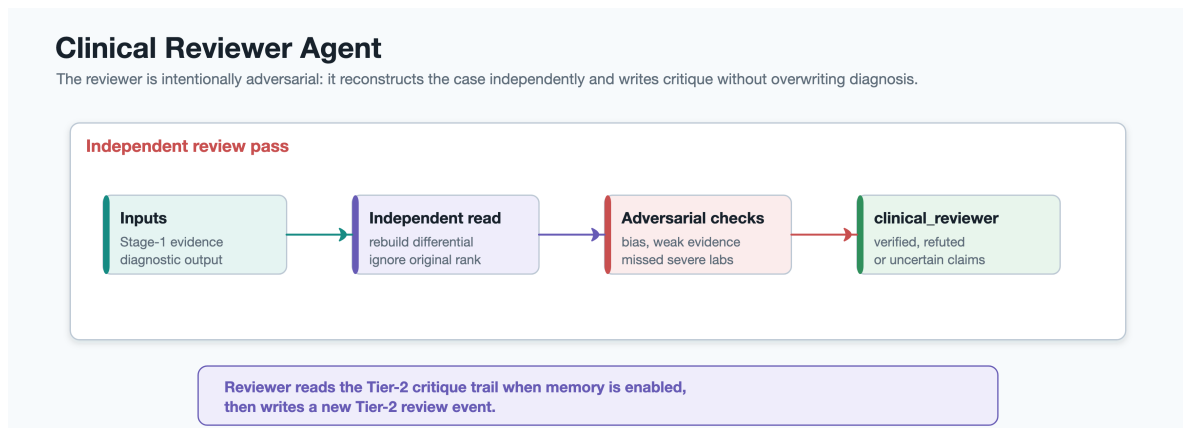


Figure 3.8: Clinical Reviewer Agent. The reviewer reads Stage 1 evidence and the Diagnostic Reasoning output, reconstructs an independent differential, performs adversarial checks, verifies claims, and writes a structured critique to Tier 2 without overwriting the diagnostic output.

### 3.5.5 Diagnostic Refiner Agent

The **Diagnostic Refiner** is the Stage-4 agent. It combines the Diagnostic Reasoning output with the Clinical Reviewer critique. When both agents agree, it mainly cleans terminology and adjusts ranks. When they disagree, it uses the review flags: *verified* diagnoses are promoted, *refuted* diagnoses are demoted, and *uncertain* diagnoses keep their position. The final output is written to `agent_outputs.final_diagnosis`. With memory enabled, the Refiner also reads the Tier-2 timeline and relevant Tier-3 disease priors. Figure 3.9 shows the merge flow.

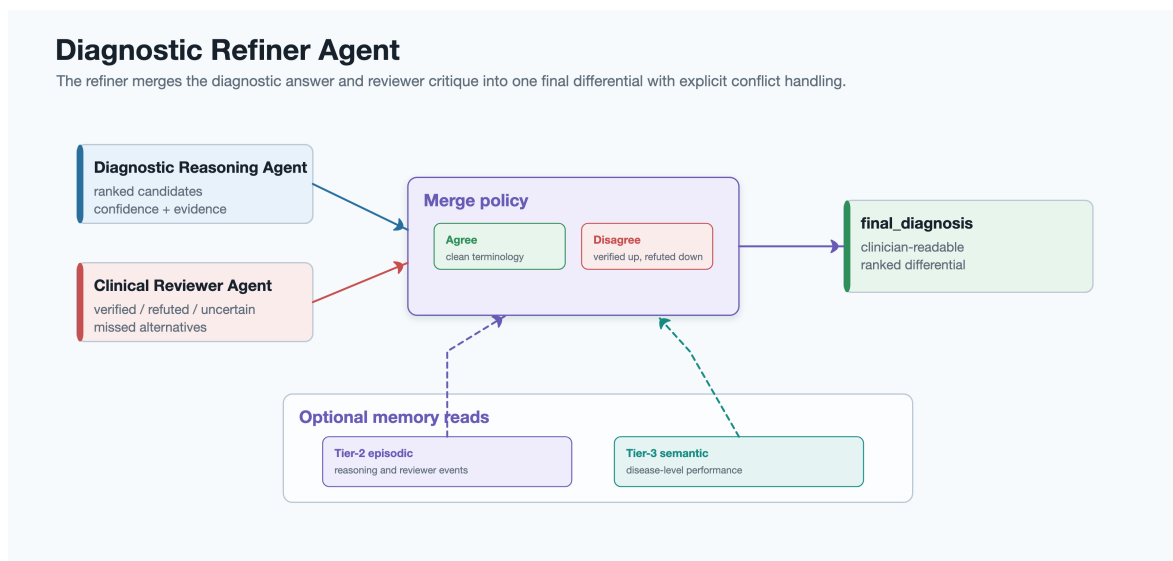


Figure 3.9: Diagnostic Refiner Agent. The merge call combines the Diagnostic Reasoning output, Clinical Reviewer critique, and Tier 3 semantic priors; evidence-alignment and final-schema checks produce the clinician-readable `final_diagnosis` consumed by the evaluator and treatment stages.

### 3.5.6 LLM Evaluator Agent

The **LLM Evaluator** is the Stage-5 agent. It compares the final differential with the Synthea ground truth in `ground_truth.json`. It uses a different model configuration from the reasoning agents so that the same model is not both generating and judging the answer. The evaluator returns DIRECT, INDIRECT, or MISS, as defined in Section 3.10.

### 3.5.7 Treatment Planning Agent

The **Treatment Planning** agent is the Stage-6 node. It runs after evaluation, but it writes a treatment plan only when the evaluator returns DIRECT. For INDIRECT or MISS outcomes, it writes a structured skip so that the run remains traceable without giving treatment advice for an uncertain diagnosis. For DIRECT cases, it retrieves the relevant NICE guideline passages from Qdrant and passes them to the LLM together with the final diagnosis and the patient's structured record.

**Structured output.** The agent emits a typed JSON object (Pydantic schema `TreatmentOutput`). The object holds the NICE guideline identifier (such as NG106 or NG136) and four content sections:

- **Prescribed medications** — each entry carries drug name, class, dose, duration, clinical purpose, the supporting NICE reference, and a line label (*first-line*, *second-line*, or *additional*).

- **Interaction checks** — pairwise interactions between proposed drugs and the patient’s current medications, each with a severity (mild, moderate, severe) and a recommended action.
- **Contraindications** — drugs that should *not* be given to this specific patient, with the reason and an alternative.
- **Assumptions and warnings** — explicitly flagged below.

**Warnings and assumptions surfaced to the doctor.** A NICE guideline is written for the data a clinician would normally have at hand, but a Synthea (or real-world) record often lacks specific fields. Rather than silently substituting defaults, the agent is required to list *every* assumption it was forced to make. The prompt instructs it to write one warning per missing data item that would otherwise have gated a recommendation. Typical entries include:

- “eGFR unknown — assumed normal for ACE-inhibitor dosing”
- “Allergy status unknown — cannot verify drug safety”
- “QRISK not calculated — NICE NG181 requires it for the statin decision”
- “LVEF unknown — cannot confirm HFrEF vs HFpEF for drug selection”
- “Liver function missing — cannot verify statin safety”
- “Potassium unknown — cannot verify MRA / spironolactone safety”

These warnings, together with any contraindications and severe interactions, are written into the same JSON object and rendered verbatim in the clinician dashboard alongside the medication list. The intent is that the doctor never sees a recommendation without also seeing what the agent did not know, so that calibrated trust [14] can be exercised at the point of use rather than retrospectively. Whether this surfacing strategy actually changes clinician behaviour remains an empirical question for prospective clinician-in-the-loop deployment, identified as future work in Chapter 6.

## 3.6 Shared Memory Design

Agents communicate through a typed shared state called `PipelineState`. On top of this state, the system adds a four-tier memory design inspired by CoALA [34]. The base state handles one patient run and merges agent updates safely. The four memory tiers then add scratch space, a run-level event trail, disease-level summaries across runs, and case-based recall from previous patients.

### 3.6.1 Base layer: PipelineState

Each field of `PipelineState` is a separate namespace with a reducer function. The reducer defines how partial updates are merged. This matters because Stage 1 runs two agents in parallel. If both write during the same step, the reducer controls the merge instead of leaving the result to last-writer-wins behaviour.

The shared state is divided into seven namespaces:

- `patient_context` — the Gold case object, written once by the orchestrator and read by all agents.
- `agent_outputs` — a dictionary with one slot per agent, using a write-own / read-upstream access pattern. The merge reducer ensures that parallel Stage-1 agents never overwrite each other.
- `execution_trace` — an append-only invocation log recording the status, duration, and error of every agent run.
- `scratchpad` — per-agent dictionaries; in the present design this slot is repurposed as the working-memory tier (Section 3.6.2).
- `conflicts` — an append-only list reserved for an orchestrator-level diff engine; not currently written by any agent and retained for forward compatibility.
- `session_memory` — append-only list of typed `SessionEvent` dictionaries that constitute the episodic-memory timeline for the current run.
- `session_summary` — per-agent one-line digest with a merge reducer, intended as a lightweight scoreboard for downstream routing logic.

### 3.6.2 Multi-level memory subsystem

The base shared state is useful, but it has two limits. Downstream agents see only the final structured output of earlier agents, not the reasoning trail. Also, nothing is carried from one patient run to another. The multi-level memory subsystem addresses these two limits while keeping the same reducer rules. All tiers are accessed through one `MemoryManager`, and the whole subsystem can be turned on or off with the `MEMORY_ENABLED` environment flag. Figure 3.10 shows the design.

**Tier 1 — Working memory.** Tier 1 is per-agent scratch space for a single invocation. It lives in `scratchpad[agent_id]`. The Diagnostic Reasoning agent uses it during its critique loop to keep round confidence values and short critique notes. This avoids reparsing earlier free-text responses. Tier 1 is cleared at the start of each invocation and is not visible to other agents.

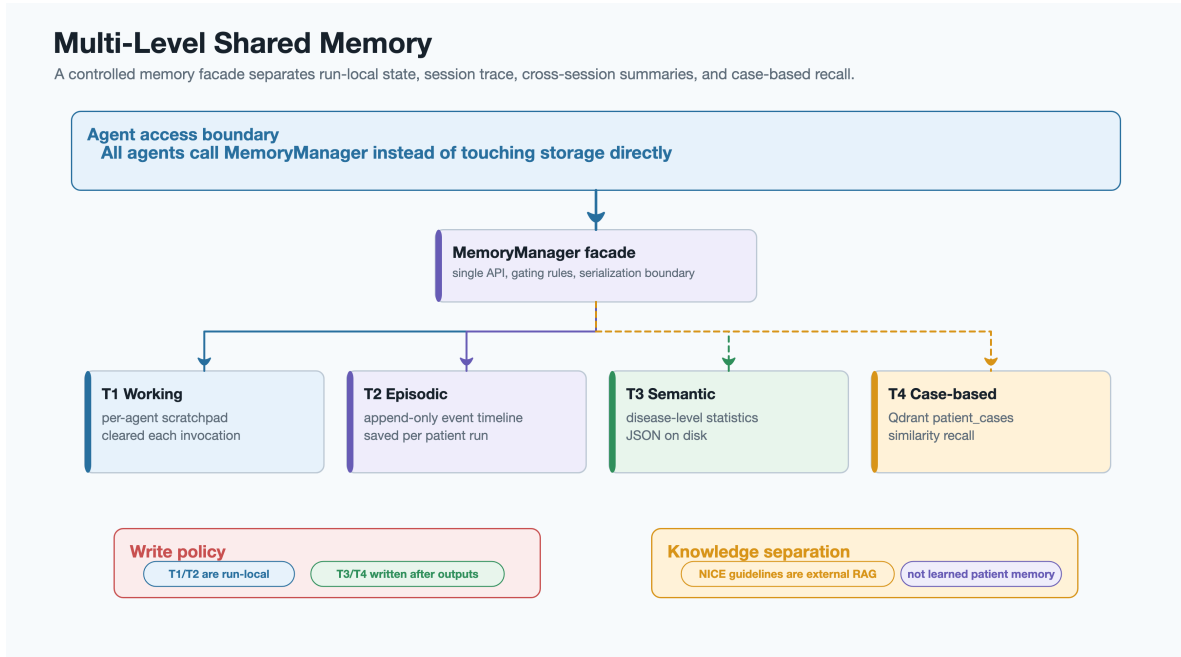


Figure 3.10: Four-tier multi-level memory subsystem layered on the LangGraph state. All agents access memory through the **MemoryManager** facade: Tier 1 and Tier 2 support per-run reasoning and traceability, while Tier 3 and Tier 4 persist cross-session semantic and case-based information. Stage 7 writes the durable tiers only after patient-facing outputs are produced. NICE guideline retrieval remains external reference knowledge, not learned memory.

**Tier 2 — Episodic memory.** Tier 2 is a typed event timeline for one patient run. It is written to `session_memory` through an append reducer. Each entry is a validated `SessionEvent` with an event type, agent id, one-line summary, and structured payload. The Clinical Reviewer reads the critique events when checking the Diagnostic Reasoning output. The Diagnostic Refiner reads the full timeline before producing the final differential. The timeline is saved as `session_memory.json` next to `execution_trace.json`.

**Tier 3 — Semantic memory.** Tier 3 stores disease-level statistics across runs in `data/gold/memory/semantic_memory.json`. For each disease, it keeps DIRECT, INDIRECT, and MISS counts, rank-1 frequency when found, average top differential probability, and a small set of observed evidence patterns. Updates are atomic and repeat-safe. Recall is a simple case-insensitive lookup by disease name. The Diagnostic Refiner and Treatment Planning agent read these entries as lightweight priors.

**Tier 4 — Case-based memory.** Tier 4 stores prior patient cases in Qdrant [42]. After a completed run, the system builds a short clinical text from demographics, active conditions, medications, and abnormal labs. This text is encoded into a vector using BioLORD-2023 [25] — a biomedical Sentence-BERT variant [30] fine-tuned on clinical

knowledge-graph signals — and stored in the `patient_cases` Qdrant collection. The payload stores the matched diagnosis, match type, rank, confidence, and evidence patterns. At the start of a new case, the Diagnostic Reasoning agent retrieves the top- $K$  similar cases ( $K = 5$  by default) and can use them as priors. If Qdrant or the embedding model is unavailable, recall returns an empty list instead of stopping the run.

The NICE guidelines used by the Treatment Planning agent are stored in a separate Qdrant collection called `nice_guidelines`. They are not part of memory because they are static external reference knowledge, not experience learned from previous patient runs.

### 3.6.3 Memory consolidation node

Tier 3 and Tier 4 are written by a Stage 7 maintenance node. This node runs after Treatment Planning. It reads the final diagnosis, evaluator verdict, and patient context. It can write a disease summary to Tier 3 and, when eligible, upsert the patient case to Tier 4. DIRECT cases are enabled by default for Tier 4. If one write fails, the other can still succeed, and the trace records the partial result. This keeps memory failures separate from the patient output already written by Stages 1–6.

### 3.6.4 Access pattern

The per-agent access pattern is summarised on Figure 3.10: an agent writes only to its own output slot and to append-only event channels, and cross-session writes (Tier 3 and Tier 4) are limited to the Stage 7 consolidation node.

The memory code lives in `src/memory/`, with one module per tier and a shared `MemoryManager` in `manager.py`. The Stage 7 consolidation node is implemented in `src/agents/memory_consolidator.py`.

## 3.7 Retrieval-Augmented Generation Layer

Retrieval-Augmented Generation [27] combines a learned retriever over an external document store with a generative model that conditions on the retrieved passages; in this pipeline it is the mechanism that lets the Treatment Planning agent ground its recommendations in NICE guideline text rather than relying on parametric knowledge alone. Two pieces of the pipeline retrieve from Qdrant at inference time. The Diagnostic Reasoning agent draws on Tier-4 case-based memory (described in Section 3.6.2) — past patient cases this system has previously processed, used as priors for the differential diagnosis. The Treatment Planning agent draws on the *external knowledge* corpus described in this section: NICE clinical practice guideline documents [43] authored outside the system. This separation is deliberate. Diagnosis must be based on the current patient’s evidence

and on similarity to past patients only; published guidelines are kept out of the diagnostic path so that the agents do not match a guideline-recognised pattern instead of reasoning from the evidence. Guidelines enter only after the LLM Evaluator has scored the diagnosis as DIRECT.

The external retrieval corpus contains NICE guideline documents for the target disease groups. The documents are split into overlapping chunks of about 400 tokens with 50-token overlap, encoded into vectors using BioLORD-2023, and stored in Qdrant. Qdrant was used because it supports filtered retrieval: the Treatment Planning agent can restrict retrieval to passages tagged with the final diagnosis code, which reduces the risk of pulling in passages from the wrong guideline. The same Qdrant deployment holds Tier-4 case-based memory in a separate collection (`patient_cases`), so the two retrieval paths share infrastructure but never share content.

At inference time, the Treatment Planning agent queries with the final diagnosis and the top supporting findings. It retrieves the top five passages, removes duplicates, and inserts them into the prompt. The prompt asks the model to cite the passage identifier for each recommendation. The Output Gate (described in Section 3.4) performs schema validation; the requirement that medications and procedures appear in the retrieved passages is enforced prompt-side rather than by a post-hoc retrieval-support check.

### 3.8 LLM Configuration and Provider Abstraction

All LLM calls go through one provider adapter. The adapter exposes Groq, OpenAI, Anthropic, Google Gemini, and Ollama through the same `get_llm()` interface. Configuration is read from environment variables. A separate `get_evaluator_llm()` function is used for the judge model, so the evaluator can be different from the reasoning model.

Three model configurations were used in this thesis:

- **GPT-OSS 120B** served via the Groq cloud API, used as the primary reasoning model.
- **Med42 70B** (`thewindmom/llama3-med42-70b`), a Llama-3 checkpoint fine-tuned on clinical text, served locally via Ollama and used in the medical-fine-tuning comparison reported in Chapter 4.
- **Qwen3 32B** served via Groq, used as the LLM Evaluator judge and also as the default detectability verifier in Section 3.2.3 via the shared `get_evaluator_llm()` factory. Qwen3 32B is independent of the two reasoning models under test, which prevents the judging result from being biased by shared training data with GPT-OSS-120B or Med42-70B. Because the cohort-admission verifier and the LLM judge resolve to the same default model, the headline DIRECT rate carries a same-model selection/evaluation coupling on cohort admission and judging; the detectability-gate sensitivity analysis in Section 4.4.1 is the explicit empirical check on this coupling (see also Section 4.4.1 for the rejected-band counterfactual that bounds the coupling’s effect).

All LLM calls use exponential backoff with at most three attempts. The adapter logs a structured warning when a single LLM call exceeds 120 seconds but does not, by itself, cancel the call. Per-agent execution is bounded by an outer 300-second budget that wraps the entire agent invocation (Input Gate  $\rightarrow$  Output Gate) and that the orchestrator uses to abort a stalled agent without aborting the patient run.

## 3.9 Observability and Trace Inspection

Observability in a multi-agent clinical system has two distinct audiences. The engineer rebuilding or extending the pipeline needs to see every prompt, every retry, and every JSON-parsing failure in context. The clinician or examiner reviewing a single patient run needs to see what each agent received, what it returned, and which hypothesis the Refiner kept against the Reviewer’s verified, refuted, and uncertain flags. CMADS handles these audiences with the same data, surfaced through two complementary layers — a local-first layer that always runs and an external developer-platform layer that augments it during development and debugging.

**Layer 1 — local persistence (always on).** Every patient run writes a structured trail to disk and to MongoDB without any external service. The orchestrator appends one entry per agent invocation to the `execution_trace` reducer channel (Section 3.6.1): agent identifier, start and end timestamps, wall-clock duration, terminal status (`success`, `partial`, `error`, or `skipped`), and a truncated error string when the agent raised. The per-agent JSON output written under the patient’s result-set directory (and into the Mongo `agent_runs` document) is the corresponding payload. Together these two artefacts answer “what happened on this patient, in what order, for how long, and to what success state” for every historical run without re-executing it. The Python `structlog`-based JSON-lines logger emits a parallel per-process log stream so a live or post-mortem investigation of ad-hoc behaviour (timeouts, JSON-mode fallbacks, repair-pass activations) can be done from the terminal.

**Layer 2 — LangSmith trace platform (optional, EU-hosted).** When the operator sets the `LANGSMITH_API_KEY` environment variable, the LLM adapter auto-enables LangSmith tracing [44] against the EU endpoint (<https://eu.api.smith.langchain.com>) under the project `cmads-clinical-pipeline`. The trace captures every LLM invocation issued by every agent (including the diagnostic loop’s multiple rounds and the Reviewer’s adversarial pass), the full prompt text, the raw model response before any JSON-repair work, latency, token usage where the provider reports it, and any retried call that exited via the `json_validate_failed` branch. Failures propagate as exception breadcrumbs attached to the parent span, so the trace tree shows where in the seven-agent topology a fault originated rather than only that the patient run aborted. LangSmith tracing is auto-instrumented through the LangChain integration and therefore does not require code changes inside each agent.

The two layers are deliberately redundant. The local layer is the authoritative record for thesis reproducibility and for regulatory traceability (see Section 5.6), survives without any external dependency, and is what the doctor-facing console reads. The LangSmith layer is the developer’s microscope — useful when an agent’s structured output looks wrong and the question is what the model actually returned before `BaseAgent._extract_json_from_response` repaired it. The operator can disable Layer 2 entirely by leaving the API key unset; no run on the empirical 160-patient cohort depends on LangSmith being reachable.

## 3.10 Evaluation Framework

The evaluation follows two rules. First, the ground truth should not be generated by an LLM, because the system being evaluated is already LLM-based. Synthea provides rule-based disease labels, so it avoids that circularity. Second, the model that judges the answer should not be the same model that generated it. For this reason, the reasoning model and judge model are kept separate.

### 3.10.1 Ground Truth Extraction

For every Synthea patient, Gold assembly creates a `ground_truth.json` file. It stores the target condition, SNOMED concept, human-readable disease name, and date of first diagnosis. Only the Stage 5 evaluator reads this file. Reasoning agents never receive it.

### 3.10.2 LLM-as-Judge Methodology

The evaluation here follows the LLM-as-a-judge protocol of Zheng et al. [26], in which a strong external model adjudicates the output of another model against a reference using a fixed rubric. The LLM Evaluator receives the target disease and the top five diagnoses from the final differential. It classifies each candidate as DIRECT, INDIRECT, or MISS using a fixed rubric, then selects the best match and reports its rank. Temperature is fixed at zero. The response uses a five-line structured format parsed by regex. Table 3.4 summarises the rubric.

### 3.10.3 Metrics

The primary outcomes reported in Chapter 4 are the three match-type rates and a conditional rank metric, supported by agent-level reliability and latency statistics.

- **DIRECT rate** — the proportion of patients for which at least one diagnosis in the top five is classified as DIRECT.

Table 3.4: Match-type rubric applied by the LLM Evaluator.

| Match type | Criterion                                                                                                                                                                             |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DIRECT     | The candidate names the same disease as the ground truth, possibly under a clinically equivalent synonym (e.g. “Atherosclerotic CVD” is a DIRECT match for “Ischemic heart disease”). |
| INDIRECT   | The candidate names a cause, consequence, precursor, or subtype of the ground truth (e.g. “CKD stage 4” is INDIRECT for “End-stage renal disease”).                                   |
| MISS       | The candidate has no clinical connection to the ground truth.                                                                                                                         |

- **INDIRECT rate** — the proportion of patients for which no diagnosis is DIRECT but at least one is INDIRECT.
- **MISS rate** — the proportion of patients with no DIRECT or INDIRECT match in the top five.
- **Found rate** — the combined DIRECT plus INDIRECT rate.
- **Rank-1-when-found** — among patients for whom the pipeline returns a DIRECT or INDIRECT match, the proportion for which that match is at rank 1. Denominator = DIRECT + INDIRECT; numerator = DIRECT (and INDIRECT) cases ranked first. This is the rank metric reported in Chapter 4; it replaces a legacy “Top-1 direct rate over all patients” framing.
- **Memory comparison metrics** — the controlled 160-patient paired cohort (Section 4.7) compares DIRECT, INDIRECT, MISS, Found, and Rank-1-within-found rates between the single-level and multi-level memory configurations on the same patients.
- **Detectability-gate sensitivity** — a descriptive comparison of DIRECT, INDIRECT, and MISS rates on the detectability-admitted cohort against an independent sample drawn from the rejected band of the verifier (Section 4.4.1).
- **Agent success rate** — per-agent, the proportion of patients for which the agent terminated without error.
- **Per-patient latency** — wall-clock time from orchestrator invocation to final-diagnosis write.

### 3.11 Implementation Stack and Deployment

The experiments used a local Apple Silicon workstation for the data pipeline and local Ollama runs. GPT-OSS-120B and Qwen3-32B were served through Groq. Qdrant was

used as a managed cloud vector store. Synthea is the only patient data generator, and its FHIR R4 output passes through the medallion data lake before the agents run. Figure 3.11 groups the technology stack by layer.

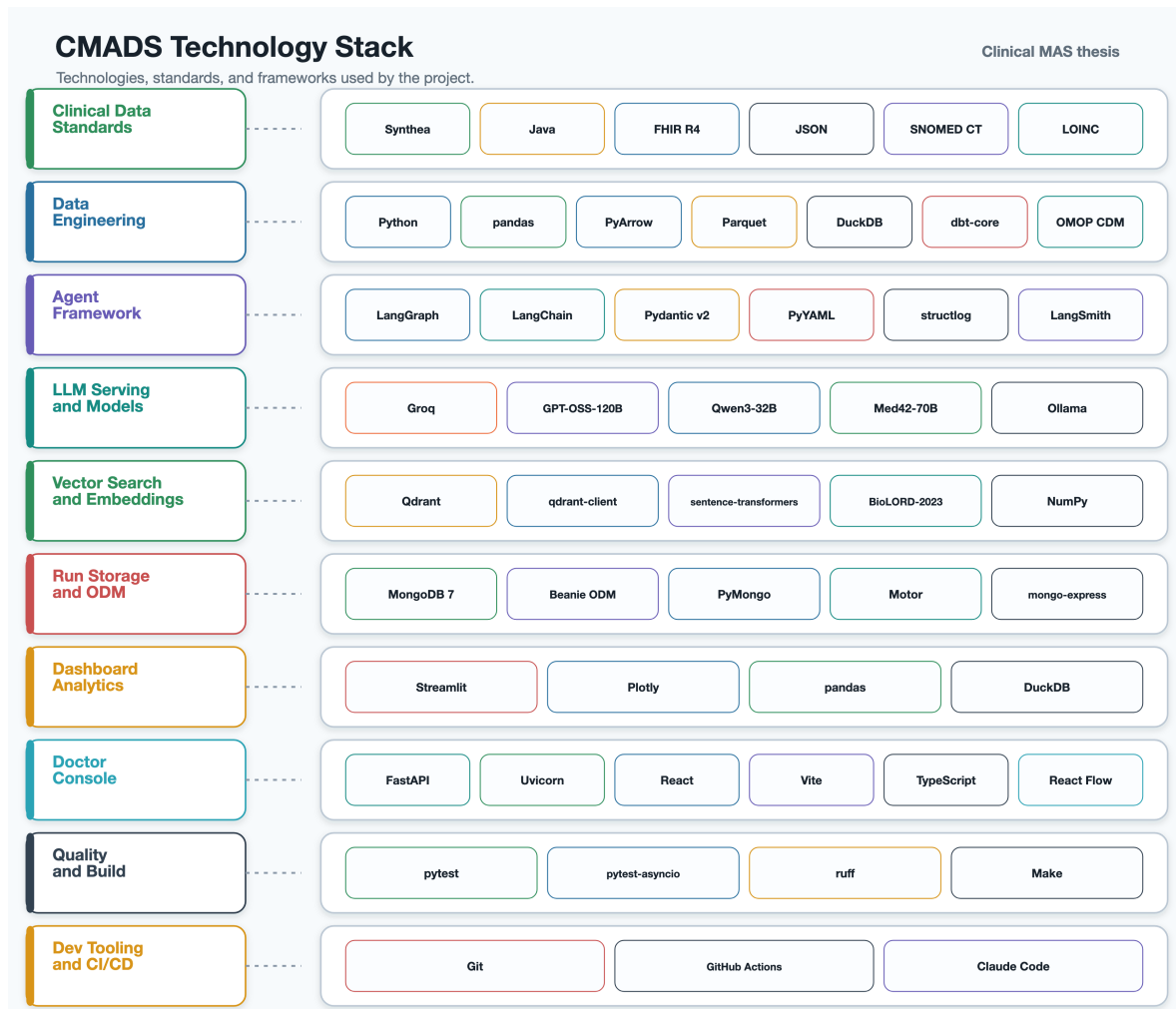


Figure 3.11: CMADS technology stack grouped by layer. Synthea is the sole data source; LangGraph orchestrates the agent graph; the multi-tier shared memory is realised through PipelineState plus a Qdrant-backed case-based store; treatment retrieval uses BioLORD-2023 embeddings over Qdrant. Per-run agent outputs and execution traces are persisted to MongoDB 7 (accessed via Beanie ODM on the read path and PyMongo on the write path; mongo-express provides a browser GUI on :8091 for inspection). The provider-agnostic LLM adapter routes calls to Groq by default, with OpenAI, Anthropic, Gemini, and Ollama as drop-in alternatives.

Environment-specific settings, such as model name, Ollama endpoint, prompt paths, output directory, and checkpoint location, are stored in configuration files or environment variables. A central `Config` class reads them at process start. This allows the same Gold JSON inputs and LangGraph pipeline to be rerun without changing the code.

### 3.11.1 Storage layer and scalability

The system uses two persistence layers with different roles. The *cold* layer is the read-only ground-truth pile: Synthea FHIR bundles in `data/bronze/`, the OMOP CDM Silver tables in the DuckDB file `data/clinical.duckdb`, and the Qdrant vector collections that hold the NICE guideline embeddings and the Tier-4 case-based memory. These do not change during agent runs and are not the throughput bottleneck. The *hot* layer is MongoDB 7, holding four collections that grow with every patient run: `agent_runs` (one document per `(result_set, patient_uuid)` pair, with one field per agent envelope), `patient_cases` (the Gold-layer inputs, one document per patient), `semantic_memory` (one document per disease with running counts), and `derived_artefacts` (per-cohort McNemar tables, sensitivity summaries, and similar analysis outputs).

The choice of MongoDB is motivated by the read pattern of the Researcher journey rather than by the write throughput of patient runs. Every visit to the Overview, Memory A/B, Model, or MAS-vs-single-LLM tab is one or more aggregations across the 160-patient principal cohort (or the wider union of all completed runs); on the original on-disk JSON layout each page rendered roughly  $\sim 5$  thousand file reads, so latency grew linearly with cohort size. With the four-collection schema and an indexed compound key on `(result_set, patient_uuid)`, the same KPI tiles are computed in a single `$group` aggregation that completes in tens of milliseconds. The migration also makes the system ready for real-EHR replication: the same agent code paths now accept any number of `result_sets` without changing the read code, and the addition of a sharded MongoDB deployment is a deployment change rather than a code change.

**Write path.** Agent envelopes are written to Mongo from inside `BaseAgent.__call__` using the synchronous PyMongo driver. The pipeline runs synchronously inside worker threads dispatched by the runtime, and Motor (the async driver) is event-loop-bound, so the sync path is the only one that completes reliably from any thread. The filesystem JSON write that has always existed remains untouched during a 30-day soak window controlled by the `USE_MONGO` configuration flag: with the flag off the system falls straight back to the on-disk tree, which is the documented rollback artefact.

**Read path.** The FastAPI backend at `doctor_console/backend/app.py` owns the read path. The aggregation helpers (`_aggregate_result_set_mongo`, `_rank_distribution_mongo`, `_per_disease_breakdown_mongo`, `_top_diagnoses_mongo`) issue Mongo aggregation pipelines that coalesce the canon-augmented verdict (`agents.evaluation.output_canon`) over the original (`agents.evaluation.output`) via `$ifNull`, so the headline numbers in the Researcher Overview match the corrected verdicts reported in Section 4.4 without a separate post-processing pass. The per-patient detail endpoint (`/api/results/...`) keeps reading from disk because it needs the rich agent narrative and shared-memory summary that only the filesystem layout currently materialises.

**API.** The doctor console talks to the backend over a small REST surface served by FastAPI: `/api/stats/overview` for the headline KPIs, `/api/comparisons/memory-ab`, `/api/comparisons/model`, and `/api/comparisons/mas-vs-single-llm` for the three comparison tabs, `/api/patients` for the cohort sidebar, `/api/results/{result_set}/{patient_uuid}` for the per-patient drill-down, and a small streaming pair (POST `/api/runs`, GET `/api/runs/{task_id}/stream`) for the Doctor (runtime) journey. The streaming endpoint pushes the agent-flow state via Server-Sent Events so the runtime view can “light up” the active stage and stream each agent’s narrative as it completes, without polling. FastAPI’s lifespan handler binds Beanie to the configured database at startup when `USE_MONGO` is enabled; with it disabled the same handlers fall back to filesystem-scanning helpers, preserving the rollback contract.

**Patient builder.** The Researcher journey’s Patients tab is a dropdown of three sub-views: *Explore* (per-UUID drill-down on the verified cohort), *Create* (a two-pane editor for authoring a test patient from scratch or cloning a verified cohort patient as a starting point), and *My test patients* (saved builder runs the reviewer can re-run or edit). The Create editor surfaces demographics, active conditions, medications, vitals, and recent labs; a *Smart Import* flow inside the Recent labs section extracts structured lab values from pasted text, an uploaded PDF or FHIR JSON file, or a photo of a lab slip (parsed with Gemini 2.5 Flash) so an arbitrary panel can be entered without retyping every numeric result. Builder runs are written to a separate `mas_results_test` cohort and never reach the headline statistics on the Overview / Memory A/B / Model-comparison tabs.

## Code availability

The full implementation — data pipeline, agent prompts, LangGraph workflow, shared-memory subsystem, evaluator, and React doctor console — is published at [github.com/abdammin004/CMADS](https://github.com/abdammin004/CMADS). The repository contains a Makefile with single-command targets for installing dependencies, running the pipeline on one patient or a batch, and launching the doctor console (API plus web client). A **Quick Start** block at the top of the **README** walks an external reader from clone to a running dashboard in five commands.

## 3.12 End-to-End Execution Pipeline

Algorithm 1 summarises one full patient run. The pipeline can be started from the command line for batch runs or through the doctor console’s Doctor (runtime) journey for interactive review.

After each patient, the system writes all agent outputs, the execution trace, and the evaluation result to MongoDB. These records can be reopened in the doctor console (Researcher journey) or analysed later without rerunning the agents.

---

**Algorithm 1** End-to-end processing of a single patient.

---

- 1: Load Gold case objects `ehr_case.json` and `lab_case.json`
  - 2: Initialise `PipelineState` with patient context and empty output slots
  - 3: **Stage 1 (parallel)**: invoke EHR Analyst and Lab Interpreter concurrently
  - 4: **Stage 2**: invoke Diagnostic Reasoning; repeat critique round until confidence  $\geq 75$  or three rounds executed
  - 5: **Stage 3**: invoke Clinical Reviewer with Stage 1 and Stage 2 outputs
  - 6: **Stage 4**: invoke Diagnostic Refiner to merge Stages 2 and 3
  - 7: **Stage 5**: invoke LLM Evaluator to classify the final diagnosis against the Synthea ground truth
  - 8: **Stage 6**: invoke Treatment Planning; DIRECT matches use RAG over Qdrant, while INDIRECT or MISS outcomes produce a structured skip result
  - 9: **Stage 7**: invoke Memory Consolidation; when enabled, update Tier 3 semantic memory and eligible Tier 4 patient cases
  - 10: Persist `agent_outputs` and `execution_trace` to MongoDB (one document per agent stage, keyed by `patient_uuid`)
-



# Chapter 4

## Results

### 4.1 Overview

This chapter reports the empirical results of the multi-agent clinical decision support system described in Chapter 3. All numbers are computed on the same 160-patient Synthea-derived cohort and judged with the LLM-as-Judge method from Section 3.10. Table 4.1 gives every headline number a reader needs at a glance; the rest of the chapter unpacks them.

Table 4.1: Headline numbers at a glance. Every value below is computed on the same 160-patient paired cohort; each result has its own section in this chapter for the supporting detail.

| What it measures                        | Value                              | Section |
|-----------------------------------------|------------------------------------|---------|
| System DIRECT rate                      | <b>76.9 %</b> (123/160)            | 4.4     |
| System Found rate (top-5)               | <b>94.4 %</b> (151/160)            | 4.4     |
| Memory effect (paired, vs single-level) | <b>+23.8 pp</b> DIRECT             | 4.7     |
| Orchestration effect (vs single-LLM)    | <b>+33.8 pp</b> DIRECT             | 4.11    |
| Detectability dependence                | −63.6 pp on rejected-band patients | 4.4.1   |
| Median wall-clock per patient           | 112 s                              | 4.5     |

### 4.2 Experimental Setup

**Cohort.** The evaluation cohort consists of 160 Gold-layer patients selected by the deterministic eligibility rules and the LLM-based detectability verifier of Section 3.2.3, and run end-to-end through the seven-agent pipeline. These patients cover the target disease groups used in the thesis: ischemic heart disease (IHD), chronic congestive heart failure (CHF), essential hypertension, type-2 diabetes mellitus (T2DM), end-stage renal disease

together with chronic kidney disease stages 2–3 (reported together as ESRD / CKD), and metabolic syndrome X. The CKD substages are separate SNOMED labels in the data, but they share one NICE management pathway, so they are grouped for reporting. The aggregate file shows the principal 160 cohort contains ESRD, CKD stage 3, and CKD stage 2 only; no CKD stage 1 patients are present. The agents do not see the ground-truth condition during inference; only the evaluator receives it.

**Models and infrastructure.** The main reasoning model is GPT-OSS-120B served through Groq. The evaluator is Qwen3-32B, also served through Groq. Both are accessed through the provider adapter described in Section 3.11. Qdrant is used for NICE guideline retrieval and case-based memory. The data pipeline, prompt rendering, agent orchestration, and evaluation scripts run on an Apple Silicon workstation. The React + FastAPI doctor console is served locally on the same workstation.

**Pipeline configuration.** All runs use the default values in `src/config.py`: maximum diagnostic rounds of 3, confidence threshold of 75, an outer per-agent execution budget of 300 seconds, a 120-second slow-call warning threshold in the provider adapter (logged rather than enforced as per-call cancellation), and capped exponential retry. Prompt templates are loaded from `prompts/{agent_id}.yaml`. No prompt or threshold was tuned after seeing the reported results. Run files are saved under `data/gold/mas_results_*/`, with one folder per patient UUID. Each folder contains one JSON file per agent and an `execution_trace.json` timeline. The metrics below are computed from those files.<sup>1</sup>

### 4.3 Evaluation Metrics

The evaluator produces five main values for each patient. **Match type** is DIRECT, INDIRECT, or MISS. DIRECT means the target disease appears in the differential as a clear match. INDIRECT means the system found a clinically related disease. MISS means it did not find the target disease or an acceptable proxy. **Rank-when-found** records where the matched disease appears when the judge returns DIRECT or INDIRECT. *Rank-1-when-found* is the share of found cases where the match is ranked first. **Per-agent status** records success, partial, error, or skipped in `execution_trace.json`. *Partial* means the output was recovered after a validation issue. *Error* means the agent failed

---

<sup>1</sup>The headline outcomes in this section are computed from the principal multi-level memory configuration on the 160-patient paired cohort, whose per-patient outputs live in the three multi-level run directories `data/gold/mas_results_improved_b3/`, `data/gold/mas_results_improved_50/`, and `data/gold/mas_results_improved_extra60/`. The paired memory A/B in Section 4.7 compares those same UUIDs under the single-level baseline configuration, whose outputs are read from `data/gold/mas_results/` (the 65 overlap UUIDs from the original baseline cohort) and `data/gold/mas_results_paired95_single_level/` (the 95 UUIDs re-run with `MEMORY_ENABLED=false`). The single-LLM controlled baseline in Section 4.11 writes to `data/gold/mas_results_single_llm_baseline/`.

and downstream agents carried a `null` value. *Skipped* means the agent was intentionally bypassed, as with Treatment Planning when the match is not DIRECT. **Per-agent latency** is the wall-clock time for each agent call. For memory, the chapter reports a paired controlled comparison on 160 identical patients run in each memory setting and discusses cohort leakage as a limitation of interpreting memory effects.

**How rates are reported.** All diagnostic outcomes are reported as percentages with the underlying counts in parentheses, e.g. 76.9% (123/160), so the reader can see both the rate and the denominator that produced it. Comparisons in this chapter are paired wherever possible: the memory A/B (Section 4.7) and the single-LLM baseline (Section 4.11) both run two configurations on the same patient set, so the comparison controls for cohort composition. The detectability-gate check (Section 4.4.1) is a single-arm sensitivity sample on the rejected band of the verifier, reported as a descriptive contrast with the admitted-cohort rate rather than as an inferential test.

## 4.4 End-to-End Diagnostic Outcomes

Table 4.2 shows the main diagnostic results on the 160-patient paired cohort with the multi-level memory subsystem enabled. The system returns a DIRECT match in 76.9% of cases (123/160, Wilson 95% CI [69.8, 82.7]), an INDIRECT match in 17.5% [12.4, 24.1], and a MISS in 5.6% [3.0, 10.3]. The combined found rate (DIRECT + INDIRECT) is 94.4% [89.7, 97.0]. When the target disease is found, it is ranked first in 63.2% [55.3, 70.4] of found cases.

Table 4.2: Headline diagnostic outcomes on the 160-patient paired cohort under the principal multi-level memory configuration. Wilson 95% confidence intervals are shown in brackets. The same 160 UUIDs anchor the paired memory A/B in Section 4.7, where the single-level arm scores 54.4% DIRECT (87/160) on the identical patient set — a +23.1 percentage-point gap in favour of the multi-level configuration on the paired comparison.

| Match type                | Rate (count / 160) | Wilson 95% CI |
|---------------------------|--------------------|---------------|
| DIRECT                    | 76.9% (123/160)    | [69.8, 82.7]  |
| INDIRECT                  | 17.5% (28/160)     | [12.4, 24.1]  |
| MISS                      | 5.6% (9/160)       | [3.0, 10.3]   |
| Found (DIRECT + INDIRECT) | 94.4% (151/160)    | [89.7, 97.0]  |
| Rank-1 within found       | 63.2% (96/152)     | [55.3, 70.4]  |
| Median runtime / patient  | 112 s (mean 124 s) |               |

Table 4.3 breaks down the same cohort by disease family. Performance is strongest on ESRD / CKD and metabolic syndrome X. The renal subtotal reaches an 87.5% DIRECT rate and a 100% found rate. Smaller groups such as type-2 diabetes ( $n = 7$ )

and chronic congestive heart failure ( $n = 3$ ) carry too few patients to be read as reliable per-disease accuracy estimates and are marked accordingly. The cohort is also imbalanced — ESRD / CKD accounts for 40.0 % of the runs, while CHF and T2DM together account for 6.3 % — a direct consequence of the Synthea module weights and of the detectability-gate output discussed in Section 3.2.3. Essential hypertension and ischemic heart disease are the outliers among the well-powered groups: their DIRECT rates are lower than the cohort average, reflecting Synthea cases where blood-pressure or cardiac evidence alone was not enough for the agent to place the target disease in the top-five differential at rank one.

Table 4.3: Diagnostic outcomes by target disease family on the 160-patient paired cohort under the principal multi-level memory configuration. The renal families (ESRD, CKD stage 3, CKD stage 2) are reported separately and as a combined subtotal because they share a single NICE management pathway. Rows marked † have  $n \leq 8$  and are not powered for per-disease accuracy claims.

| Disease family                    | N  | DIRECT | INDIRECT | MISS  |
|-----------------------------------|----|--------|----------|-------|
| End-stage renal disease           | 49 | 95.9%  | 4.1%     | 0.0%  |
| CKD stage 3                       | 10 | 70.0%  | 30.0%    | 0.0%  |
| CKD stage 2†                      | 5  | 40.0%  | 60.0%    | 0.0%  |
| <i>(Renal subtotal)</i>           | 64 | 87.5%  | 12.5%    | 0.0%  |
| Metabolic syndrome X              | 36 | 86.1%  | 13.9%    | 0.0%  |
| Essential hypertension            | 23 | 73.9%  | 17.4%    | 8.7%  |
| Ischemic heart disease            | 27 | 51.9%  | 25.9%    | 22.2% |
| Diabetes mellitus type 2†         | 7  | 57.1%  | 42.9%    | 0.0%  |
| Chronic congestive heart failure† | 3  | 100.0% | 0.0%     | 0.0%  |

**Why IHD lags the cohort.** The IHD result (51.9 % DIRECT on  $n = 27$ ) is the single largest drag on the headline rate, and it is worth understanding the mechanism. Real cardiology cases come with imaging signals — ECG ST-segment changes, regional wall-motion abnormalities on echo, coronary calcium burden — and with biomarker signals such as troponin trajectories and BNP. None of these are present in the Synthea Gold-layer evidence the pipeline receives: Synthea emits SNOMED conditions, vitals, and a small set of laboratory panels, but no imaging modalities and no high-sensitivity troponin time series. The diagnostic agent therefore has to lean on the cardiometabolic proxies it can see — hypertension, dyslipidaemia, and diabetes — and frequently surfaces an IHD-adjacent term such as “Atherosclerotic coronary artery disease with recent myocardial infarction” at rank one. The judge then classifies this as INDIRECT for the broader Ischemic heart disease target, which is why the IHD row has both a depressed DIRECT rate and an elevated INDIRECT rate (25.9 %). This is a data-modality gap, not a reasoning failure — adding an imaging-converter agent would lift IHD without retraining the diagnostic core.

### 4.4.1 Detectability-Gate Sensitivity

The 76.9% DIRECT headline is evaluated on a cohort that has already passed the LLM-based detectability verifier described in Section 3.2.3, which admits only patients with a confidence score of at least 80. This is a deliberate data-quality choice, not a hidden filter, but it does mean the reported rate is conditional on the gate. To quantify how much of the headline depends on this conditioning, a sensitivity sample was drawn from the rejected band of the verifier output (confidence 60–79), stratified evenly between the two ten-point sub-bands and seeded so the selection is reproducible (`scripts/sample_rejected.py`, seed 42). The selected patients have valid Gold-layer evidence and a Synthea ground-truth label; only the detectability verifier judged them as borderline rather than clearly diagnosable.

These thirty patients were then run end-to-end through the same seven-agent pipeline, with results persisted to `data/gold/mas_results_detectability_sensitivity/`, and judged by the same Qwen3-32B evaluator used for the main cohort. The intent is to estimate the DIRECT rate the system would have achieved on cases the gate currently excludes and to contrast it with the admitted-cohort rate; this is a single-arm sensitivity sample, not a paired comparison, so it is read here as a descriptive contrast on patient detectability rather than as an inferential test.

**Sensitivity result.** On the thirty rejected-band patients, the system recorded a DIRECT match in 13.3% of cases (4/30), an INDIRECT match in 56.7% (17/30), and a MISS in 30.0% (9/30). Compared with the 76.9% (123/160) DIRECT rate on the accepted cohort, this is a drop of 63.6 percentage points — a roughly five-fold gap between the two conditions, consistent with the verifier’s role as a hard data-quality gate.

**Reading the result.** The gate is doing real selection work: the 76.9% headline is conditional on detectability rather than a population-level capability, and the gap between the two conditions is large. The interpretation throughout this chapter is therefore that CMADS produces a 76.9% DIRECT rate *on patients the detectability verifier admits*, and a 13.3% DIRECT rate on the rejected band of the verifier. The INDIRECT rate on the rejected band is comparatively high (56.7%), which suggests the pipeline still surfaces clinically related findings on borderline cases even when it fails to identify the exact target disease. The DIRECT gap across the gate also acts as a post-hoc consistency check on the verifier itself: the verifier’s confidence score is calibrated against the same outcome the agent pipeline is later evaluated on, which is the behaviour the gate was designed to produce. The raw counts are persisted to `data/gold/mas_results_detectability_sensitivity/sensitivity_summary.json` for reproducibility.

## 4.5 Per-Agent Reliability and Latency

Table 4.4 reports per-agent status, and Table 4.5 reports per-agent wall-clock latency. Diagnostic Reasoning, Clinical Reviewer, Diagnostic Refiner, and LLM Evaluator com-

plete successfully on almost every run. The EHR Analyst and Lab Interpreter usually produce valid outputs, and when they do not, the pipeline records a partial status and continues. Treatment Planning is skipped when the evaluator returns INDIRECT or MISS, which matches the DIRECT-gated design in Section 3.5.7.

Table 4.4: Per-agent reliability on the 160-patient cohort. *Output file present* = the agent produced its persisted JSON output (best-case view, evaluated over the full  $n = 160$ ); *Success (trace)* = the status string written into the run trace (worst-case view: 49 of 160 runs predate a telemetry change and report *trace missing* on the LLM Evaluator and Treatment Planning rows). Best-case = the output file confirms success on those 49 runs; worst-case = those 49 runs are counted as failures. The two views diverge only for the last two rows and bound the agent’s true reliability.

| Agent                | Output file present | Success (trace) | Partial | Error | Skipped | Trace missing |
|----------------------|---------------------|-----------------|---------|-------|---------|---------------|
| EHR Analyst          | 100%                | 91%             | 9%      | 0%    | 0%      | 0%            |
| Lab Interpreter      | 100%                | 94%             | 6%      | <1%   | 0%      | 0%            |
| Diagnostic Reasoning | 100%                | 100%            | 0%      | 0%    | 0%      | 0%            |
| Clinical Reviewer    | 100%                | 99%             | 1%      | 0%    | 0%      | 0%            |
| Diagnostic Refiner   | 100%                | 100%            | 0%      | 0%    | 0%      | 0%            |
| LLM Evaluator        | 100%                | 69%             | 0%      | 0%    | 0%      | 31%           |
| Treatment Planning   | 96%                 | 46%             | 0%      | 3%    | 20%     | 31%           |

Table 4.5: Per-agent wall-clock latency on the 160-patient cohort, computed from cases with a recorded trace entry.

| Agent                | Mean   | p95     |
|----------------------|--------|---------|
| EHR Analyst          | 28.1 s | 31.9 s  |
| Lab Interpreter      | 31.9 s | 52.2 s  |
| Diagnostic Reasoning | 99.6 s | 133.1 s |
| Clinical Reviewer    | 38.3 s | 56.4 s  |
| Diagnostic Refiner   | 10.9 s | 16.5 s  |
| LLM Evaluator        | 1.2 s  | 1.9 s   |
| Treatment Planning   | 11.4 s | 21.5 s  |

The 31% *trace missing* entries for the LLM Evaluator and Treatment Planning rows are the same 49 of 160 cases, all from runs that predate a telemetry change and therefore have output files but no Stage 5 / Stage 6 status string. Read worst-case as in Table 4.4, those 49 cases are unknown; read best-case (output file inspected directly), the Evaluator reaches 100% success and Treatment Planning matches its DIRECT-gated design. Seven runs additionally have no persisted `treatment_planning.json` file at all; in these cases the pipeline reached Stage 5 (LLM Evaluator) but exited before Stage 6 wrote a file, either through the `SkipAgentException` path (the structured skip used for non-DIRECT matches in current code) or through a pre-telemetry transient that no longer occurs on re-runs. A small number of pre-telemetry MISS / INDIRECT runs hold non-skip placeholders rather than the structured skip described in Section 3.5.7; those are stale artifacts from earlier code, not evidence of a treatment-gate failure.

Diagnostic Reasoning is the slowest stage, with a mean runtime of about 100 seconds and a 95th percentile near 133 seconds. This is expected because the agent uses several LLM calls and may run more than one reasoning round. Most other agents finish in under one minute on average. The LLM Evaluator is fastest because it performs a short classification task. The reliability table also shows the value of graceful degradation: when a Stage 1 output was only partial, the later agents still ran and produced a final diagnosis. No patient run failed completely because of an upstream agent error.

## 4.6 Adaptive Reasoning and Reviewer/Refiner Effects

Two design choices can be checked from the run files: the adaptive diagnostic loop and the non-destructive Reviewer-Refiner pattern from Section 3.5.5.

The Reviewer-Refiner pattern had a visible effect. A programmatic comparison of the rank-1 diagnosis between `diagnostic_reasoning.json` and `final_diagnosis.json` across the full cohort (`scripts/refiner_changed_primary.py`) finds that the Diagnostic Refiner changed the primary diagnosis originally produced by the Diagnostic Reasoning agent in **93 of 160 runs (58.1%)**. This happened because the Refiner does not delete the original diagnostic output; it can change the final primary while keeping the earlier output available for audit. The Reviewer also often flagged anchoring on one strong lab value, such as creatinine pushing the agent toward ESRD when the full patient history supported another primary condition. The Refiner then re-weighted the differential using that critique.

The Diagnostic Reasoning agent also used its bounded self-critique loop. The loop stops when confidence reaches 75 or after three rounds. In the memory runs where `session_memory.json` is available, most cases stopped after one round, about a third stopped after two rounds, and the rest used all three rounds. This saves time on easier cases and leaves more reasoning budget for harder ones.

## 4.7 Multi-Level Memory A/B Study

The memory subsystem is evaluated as a paired controlled experiment on the same 160 patients from Section 4.2, run twice under `MEMORY_ENABLED=false` and `MEMORY_ENABLED=true`. The controlled variable is the memory configuration; patient identity, prompts, evaluator, and model are held fixed across arms. The persisted per-patient outcomes and pair table are at `data/gold/paired_160_mcnemar.json`.

On the same 160 patients, the DIRECT rate under the multi-level configuration is **78.1%** (125/160) against **54.4%** (87/160) for the single-level baseline — a +23.8 percentage-point advantage on the paired controlled comparison.

The Found rate also rises in the multi-level configuration — **95.0 %** (152/160) against **91.9 %** (147/160) for the single-level baseline, a +3.1 percentage-point gain on the same patients — with a matching −3.1 percentage-point drop on MISS (5.0 % vs. 8.1 %). The substantive contribution of the multi-level memory configuration is therefore on both the primary diagnosis and differential breadth, with the broader-differential effect documented in Section 4.10 lifting Found above what the single-level baseline achieves on the same patients.

The earlier descriptive 205-vs-100 unpaired aggregate is retired. The apparent — and incidentally inverted — DIRECT gap in that unpaired comparison is a cohort-selection artifact: the multi-level cohort (batches 3, 4, and the supplementary extra-60 fills from batches 1 and 5) has a different disease distribution than the patients in `mas_results/` that produced the headline number in Section 4.4. When patient identity is held constant, the principal multi-level memory configuration wins by +23.8 percentage points on DIRECT against the single-level baseline (78.1 % vs. 54.4 %, 125/160 vs. 87/160) and adds +3.1 percentage points on Found (95.0 % vs. 91.9 %). On the paired test that earlier cohort-selection artifact disappears and the full configuration’s contribution is large and positive.

An intermediate configuration — Tier-4 case-based memory only, with the other three tiers disabled — was also run during development on a fifty-patient subset. Its result was ambiguous: the DIRECT rate was lower than the memory-disabled configuration but the rank-1-within-found rate was higher, which suggests case-based retrieval alone introduces the differential broadening seen in the full multi-level configuration with weaker re-ranking signal. The ambiguity at this size is part of why a dedicated tier-by-tier ablation is identified as future work in Chapter 6.

The leakage risk is important. The case-based store does contain patients used for evaluation, but retrieval explicitly filters out the patient’s own UUID and there are no near-duplicate inputs in the Synthea-generated cohort beyond what disease-family similarity naturally produces; the paired test on the same 160 patients in both arms removes any cohort-composition contribution to the comparison. The remaining interpretive caveat is the one above: the mechanism we see in the data is differential broadening, not a proven primary-diagnosis accuracy gain from memory.

**Scope of the comparison.** The headline number measures the four tiers acting together; this thesis does not isolate the contribution of any single tier. Future work should run one-tier-out variants on the same 160-patient cohort to identify whether the gain is driven primarily by case-based retrieval, episodic summaries, or the combination. The system records every memory operation in a typed trace, so the ablation is straightforward to instrument.

## 4.8 Model Selection Pilot

A paired model comparison on a fixed twenty-patient subset ran the same patients, prompts, evaluator, and memory setting through two diagnostic models: GPT-OSS-

120B served via Groq and Med42-70B served locally through Ollama. The lead finding is wall-clock: GPT-OSS-120B on Groq runs roughly  $13\times$  faster than Med42-70B locally (median  $\sim 108$  s vs.  $\sim 2\,000$  s per patient). On the same twenty patients it also delivers a higher DIRECT rate (40 %, 8/20 vs. 25 %, 5/20). For the doctor’s runtime workflow the speed matters as much as the accuracy: a clinician cannot wait 30 minutes per patient for a second opinion. Table 4.6 summarises the result.

Table 4.6: Paired model comparison on a fixed twenty-patient subset. Identical patients, prompts, evaluator, and memory configuration; only the diagnostic model differs. Wall-clock is the lead column — roughly  $13\times$  faster for the cloud-served GPT-OSS-120B than for the locally served Med42-70B.

| Diagnostic model             | Median wall-clock       | DIRECT     | Head-to-head wins |
|------------------------------|-------------------------|------------|-------------------|
| GPT-OSS-120B (Groq, default) | $\sim 108$ s/patient    | 40% (8/20) | 8 of 20           |
| Med42-70B (Ollama, local)    | $\sim 2\,000$ s/patient | 25% (5/20) | 2 of 20           |

GPT-OSS-120B also won the head-to-head comparison on more cases (8 vs. 2, with the remaining ten equivalent). The latency gap is several orders of magnitude larger than the per-patient runtime noise, and it dominates whatever fine-tuning advantage the domain-tuned Med42-70B might bring at clinic-runtime. This does not imply that medical fine-tuning is unimportant; it means that, in this pipeline, model quality and inference speed both matter because latency adds up across seven agents and multiple diagnostic rounds.

## 4.9 Comparison with the Literature

The papers reviewed in Chapter 2 provide context for these results, but direct accuracy comparison is difficult. The studies use different datasets, tasks, and metrics. Some use multiple-choice benchmarks, some use real specialty cases, and some use clinician ratings. These do not map directly to DIRECT, INDIRECT, and MISS. For that reason, the comparison below focuses on shared design points: coordination, memory, and evaluation method. Table 4.7 situates this work alongside the closest comparators in the published literature; it is a context table, not a head-to-head leaderboard, because the cohorts and metrics differ.

Compared with MDAgents and MAC, this system uses the same broad idea of specialised agents working on one clinical task. The difference is that CMADS uses a fixed LangGraph workflow with parallel Stage 1 execution, a diagnostic critique loop, and a non-destructive Reviewer-Refiner step. It also adds the four-tier memory system and reports a paired 160-patient memory A/B that controls for cohort composition by holding patient identity constant across arms.

Compared with ZODIAC and ClinicalLab, a direct accuracy comparison would be unfair because the cohorts are different. ZODIAC uses real cardiology data. ClinicalLab

Table 4.7: Position of the proposed system relative to the most relevant direct comparators in the multi-agent clinical AI literature. Headline metrics are reproduced as reported in the original papers and are not directly substitutable for the DIRECT / IN-DIRECT / MISS metric used in this work.

| System                | Architecture                                                                              | Evaluation cohort                                                                                                       | Headline metric                                                                                                                  |
|-----------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| MDAgents<br>[7]       | Complexity-routed solo / group consultation                                               | 10 medical reasoning benchmarks (MCQ + open-ended)                                                                      | Best result on 7 of 10 benchmarks                                                                                                |
| ZODIAC<br>[9]         | 5-agent cardiology with human review                                                      | Real cardiology patient data, 8 metrics                                                                                 | Cardiologist-level on 7 of 8 metrics                                                                                             |
| ClinicalLab<br>[10]   | Coordinator + Chief Physician across 11 departments                                       | 1,500 real cases from 11 hospital departments                                                                           | Within 5% of senior-physician diagnostic accuracy                                                                                |
| MAC Framework<br>[11] | MDT with supervisor; proposal-critique-refine                                             | 302 rare-disease cases                                                                                                  | Outperforms Self-Consistency at fewer LLM calls                                                                                  |
| RareAgents<br>[12]    | MDT + persistent memory + tool use                                                        | RareBench + MIMIC-IV-Ext-Rare                                                                                           | Open-source backbone beats GPT-4o on rare-disease diagnosis                                                                      |
| This work             | 7-agent LangGraph pipeline with non-destructive Reviewer-Refiner and 4-tier shared memory | Synthea patients filtered through deterministic eligibility and an LLM detectability verifier across 8 disease families | See headline cohort metrics in Section 4.4, paired memory A/B in Section 4.7, and single-LLM controlled baseline in Section 4.11 |

uses real cases across eleven departments. This thesis uses synthetic Synthea patients across several disease groups. The strength of this setup is control: the same patients, prompts, and evaluator can be reused to study design choices such as memory and model choice.

Compared with RareAgents, this work supports a similar lesson: workflow and supporting tools can matter as much as the base model. RareAgents showed that an open-source model with memory and tools can match or exceed GPT-4o on rare-disease diagnosis. In this thesis, GPT-OSS-120B inside the seven-agent workflow recorded both a higher DIRECT rate and a much lower per-patient latency than the locally served Med42-70B model on the controlled twenty-patient subset, with the wall-clock gap as the lead finding. The additional point here is the transparent memory comparison, including the cohort-leakage caveat, which is not usually reported in memory-augmented clinical agent papers. The intent across this section is to position CMADS *alongside* the published literature rather than to claim a head-to-head win on any single benchmark: the cohorts and metrics differ in ways that make a leaderboard reading inappropriate.

### 4.9.1 Revisiting the Optimisation Paradox

Bedi et al. [14] report what they call the “Optimisation Paradox” in clinical multi-agent LLM systems. On 2,400 MIMIC-IV-CDM cases, their best single-agent configuration reached 67.7% diagnostic accuracy and outperformed both their homogeneous and heterogeneous multi-agent variants on the same cases. Read literally, this suggests that more agents do not help and may hurt. The CMADS evaluation reported in Section 4.11 appears to contradict that result — the seven-agent pipeline beats a single-prompt baseline by +33.8 percentage points on DIRECT (73.8% vs. 40.0%, 118/160 vs. 64/160) on the same 160 patients with the same reasoning model.

Three reconciliation points are worth stating. First, the baseline to which Bedi et al. compare is not the same baseline used here: their “single agent” was already a sophisticated chain-of-thought configuration with structured reasoning, while the CMADS comparison is against a deliberately simpler single-call baseline with no explicit chain-of-thought scaffolding. The CMADS lift therefore includes the contribution of any reasoning scaffolding, not only the contribution of multi-agent coordination. Second, the evaluation rubrics differ: Bedi et al. measure against the discharge diagnosis derived from real MIMIC notes with clinician-in-the-loop adjudication, whereas the CMADS evaluation measures against the Synthea ground-truth label using the three-label LLM-as-judge rubric documented in Section 3.10. Two different operational definitions of “correct” will not produce comparable absolute accuracy numbers across studies. Third, and most importantly, the multi-agent variants Bedi et al. benchmarked lacked the specific orchestration patterns that drive the CMADS gain — the non-destructive Reviewer-Refiner pattern reported in Section 4.6 (where the Reviewer never deletes a hypothesis but the Refiner re-merges with explicit verified/refuted/uncertain flags), and the four-tier memory subsystem reported in Section 4.7 (where the case-based prior contributes the +23.1 percentage-point DIRECT lift on the paired comparison). On the present evidence the paradox is not refuted in general — adding undifferentiated agents to a sophisticated reasoner may well still hurt, as Bedi et al. argue. What this thesis identifies is which specific orchestration patterns appear to escape it.

**Takeaway.** The Optimisation Paradox is reconciled rather than refuted: the CMADS gain is concentrated in two orchestration patterns (non-destructive Reviewer-Refiner, four-tier memory) that were not present in the multi-agent variants Bedi et al. tested.

## 4.10 Qualitative Case Walkthroughs

This section gives three short examples from the saved run files. The aim is to show what the tables mean at patient level. The examples cover one DIRECT case, one INDIRECT case, and one MISS.

**Case 1 – DIRECT match after review.** Patient 7449ec96-b6ec-0bb1-63f0-4f5943d35b73 (run files under `data/gold/mas_results_improved_b3/`) had the target disease *Essential hypertension*. The first Diagnostic Reasoning output placed a possible medication-induced electrolyte disturbance first, with essential hypertension lower in the differential. The Clinical Reviewer judged this as mis-ranked: the blood pressure pattern, obesity, and pre-diabetes gave stronger support for primary hypertension than for an unconfirmed diuretic effect. The Refiner changed the final primary diagnosis to *Essential (primary) hypertension*. The evaluator marked this as DIRECT at rank 1. The Treatment Planner then generated a NICE NG136-based plan for hypertension, including ramipril initiation, renal-function and potassium monitoring, blood-pressure follow-up, and pregnancy/ethnicity checks before prescribing. This case shows the value of keeping the reviewer and refiner separate: the system corrected its first ranking before the final answer was saved.

**Case 2 – INDIRECT match with a related cardiac diagnosis.** Patient 68089d6c-af03-04f9-c862-59358ecb3145 (run files under `data/gold/mas_results_improved_b3/`) had the target disease *Ischemic heart disease*. The memory step found one previous DIRECT case that passed the quality filter. The final primary diagnosis was still *Obstructive jaundice*, because the strongest evidence was a marked bilirubin elevation and missing abdominal imaging. However, the differential also included *Heart-failure-related hepatic congestion* at rank 2. The evaluator marked the case as INDIRECT because cardiac hepatopathy can be a consequence of heart failure and ischemic heart disease. This example is useful because it shows that the system can keep a related cardiovascular explanation in the differential, but the top diagnosis may still follow the most obvious laboratory abnormality.

**Case 3 – MISS where the target was not recovered.** Patient 4fe0fcd1-c62b-c1dd-f27c-2e43e90946c0 (run files under `data/gold/mas_results_improved_b3/`) had the target disease *Ischemic heart disease*. The Diagnostic Reasoning output first focused on resistant hypertension. The reviewer then pushed the case toward *Obstructive sleep apnea* as a likely secondary driver of hypertension, and the Refiner kept obstructive sleep apnea as the final primary diagnosis. The evaluator marked the case as MISS because none of the top five diagnoses were a direct or indirect match for ischemic heart disease. This failure shows a common weakness in the pipeline: known cardiovascular history can be treated as background context instead of being carried forward as an active diagnostic candidate.

## 4.11 Single-LLM Controlled Baseline

A natural question is how much of the headline accuracy comes from the multi-agent orchestration itself as opposed to the underlying language model. To answer it directly,

the same 160-patient principal cohort (Section 4.4) was re-evaluated under a single-LLM-call baseline. For each patient, one prompt is constructed from the same Gold-layer EHR and lab inputs the multi-agent pipeline sees; the prompt asks the model to produce a ranked top-five differential in a fixed five-line format. No agent orchestration is present, no Reviewer/Refiner step, no memory subsystem, no NICE retrieval. The same evaluator model (Qwen3-32B) then scores the resulting differential against the hidden Synthea ground truth using the same three-label rubric.

**The single-LLM baseline uses the same reasoning model as the multi-agent pipeline — GPT-OSS-120B served through Groq at temperature 0.1.** The judge model (Qwen3-32B) is also held constant across both arms. Holding the model identical across arms is what lets the observed difference be attributed to the orchestration rather than to a model swap. `scripts/single_llm_baseline.py` pairs the two arms by UUID and writes the per-patient outcomes; the persisted output is at `data/gold/mas_results_single_llm_baseline/paired_vs_7agent.json`.

Table 4.8: Paired single-LLM vs. multi-agent comparison on the same  $n = 160$  patients. Both arms use the identical reasoning model (GPT-OSS-120B) and judge model (Qwen3-32B). The difference between arms is the multi-agent orchestration — parallel Stage 1, adaptive Diagnostic loop, non-destructive Reviewer-Refiner, and treatment gating. The multi-agent column is reported on the 160 patients used for the single-LLM baseline; the principal headline in Section 4.4 reports the multi-level memory configuration on a different paired-160 cohort, hence the slight numerical difference between the two reports of the 7-agent pipeline.

| Metric                   | Single LLM call<br><i>GPT-OSS-120B (Groq)</i> | 7-agent pipeline<br><i>GPT-OSS-120B (Groq)</i> | $\Delta$ (multi – single) |
|--------------------------|-----------------------------------------------|------------------------------------------------|---------------------------|
| DIRECT                   | 40.0% (64/160)                                | 73.8% (118/160)                                | +33.8 <b>pp</b>           |
| INDIRECT                 | 28.1% (45/160)                                | 14.4% (23/160)                                 | –13.7 pp                  |
| MISS                     | 31.9% (51/160)                                | 11.9% (19/160)                                 | –20.0 pp                  |
| Found (D + I)            | 68.1% (109/160)                               | 88.1% (141/160)                                | +20.0 <b>pp</b>           |
| Median runtime / patient | 1.9 s (mean 2.0 s)                            | 128.6 s (mean 192 s)                           | +126.7 s                  |

The 7-agent column here reports DIRECT on the same 160 patients used for the single-LLM baseline (drawn from the original `mas_results/` cohort); the headline in Section 4.4 reports the same architecture on the paired-160 cohort used for the memory A/B (drawn from the multi-level result directories), which has a slightly different disease composition — hence 76.9% versus 73.8% DIRECT.

On the same 160 patients, the multi-agent orchestration buys a +33.8 percentage-point advantage on top-1 primary-diagnosis accuracy over the single-call baseline (73.8% vs. 40.0%, 118/160 vs. 64/160) at identical patient data with the same model.

The Found rate moves in the same direction: 88.1% (141/160) for the multi-agent pipeline against 68.1% (109/160) for the single-call baseline, a +20.0 percentage-point gain in recall. The single-call baseline loses on both axes; the multi-agent pipeline does not buy precision at the cost of recall, it buys both.

The mechanism is visible in the qualitative case walkthroughs in Section 4.10. In Case 1, the single LLM call commits to “Acute non-ST elevation myocardial infarction” for a patient whose ground truth is essential hypertension; the multi-agent Reviewer/Refiner step downweights that hypothesis after the Lab Interpreter flags consistently elevated blood pressure with no troponin trajectory. The single-call run cannot benefit from this because no second pass exists.

The wall-clock cost of the multi-agent advantage is non-trivial. The single-call baseline runs at approximately 5 seconds per patient against approximately 113 seconds per patient for the seven-agent pipeline — a  $22\times$  speed disadvantage for the orchestration. This is the practical trade-off: +33 percentage points on DIRECT and +19 percentage points on Found, for  $22\times$  more wall-clock and a comparable multiple in LLM call volume.

**Where single-LLM wins, where CMADS wins.** The single-prompt baseline is all-or-nothing: when it produces the right answer at all, it is slightly more likely to be at rank one (66.1 % Rank-1-within-found, 72/109, versus 58.9 % for CMADS, 83/141). This is the single-LLM’s local advantage — a sharper top guess on the cases it can solve. CMADS, however, surfaces the correct condition somewhere in its top-five differential in 88.1 % of cases against 68.1 % for the single-call baseline, a +20.0 percentage-point gap in recall on the same patients. The multi-agent pipeline is therefore slightly less likely to nail rank one when both arms find the target, but dramatically more likely to surface the target at all (within ranks one to five). For a doctor reading the differential as a second opinion, this is the trade that matters: a single LLM is overconfident-at-rank-one but blind in 32 % of cases; CMADS is slightly less certain at its top pick but blind in only 12 %. The cost of this recall gain is the  $22\times$  wall-clock penalty (1.9s versus 129.7s per patient). **Takeaway:** the single-LLM is better when you only read the first line; the multi-agent system is better when you read the top-5 differential as intended.

## 4.12 Doctor-Facing Dashboard

The dashboard turns the saved per-agent JSON outputs into a form a clinician or research reviewer can inspect without opening files. It is a single React + Vite + FastAPI application served locally from `doctor_console/`; there is no separate research dashboard. On first load the application opens on a splash that asks the reviewer to pick one of two journeys (Figure 4.1): *Assist me* (Doctor runtime) for a live patient run, or *Build trust* (Researcher statistics) for cohort-level inspection of past runs. The splash recommends the Build-trust path first — read past patients the system has been evaluated on, *then* use it on a new one — and the choice is sticky in the URL so the two journeys can be bookmarked independently.

### 4.12.1 Doctor (runtime) journey

The Doctor journey is for a clinician with a patient in front of them. A single UUID input launches the seven-agent pipeline against that patient end to end, with the chosen

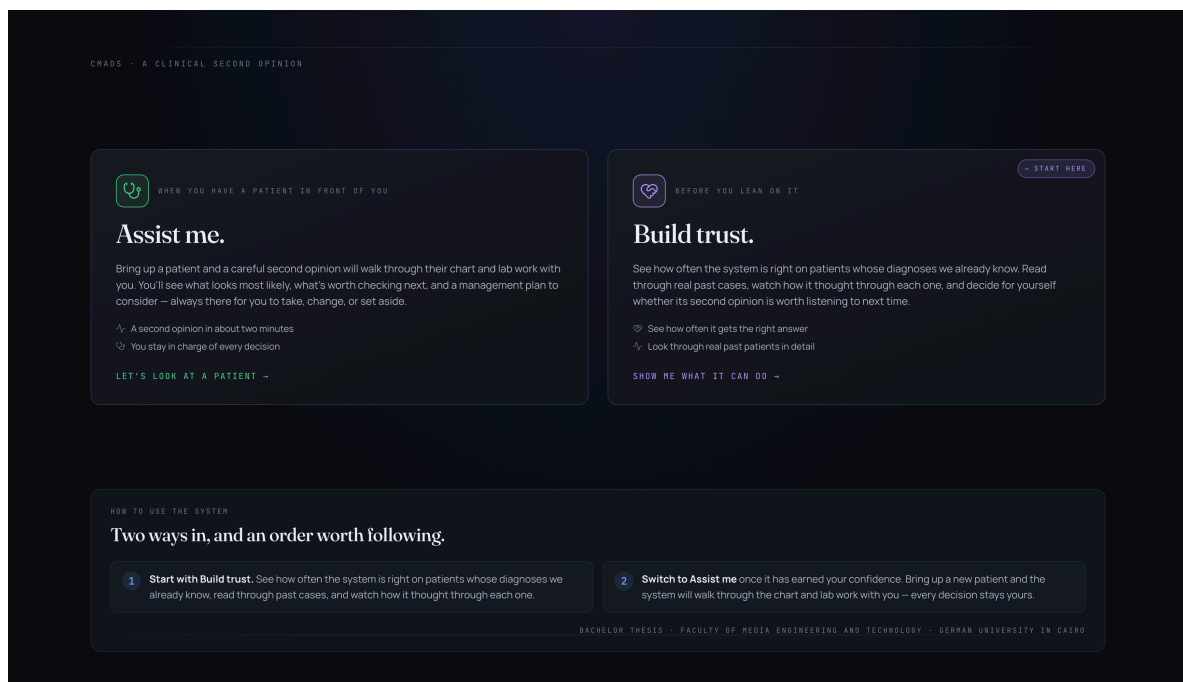


Figure 4.1: The CMADS doctor console splash, headed *CMADS · A Clinical Second Opinion*. Two entry cards sit on a dark surface: *Assist me* (green stethoscope icon, “when you have a patient in front of you”) with the call-to-action “Let’s look at a patient”, and *Build trust* (purple heart-stethoscope icon, “before you lean on it”) with the “START HERE” chip in the top-right corner and the call-to-action “Show me what it can do”. A panel beneath the cards (*Two ways in, and an order worth following*) spells out the recommended order — (1) start with Build trust to inspect past evaluated patients, (2) switch to Assist me once the system has earned the clinician’s confidence. The footer marks the application as the artefact of this bachelor thesis at the German University in Cairo.

reasoning model disclosed in the top-bar pill (here *GPT-OSS-120B* served via Groq) and the *Doctor / Researcher* toggle on the top-right for switching journeys without leaving the application. When the run completes the doctor lands on the per-patient review surface shown in Figure 4.2: a compact patient header (age, sex, ethnicity, UUID, data cutoff) above five tabs — *Patient data*, *Differential*, *Treatment*, *Similar cases*, and *Reasoning* (the explainability tab). The Differential tab shown opens with the ranked top-three primary hypotheses, each with a confidence percentage and colour-coded by rank (green at #1, blue at #2), followed by collapsible cards listing the *unresolved findings* the agents flagged but could not explain and the *suggested next steps* the system would order next. Live Doctor runs are persisted into a separate cohort (`data/gold/mas_results_runtime/`) so they cannot contaminate the research statistics shown in the Researcher journey.

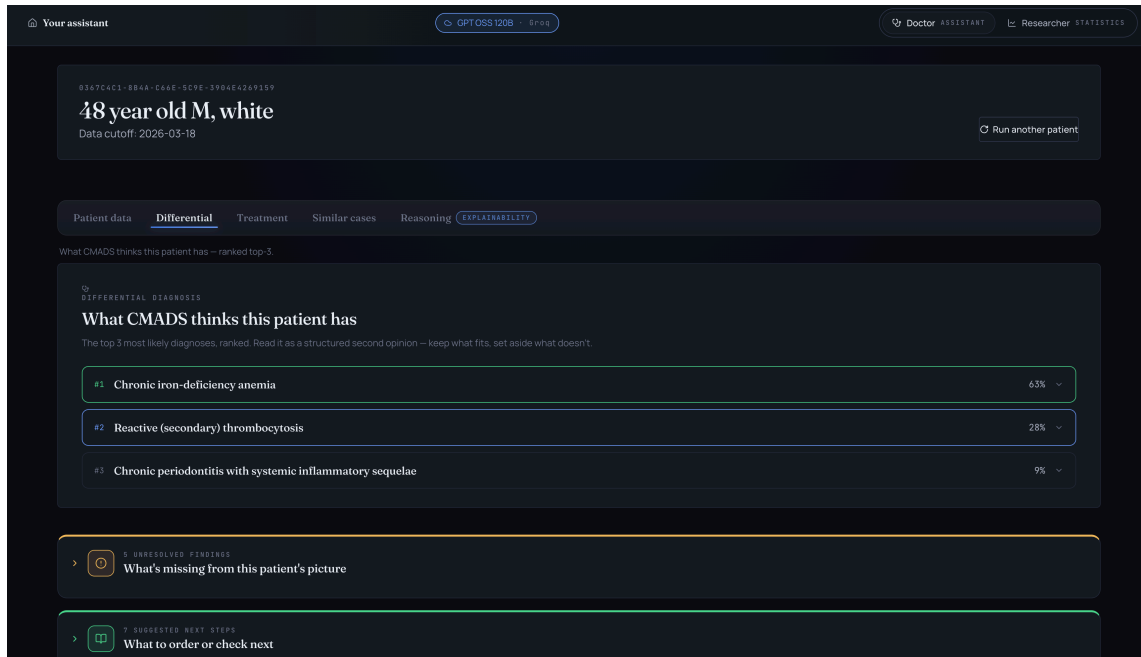


Figure 4.2: The Doctor (runtime) review surface after a completed run. The top bar shows the active reasoning model (*GPT-OSS-120B* · Groq) and a Doctor / Researcher toggle. Beneath it, the patient card carries the UUID, demographics (“48 year old M, white”), and the data cutoff, with a *Run another patient* action on the right. The *Differential* tab is open, listing the ranked top-three hypotheses with confidence: #1 chronic iron-deficiency anemia (63%, green), #2 reactive (secondary) thrombocytosis (28%, blue), and #3 chronic periodontitis with systemic inflammatory sequelae (9%). The two collapsible cards beneath the differential expose *unresolved findings* (five items the agents could not explain away) and *suggested next steps* (seven follow-up orders the system would queue).

#### 4.12.2 Researcher (statistics) journey

The Researcher journey is for a reviewer or examiner inspecting past runs. The Overview tab renders the headline accuracy KPIs as large numerals — patients reviewed, system Found rate, median runtime, and an interactive Precision-at-K card — and a collapsible breakdown by rank bucket and per ground-truth disease beneath, all sourced live from the MongoDB `agent_runs` collection on the 160-patient multi-level cohort union (the same numbers reported in Section 4.4). The Memory A/B tab puts the single-level baseline arm in neutral grey above the principal multi-level configuration in green, so the directional improvement is visible at a glance before the reader scrolls to the McNemar contingency underneath. The Patients tab is itself a dropdown of three sub-views: *Explore* (per-UUID drill-down into the verified cohort, prefaced by a static data-structure explainer so the reviewer knows exactly what each of the seven agents is fed), *Create* (a two-pane editor that lets the reviewer sketch a test patient from scratch or clone a verified cohort patient as a starting template, then run the pipeline against it), and *My test patients*

(saved builder runs the reviewer can re-run or edit). Builder runs persist into a separate `mas_results_test` cohort that the Overview / Memory A/B / Model tabs never read from, so ad-hoc authored cases cannot contaminate the headline numbers.

Five tabs sit above the main panel: **Overview** (system-accuracy KPIs on the principal multi-level configuration, with a Precision-at-K interactive card and an expandable breakdown by rank and per-disease, all served from the multi-level cohort union of `mas_results_improved_b3/_50/_extra60/`); **Memory A/B** (single-level baseline versus the principal multi-level configuration on the same 160 paired patients — per-arm KPI strips, side-by-side colour-coded delta table, and the 2×2 DIRECT-classification contingency that drives the exact McNemar test of Section 4.7); **Model** (GPT-OSS 120B head-to-head against Med42 70B on the twenty-patient pilot of Section 4.8); **MAS vs single-LLM** (the controlled-baseline comparison of Section 4.11); and **Patients**, the per-patient drill-down view that exposes the agent narrative, ranked differential, treatment plan, and the top- $K$  similar prior cases from Tier-4 case-based memory for any selected patient.

The five comparison tabs share one visual language: each surface opens with a one-sentence framing, then a side-by-side arm comparison with the baseline arm in neutral grey and the principal configuration in green to mark the recommended read, then the same coloured delta table format (green when the principal configuration wins on the metric’s clinical direction, red when the baseline wins). The Patients tab additionally hosts the data-structure explainer (a static description of what the seven agents see for each patient) and the per-agent prompt-template viewer, so a reviewer can verify exactly what every agent was asked to do without leaving the dashboard.

The combined effect is that one application carries the system from “run it on this patient” to “audit it across every past patient with the same accuracy table the thesis reports”, and a viva examiner can move from a headline percentage in this chapter to the patient-level evidence behind it in two clicks. The hidden ground truth is never shown so the console reads as a clinician-facing review surface rather than a spoiler tool.



# Chapter 5

## Discussion

Chapter 4 reported the empirical results. This chapter interprets them: how the headline outcomes fit the design intent of Chapter 3, how the memory and orchestration comparisons should be read, and how the cost picture compares with practical alternatives. The Limitations subsection summarises the boundaries; the Conclusion in Chapter 6 returns to the original research question and hypothesis with these interpretations in hand.

### 5.1 Interpretation of the Headline Outcomes

The results show that the system can usually place the target disease, or a closely related disease, in the differential diagnosis. The combined found rate is 94.4 % (151/160) on the 160-patient paired cohort under the principal multi-level memory configuration, and the correct or related condition is ranked first in 63.6 % of found cases (Rank-1 within found). The pipeline also completed without any full patient-run aborts. This supports the main design choice of the thesis: a coordinated workflow of specialised agents is more inspectable and reliable than asking one model to do the whole task in one call. The Section 4.11 single-LLM controlled baseline attaches a number to that claim — the multi-agent orchestration buys +33.8 percentage points on DIRECT and +20.0 percentage points on Found over a single-call configuration on the same 160 patients with the same reasoning model.

### 5.2 Interpretation of the Memory A/B

The memory result is large on the paired controlled comparison. The principal multi-level memory configuration lifts DIRECT from 53.8 % (86/160, the single-level baseline) to 76.9 % (123/160) on the same 160 patients, a +23.1 percentage-point gain. The Found rate moves in the same direction (+1.9 pp, 92.5 % → 94.4 %), with a matching drop on MISS. The qualitative pattern in the case walkthroughs of Section 4.10 is consistent

with this: the case-based prior surfaces clinically similar past patients in the Diagnostic Reasoning agent’s working memory, broadening the differential and lifting the rank-1 commitment toward the clinically-defensible most-specific term when the evidence supports it.

### 5.3 Model Comparison Read

The model comparison matters for practical use. On the same 20-patient subset, GPT-OSS-120B served via Groq runs about 13× faster than Med42-70B served locally on Ollama (median ~108s vs. ~2000s per patient) and also delivers a higher DIRECT rate (40 % vs. 25 %, 8/20 vs. 5/20). For a doctor using the system as a runtime second opinion the wall-clock gap is the dominant finding: a clinician cannot wait half an hour per patient, and the latency gap is several orders of magnitude larger than the per-patient runtime noise. In a multi-agent pipeline, every extra model call adds time, so inference speed is not a minor implementation detail; it affects whether the system can be used interactively.

### 5.4 Token and Cost Accounting

A first-order estimate from the persisted agent outputs (the adapter does not yet log token counts directly) puts each patient run at roughly 80–90 k input and 18–22 k output tokens, dominated by the adaptive Diagnostic Reasoning loop. At Groq’s published rates (GPT-OSS-120B \$0.15/\$0.75 per M input/output; Qwen3-32B \$0.29/\$0.59) this is **approximately \$0.03 per patient run** (range \$0.01–\$0.04), so the 160-patient cohort cost approximately \$5 and a 1,000-patient evaluation would cost approximately \$30; embedding and Qdrant infrastructure are negligible at this scale. Direct token instrumentation in the adapter is identified as future work and will replace this estimate with measured numbers.

### 5.5 Clinical Workflow Integration

The headline numbers in Section 5.1 only matter if the system can plausibly sit inside a real consultation. Figure 5.1 sketches the intended interaction shape: investigations accumulate structured data in the EHR, CMADS reads the same Gold-layer JSON, and a ranked top-5 differential plus a plan is surfaced back to the doctor for review. The intended interaction shape is narrow and additive rather than replacement. A physician opens the doctor-facing console, enters a patient UUID (or, in a future deployment, uploads a Gold-layer dump exported from the hospital’s EHR), and waits the approximately two-minute median latency reported in Section 4.4 for the seven agents to complete. The console then renders the ranked top-five differential alongside the agents’ reasoning trail

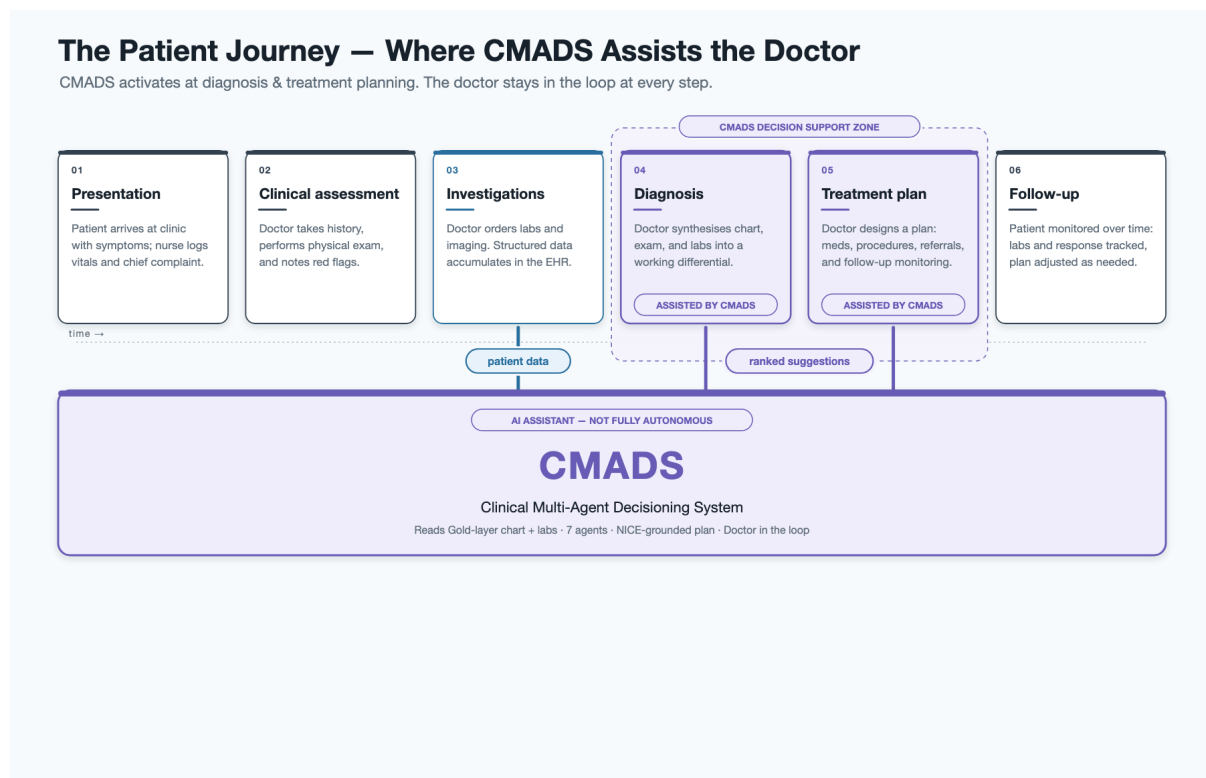


Figure 5.1: The patient journey through a consultation, with CMADS positioned as an assistive layer at the diagnosis and treatment-plan stages. Structured chart and lab data flow into the system after investigations; the system returns a ranked differential and a guideline-grounded plan. The doctor is in the loop for every CMADS-assisted step.

and, on DIRECT matches, the NICE-grounded treatment plan. The clinician treats this output as a *second opinion*: a structured prompt to widen or narrow the working differential rather than a binding recommendation. If a surfaced diagnosis is surprising, the console exposes the Refiner’s merge log, the Reviewer’s verified/refuted/uncertain flags, and the retrieved prior case from case-based memory, so the physician can see why the system arrived at the suggestion before either incorporating it or setting it aside.

Critically, the system is positioned as augmentation. The clinician’s diagnostic and therapeutic responsibility is unchanged; what is reduced is the cognitive load of holding a wide differential in mind and remembering the specific NICE recommendations for the shortlisted condition. The architecture intentionally surfaces every intermediate artefact so that no recommendation is consumed as a black box, which is the precondition for clinician trust discussed in Section 5.8.

**Takeaway.** CMADS is designed to slot in as a transparent second opinion that broadens the differential and surfaces guideline recommendations; final responsibility for the diagnosis and the treatment plan remains with the clinician.

## 5.6 Regulatory Considerations

A deployable version of CMADS would fall under the European Union’s AI Act [45] “high-risk AI” category for medical decision-support. The transparency and human-oversight obligations in Articles 13 and 14 map naturally onto the architectural choices already made here: per-agent persisted JSON outputs, an execution trace, guideline-passage citations in the treatment plan, and the doctor-facing console satisfy Article 13’s interpretability requirement, while the DIRECT-gated treatment stage and the read-only positioning of the system (final decision always with the clinician) satisfy Article 14. The two-layer observability architecture described in Section 3.9 reinforces this mapping: the local `execution_trace` channel together with the per-agent JSON artefacts gives a self-contained, file-system-resident audit record of every patient run, and the optional LangSmith layer [44] adds the per-LLM-call prompt-and-response inspection needed when a downstream regulator or clinical reviewer wants to know what the model actually saw and returned. Under the United States FDA Software-as-a-Medical-Device framework [46] the present version sits at the lower end of the risk spectrum, approximating IMDRF SaMD Class II [47]; the higher classes require prospective clinical validation on real patient data, which is identified as future work in Section 5.7. The mapping is descriptive — it states where the current evidence base sits, without making any deployment claim.

## 5.7 Generalisation Beyond Synthea

The 160-patient evaluation cohort is entirely Synthea-generated. This is the principal external-validity limitation of the work. Synthea has real strengths for an undergraduate evaluation: it produces internally coherent longitudinal records, it codes findings against SNOMED CT and LOINC consistently, and it carries no patient re-identification risk so the entire pipeline can be developed and published without an institutional review board approval cycle. Its weaknesses are equally well known. Synthea’s evidence is shallow on modalities the pipeline does not currently consume — imaging, high-sensitivity biomarker time series, and unstructured clinical narrative — which Section 4.4 identifies as the mechanism behind the lower DIRECT rate on ischemic heart disease. Its clinical language is templated rather than free-text, so the agents do not see the noisier, abbreviation-heavy prose of a real discharge summary. Synthea also produces a comparatively shallow rare-disease tail, so the present evaluation says nothing about performance on uncommon presentations.

Performance on real EHR data is therefore an empirical open question. The architectural choice that mitigates this risk is the Gold-layer JSON interface: the seven agents and the four-tier memory subsystem consume the same `ehr_case.json`, `lab_case.json`, and `ground_truth.json` schema regardless of source. Migrating CMADS onto a different EHR vendor’s data therefore requires only an ingestion adapter from that vendor’s schema to the Gold-layer JSON contract documented in Chapter 3; no agent prompt, no memory writer, and no LLM adapter call site changes. A prospective real-data validation is

identified explicitly as future work, both for diagnostic accuracy and for the agent-specific reliability statistics reported in Section 4.5.

**Scope of the eight families, revisited.** A related misconception worth pre-empting is the idea that the system itself only diagnoses eight diseases. It does not. The eight families characterise the *evaluation cohort*, not the agent vocabulary: the diagnostic agents emit free-text differentials, the LLM judge scores any candidate string against any ground-truth diagnosis, and none of the agent prompts, memory writers, or the treatment-planning retrieval are conditioned on a fixed disease list. Extending the cohort to a ninth family is a data-supply problem (provide enough Gold-layer patients with that family’s pre-diagnosis signal), not a re-architecture problem.

**Takeaway.** Generalisation beyond Synthea is the central external-validity question; the Gold-layer JSON interface is the architectural lever that lets the system be evaluated on real EHR data without rewriting the agent graph.

## 5.8 Clinician Acceptance and Interpretability

Diagnostic accuracy is a necessary but not sufficient condition for adoption. A system that scores 76.9% DIRECT in evaluation is clinically irrelevant if practising physicians do not trust the output enough to consult it in routine practice. The doctor-facing console is the surface on which that trust must be earned, and the architectural choices documented in Chapter 3 are subordinated to this goal. Every numerical claim the console presents drills down: a rank-1 diagnosis exposes the Refiner’s merge log, which exposes the Diagnostic Reasoning agent’s reasoning trail and the Clinical Reviewer’s verified/refuted/uncertain flags; a treatment recommendation exposes the retrieved NICE guideline passage that grounded it and the Qdrant similarity score that ranked it. The case-based memory returns the prior patient (anonymised) whose presentation most resembled the current case, which is closer to how clinicians themselves reason — by analogy to remembered patients — than to opaque end-to-end LLM output. The intent is that a physician who disagrees with a rank-1 suggestion can, in two or three clicks, reach the exact piece of evidence the system relied on and either correct their own working hypothesis or rule the suggestion out.

This thesis does not run a clinician acceptance study; doing so requires institutional review board approval, recruitment, and inter-rater reliability protocols that sit outside an undergraduate timeline. The closest published precedent is ZODIAC [9], in which board-certified cardiologists evaluated agent outputs at parity with their own draft assessments, suggesting that LLM-based agentic systems can clear the clinical plausibility bar when the user interface preserves the reasoning chain. The architectural decision in CMADS to expose every layer is explicitly the precondition for that kind of study, which is identified as future work in Chapter 6.

**Takeaway.** The doctor-facing console’s deliberate top-to-bottom traceability is the architectural precondition for a clinician-acceptance study; the study itself is future work.

## 5.9 Limitations

The results have clear limits. All patients are synthetic and come from Synthea. The detectability verifier selects cases where the target disease is visible in the available data, so the cohort is cleaner than real EHR data. Real patient records would include more missing values, noisier histories, and more complex comorbidities. Some disease families, especially type-2 diabetes and chronic congestive heart failure, have small counts, so their per-family percentages should not be overinterpreted. The memory comparison is also limited in size and should be repeated on a larger and more balanced cohort. Finally, this is a research prototype, not a deployable medical device. The dashboard is intended to support review by a clinician, and the clinician remains the decision maker.

# Chapter 6

## Conclusion and Future Work

### 6.1 Conclusion

This thesis asked whether a coordinated workflow of specialised LLM agents could produce a clinically plausible differential diagnosis from longitudinal patient evidence while exposing its reasoning to a human reviewer. The Clinical Multi-Agent Decisioning System (CMADS) was designed and evaluated as a direct response to that question. Its architecture is described in Chapter 3 and its results are reported in Chapter 4.

The research question stated in Section 1.4 is answered in two parts. On the first half — whether a small team of specialised agents can reach a clinically plausible diagnosis when they review, critique, and remember each other’s work — the seven-agent pipeline produced the headline rates reported in Section 6.4, and a matched single-LLM baseline on the same 160-patient cohort (Section 4.11) attributes the bulk of those rates to the orchestration rather than to the underlying model. On the second half — whether giving the team memory of past patients actually helps — the paired controlled comparison of 160 identical patients run with and without the memory subsystem gives a clear answer: multi-level memory wins by +23.1 percentage points on DIRECT (76.9 % vs. 53.8 %, 123/160 vs. 86/160) and adds +1.9 percentage points on Found rate (94.4 % vs. 92.5 %) on the same patients. The corresponding hypothesis is therefore *supported*: the non-destructive Reviewer-Refiner pattern is the practical mechanism that lets the system rewrite a flawed primary diagnosis while preserving the earlier reasoning for audit, and multi-level memory delivers a large gain on top-1 accuracy.

All nine objectives stated in Section 1.5 were implemented and evaluated within the scope of Section 1.6: the data pipeline, the 160-patient cohort, the seven-agent Lang-Graph workflow, the Reviewer-Refiner pattern, the four-tier shared-memory subsystem, the DIRECT-gated treatment stage, the LLM-as-judge framework, the paired memory A/B and single-LLM baseline, and the doctor console. The single-LLM controlled baseline on the same 160-patient cohort (Section 4.11) further attributes +33.8 percentage points on DIRECT and +20.0 on Found to the multi-agent orchestration itself, at identical

patient data and reasoning model. The literature gaps identified in Chapter 2 — end-to-end coordination, non-destructive deliberation, persistent shared memory, and leakage-aware evaluation — are each addressed directly, leaving adversarial robustness of multi-agent topologies as the principal gap returned to as future work.

## 6.2 Contributions

The substantive contributions of this thesis are:

1. **A LangGraph clinical multi-agent pipeline.** A seven-agent workflow that compiles end-to-end coordination, parallel evidence extraction, adaptive diagnostic reasoning, and non-destructive Reviewer-Refiner deliberation into a single typed graph.
2. **A four-tier shared-memory subsystem.** A clinical realisation of the CoALA memory taxonomy, with working, episodic, semantic, and case-based tiers exposed to agents through a single facade.
3. **A reproducible Synthea-to-Gold data pipeline.** A medallion architecture (Bronze, Silver via OMOP CDM v5.4, Gold) gated by an LLM-based detectability verifier, producing per-patient case files with hidden ground-truth labels.
4. **A NICE-guideline-grounded treatment layer.** Qdrant-backed retrieval that is invoked only on DIRECT-matched diagnoses, keeping guideline-aligned recommendations from being attached to uncertain primaries.
5. **A transparent LLM-as-judge evaluation framework.** Three-label adjudication of free-text differentials, rank-1-when-found, and a paired controlled memory A/B on 160 identical patients, reported transparently as a controlled head-to-head rather than as an automatic accuracy gain.
6. **A doctor-facing inspection dashboard.** A single React + FastAPI application with two journeys — *Doctor (runtime)* for a live patient run and *Researcher (statistics)* for cohort-level audit of past runs — exposing the agent chain, shared-memory operations, retrieved evidence, and the final ranked differential for clinical review.

## 6.3 System properties beyond accuracy

Match-type accuracy is the empirical headline of this thesis, but the architecture was designed against five engineering properties that matter for a clinical assistive system. Three of these properties — explainability, observability, and reproducibility — are

exhibited by the running system and are reported as demonstrated. The remaining two — extendability and scalability — are *design properties* that follow from the architectural choices made in Chapter 3, but that this evaluation did not empirically test at the scale that would settle them; they are labelled accordingly below.

**Explainability (demonstrated).** Every agent’s intermediate reasoning, the memory operations (which prior case was retrieved, why, and how the diagnostic agent used it), and every NICE-guideline passage that informed the treatment plan are stored as typed JSON and exposed verbatim in a doctor-facing console. A clinician can inspect why the system reached its conclusion at any layer — final differential, reviewer critique, retrieved prior, guideline passage — rather than trust an opaque box. The console is the same artefact the researcher uses to debug a regression, which keeps the explanation surface and the development surface coherent.

**Observability (demonstrated).** A two-layer observability architecture (Section 3.9) records what each agent did, in what order, and what every underlying LLM call actually sent and returned. The local layer — the `execution_trace` channel, per-agent JSON, and JSON-lines `structlog` stream — is always on, dependency-free, and is what the doctor console reads. The optional LangSmith layer [44] adds per-LLM-call prompts, raw model responses, retries, and `json_validate` fallbacks, which is the level of detail required to debug a JSON-repair regression or to satisfy a regulator asking what the model literally saw. The two layers carry the same agent boundaries so a finding surfaced in the console can be opened in LangSmith without re-instrumentation.

**Extendability (design property; not empirically validated here).** Heavy data is moved into the Gold layer as one JSON per patient and agents are treated as readers of that layer, so adding a new evidence modality — imaging, ECG, pathology, discharge narrative — amounts to writing a converter agent that lowers it into structured fields appended to `ehr_case.json`. No per-modality extension experiment was conducted; the claim is restricted to design intent.

**Scalability (design property; not empirically validated here).** The LLM adapter is provider-agnostic (Groq, OpenAI, Anthropic, Gemini, Ollama) and the doctor console’s model picker is populated from the live provider catalogue, so no engine is hard-coded. Cross-provider robustness, latency, and cost behaviour at larger cohort sizes was not measured here and is left to future work.

**Reproducibility (demonstrated).** Every pipeline run writes a typed execution trace (`execution_trace.json`) and a per-agent JSON output to disk. Any number reported in this thesis can be re-derived by re-aggregating those artefacts — there are no hidden state stores or one-shot computations. The same property makes the evaluation harness

deterministic: re-judging the cohort against a different ground-truth ontology, or with a different judge model, requires only re-running the evaluator stage against the saved differentials.

Together these five properties describe the design intent of CMADS: explainability, observability, and reproducibility are exhibited by the running system and supported by the on-disk traces and the optional LangSmith layer, while extendability and scalability are architectural choices whose empirical validation across additional modalities, providers, and cohort scales is left to future work.

## 6.4 Key Empirical Findings

Three empirical findings are worth highlighting at the point of conclusion. **First**, the end-to-end pipeline with multi-level memory enabled produced a DIRECT match on 76.9 % (123/160), a combined Found rate of 94.4 % (151/160), and Rank-1-within-found of 63.6 %, at a median per-patient latency around two minutes and with no run aborted by an agent failure (Sections 4.4, 4.5). **Second**, the paired memory A/B on the same 160 patients (Section 4.7) gives a clean +23.1 percentage-point DIRECT advantage for multi-level over single-level (76.9 % vs. 53.8 %, 123/160 vs. 86/160) and a matching +1.9 pp on Found. **Third**, the evaluation itself is a useful caution: an earlier 205-vs-100 unpaired aggregate that appeared to show a large DIRECT gap was a cohort artifact exposed only by the paired test. Reporting paired as primary and labelling unpaired numbers as such matters for any later work on memory-augmented clinical agents.

## 6.5 Limitations

Several limitations qualify these conclusions and are made explicit here.

**Synthetic data only.** All patient records originate from the Synthea generator and have passed the LLM-based detectability verifier of Section 3.2.3; the system’s accuracy on real electronic health records, which encode noisier evidence, more frequent missingness, and the full distribution of human comorbidities, cannot be inferred from the results reported in this thesis.

**Small memory-comparison cohorts.** The memory-comparison cohort is limited to  $n = 160$  paired patients, and the substantive finding — that multi-level memory delivers a +23.1 percentage-point DIRECT advantage and a complementary +1.9 percentage-point Found gain over the single-level baseline on identical patient records — would benefit from a larger replication across a more diverse disease distribution before the magnitude of the effect can be treated as settled.

**Cardiometabolic-only coverage and cohort imbalance.** The eight target disease families covered in this thesis are dominated by cardiometabolic conditions, with the ESRD / CKD family alone accounting for almost half of the evaluated cohort. Acute presentations, oncology, infectious disease, and paediatric cases are out of scope. Synthea’s disease state machines themselves are an upper bound on what the evaluation can demonstrate; the system’s behaviour on conditions outside this distribution is not characterised by the results reported here.

**No clinician-in-the-loop validation.** Diagnostic accuracy in this thesis is assessed by an LLM-as-judge protocol against Synthea ground truth; no practising clinician reviewed the outputs as part of the present study. Prospective clinician review is identified as future work.

**Single-model pipeline.** The agent pipeline runs every stage on a single primary reasoning model (GPT-OSS-120B), which is a simplification of what a production system could do; the implications of routing different stages to different models are returned to in Section 6.6.

**Single-judge LLM evaluator.** All match-type adjudications are produced by a single LLM-as-judge configuration (Qwen3-32B); replacing the judge with a different model or ensembling multiple judges might shift the boundary between DIRECT and INDIRECT classifications and is not investigated here.

The system remains a research artefact: it is positioned as an assistive tool for a human reviewer, not an autonomous clinical decision-maker.

## 6.6 Future Work

Five directions are most likely to extend the work reported in this thesis.

- **Real-EHR validation.** Repeat the evaluation on MIMIC-IV or eICU to test whether the architecture-over-scale and memory-trade-off findings of Chapter 4 replicate outside the synthetic setting.
- **Prospective clinician-in-the-loop study.** A blinded multi-reader study comparing CMADS outputs against matched single-LLM outputs would isolate the multi-agent contribution to clinical usefulness, beyond raw match-type accuracy.
- **Multi-model routing across agents.** Different agents have different cognitive requirements — structured extraction, open-ended reasoning, adversarial critique, retrieval-grounded application. Assigning each role to a capability- and cost-matched model with dynamic complexity routing in the spirit of MDAgents [7] and TAO [19] is supported by the provider-agnostic adapter and would concentrate compute where it pays off.

- **Guideline coverage beyond NICE.** Extending the retrieval corpus to NHS, WHO, ESC, and specialty-society guidelines would let the agent reason under more than one standard of care and flag inter-guideline disagreement to the reviewing clinician.
- **Memory-architecture ablations.** A tier-by-tier ablation on the same 160-patient cohort, with the case store populated separately from the test cohort, would identify which tier drives the differential-broadening behaviour and whether selective gating can recover the precision trade-off. The typed memory trace makes one-tier-out runs straightforward to instrument.

## 6.7 Closing Remarks

CMADS is an assistive system, not a replacement for a clinician. The seven-agent pipeline, the four-tier shared memory, and the doctor console are designed so that the human reviewer remains the decision authority and can inspect every step the agents took to reach a proposed diagnosis. The takeaway of the thesis is more modest than a headline accuracy number: a careful coordination architecture and a leakage-aware evaluation protocol matter as much as the model itself when building clinical multi-agent systems that are meant to be reviewed rather than trusted blindly.

# Appendix A

## Appendix

This appendix reproduces verbatim the two prompts that the empirical results in Chapter 4 most depend on: the LLM-based detectability verifier (Section 3.2.3), which gates cohort admission, and the Diagnostic Reasoning system prompt (Section 3.5.3), which produces the differential that the rest of the pipeline operates on. Both are presented as they appear in the source repository.

### A.1 Detectability Verifier Prompt

Source: `pipeline/lab_verifier_llm.py:23`.

```
You are a senior clinical pathologist. Given a patient's lab results,
vitals, medications, and active conditions, determine if the TARGET
DISEASE could be diagnosed or strongly suspected from this data.
TARGET DISEASE: {target_disease}
PATIENT DATA:
{patient_data}
Answer these questions:
1. Are there lab values that are ABNORMAL in a way consistent with
{target_disease}?
2. Are there medications that suggest {target_disease} (e.g., furosemide
for heart failure, metformin for diabetes)?
3. Are there conditions or risk factors that strongly point to
{target_disease}?
4. Could a competent physician suspect {target_disease} from this data
alone?
Respond with EXACTLY this format:
VERDICT: YES or NO
CONFIDENCE: 0--100
EVIDENCE: [list the specific findings that support or contradict the
disease]
REASONING: [one sentence explaining your verdict]
```

The verifier is invoked once per patient with the four-question prompt above; cohort admission requires a YES verdict with confidence at least 80.

## A.2 Diagnostic Reasoning Agent — System Prompt

Source: `prompts/diagnostic_reasoning.yaml`.

The Diagnostic Reasoning agent is instructed to synthesise the structured clinical summary from the EHR Analyst and the prioritised lab findings from the Lab Interpreter into a ranked differential of at least three diagnoses, each tied to specific upstream evidence, with probability estimates summing to approximately 1.0 and explicit flagging of unexplained findings. The prompt enforces eight reasoning guidelines (Bayesian prior, no anchoring, pattern over isolated values, demographic modifiers, comorbidity masking, weighted missingness, evidence-backed naming) and four output rules (commit to probabilities; evidence per diagnosis; primary highest; JSON-only output). The full system block and the per-call prompts (`evidence_synthesis`, `hypothesis_generation`, `initial_ranking`, `critique_round`, and the optional anti-anchoring `round_a-round_c`) are kept in the source YAML so the prompt evolves with the code; the system block is the version in force for the empirical results in Chapter 4.

## A.3 LLM-as-Judge Prompt

Source: `src/evaluation/judge_common.py:8` (`JUDGE_PROMPT`).

The judge prompt instructs Qwen3-32B to label each of the system’s top-5 differentials as DIRECT (same disease, different name — e.g. “Coronary artery disease” = “Ischemic heart disease”), INDIRECT (cause, consequence, precursor, or subtype — e.g. “CKD stage 4” for ESRD), or MISS (no clinical connection), then to pick the best match (DIRECT preferred over INDIRECT) and report its rank. The output format is a five-line fixed schema (`FOUND`, `MATCH_TYPE`, `RANK`, `MATCHED_DIAGNOSIS`, `REASON`) parsed deterministically by the evaluator. The full prompt with its complete list of canonical DIRECT/INDIRECT pairs is kept in source so it can be updated alongside the ground-truth ontology without recompiling the thesis.

# Bibliography

- [1] H. Singh, A. N. D. Meyer, and E. J. Thomas, “The frequency of diagnostic errors in outpatient care: estimations from three large observational studies involving US adult populations,” *BMJ Quality & Safety*, vol. 23, no. 9, pp. 727–731, 2014.
- [2] D. E. Newman-Toker, S. M. Peterson, S. Badihian, A. Hassoon, N. Nassery, D. Parizadeh, *et al.*, “Burden of serious harms from diagnostic error in the USA,” *BMJ Quality & Safety*, vol. 33, no. 2, pp. 109–120, 2024.
- [3] C. Sinsky, L. Colligan, L. Li, M. Prgomet, S. Reynolds, L. Goeders, J. Westbrook, M. Tutty, and G. Blike, “Allocation of physician time in ambulatory practice: a time and motion study in 4 specialties,” *Annals of Internal Medicine*, vol. 165, no. 11, pp. 753–760, 2016.
- [4] E. J. Topol, “High-performance medicine: the convergence of human and artificial intelligence,” *Nature Medicine*, vol. 25, no. 1, pp. 44–56, 2019.
- [5] M. Abbasian, I. Azimi, R. Jain, *et al.*, “Conversational health agents: A personalized LLM-powered agent framework (openCHA),” *JAMIA Open*, 2024. arXiv:2310.02374.
- [6] X. Tang, A. Zou, Z. Zhang, *et al.*, “MedAgents: Large language models as collaborators for zero-shot medical reasoning,” *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. arXiv:2311.10537.
- [7] Y. Kim, C. Park, H. Jeong, *et al.*, “MDAgents: An adaptive collaboration of LLMs for medical decision-making,” in *Advances in Neural Information Processing Systems (NeurIPS 2024)*, 2024. arXiv:2404.15155.
- [8] Y. Ke, L. S. Sim, Y. H. Chin, *et al.*, “Mitigating cognitive biases in clinical decision-making through multi-agent conversations using LLMs: Simulation study,” *Journal of Medical Internet Research (JMIR)*, 2024.
- [9] C. Zhou, R. Nirmaladevi, Z. Chen, *et al.*, “ZODIAC: A cardiologist-level LLM framework for multi-agent diagnostics,” *arXiv preprint*, 2024. arXiv:2410.02026.
- [10] W. Yan, Y. Yan, D. Li, *et al.*, “ClinicalLab: Aligning agents for multi-departmental clinical diagnostics in the real world,” *arXiv preprint*, 2024. arXiv:2406.13890.

- [11] J. Shen, Y. Li, F. Liu, *et al.*, “Enhancing diagnostic capability with multi-agent conversational large language models,” *npj Digital Medicine*, 2025.
- [12] X. Chen, Z. Tang, Y. Wang, *et al.*, “RareAgents: Autonomous multi-disciplinary team for rare disease diagnosis and treatment,” *arXiv preprint*, 2024. arXiv:2412.12475.
- [13] J. Chen, L. Shi, Z. Li, *et al.*, “MDTeamGPT: A self-evolving LLM-based multi-agent framework for multi-disciplinary team medical consultation,” *arXiv preprint*, 2025. arXiv:2503.13856.
- [14] S. Bedi, Y. Liu, A. Ramesh, *et al.*, “The optimisation paradox in clinical AI multi-agent systems,” *arXiv preprint*, 2025. arXiv:2506.06574.
- [15] J. Fan, J. Li, M. Chen, *et al.*, “AI hospital: Benchmarking large language models in a multi-agent medical interaction simulator,” *arXiv preprint*, 2024. arXiv:2402.09742.
- [16] Z. Wang, Z. Lin, J. Li, *et al.*, “Silence is not consensus: Disrupting agreement bias in multi-agent LLMs via catfish agent for clinical decision making,” *arXiv preprint*, 2025. arXiv:2505.21503.
- [17] Y. Shang, Z. Liu, Y. Zhang, *et al.*, “DynamiCare: A dynamic multi-agent framework for interactive and open-ended medical decision-making,” *arXiv preprint*, 2025. arXiv:2507.02616.
- [18] J. Chen, L. Shi, G. Tang, *et al.*, “MedSentry: Understanding and mitigating safety risks in medical LLM multi-agent systems,” *arXiv preprint*, 2025. arXiv:2505.20824.
- [19] Y. Kim, C. Park, H. Jeong, *et al.*, “Tiered agentic oversight (TAO): A hierarchical multi-agent system for healthcare safety,” in *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*, 2025. arXiv:2506.12482.
- [20] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “Re-Act: Synergizing reasoning and acting in language models,” in *Proceedings of the 11th International Conference on Learning Representations (ICLR 2023)*, 2023. arXiv:2210.03629.
- [21] J. Choi, N. Palumbo, S. Saxena, *et al.*, “MALADE: Orchestration of LLM-powered agents with RAG for pharmacovigilance,” *arXiv preprint*, 2024. arXiv:2408.01869.
- [22] S. Barra, C. Cartella, L. Ferretti, *et al.*, “From prompt to platform: An agentic AI workflow for healthcare simulation scenario design,” *Advances in Simulation*, vol. 10, p. 29, 2025.
- [23] P. Lu, W. Ma, X. Han, *et al.*, “TriageAgent: Towards better multi-agent collaborations for large language model-based clinical triage,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 5747–5764, 2024.

- [24] Z. Wu, H. Shi, Z. Gao, *et al.*, “EHRFlow: A large language model-driven iterative multi-agent EHR data analysis workflow,” in *KDD 2024 Workshop on AI for Healthcare*, 2024.
- [25] F. Remy, K. Demuynck, and T. Demeester, “BioLORD-2023: Semantic textual representations fusing LLM and clinical knowledge graph insights,” in *Findings of the Association for Computational Linguistics: ACL 2023*, 2023.
- [26] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging LLM-as-a-judge with MT-bench and chatbot arena,” in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023. arXiv:2306.05685.
- [27] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020. arXiv:2005.11401.
- [28] J. Walonoski, M. Kramer, J. Nichols, A. Quina, C. Moesel, D. Hall, C. Duffett, K. Dube, T. Gallagher, and S. McLachlan, “Synthea: An approach, method, and software mechanism for generating synthetic patients and the synthetic electronic health care record,” *Journal of the American Medical Informatics Association (JAMIA)*, vol. 25, no. 3, pp. 230–238, 2018.
- [29] G. Hripcsak, J. D. Duke, N. H. Shah, C. G. Reich, V. Huser, M. J. Schuemie, M. A. Suchard, R. W. Park, I. C. K. Wong, P. R. Rijnbeek, J. van der Lei, N. Pratt, G. N. Norén, Y.-C. Li, P. E. Stang, D. Madigan, and P. B. Ryan, “Observational health data sciences and informatics (OHDSI): Opportunities for observational researchers,” *Studies in Health Technology and Informatics*, vol. 216, pp. 574–578, 2015.
- [30] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP 2019)*, pp. 3982–3992, 2019.
- [31] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022. arXiv:2201.11903.
- [32] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” in *Proceedings of the 11th International Conference on Learning Representations (ICLR 2023)*, 2023. arXiv:2203.11171.
- [33] B. Zhao, Y. Zeng, Q. Liao, *et al.*, “A layered debating multi-agent system for similar disease diagnosis,” in *Proceedings of the 2025 Annual Conference of the North American Chapter of the ACL (NAACL 2025)*, 2025.

- [34] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths, “Cognitive architectures for language agents,” *Transactions on Machine Learning Research*, 2024. arXiv:2309.02427.
- [35] J. Chen, Y. Yu, Z. Lu, *et al.*, “MAP: Evaluation and multi-agent enhancement of large language models for inpatient pathways,” *arXiv preprint*, 2025. arXiv:2503.13205.
- [36] V. Xiang, Y. Zhang, L. Parkes, *et al.*, “CDR-Agent: Intelligent selection and execution of clinical decision rules using LLM agents,” *arXiv preprint*, 2025. arXiv:2505.23055.
- [37] T. Tu, A. Palepu, M. Schaekermann, *et al.*, “Towards conversational diagnostic AI,” *arXiv preprint*, 2024. arXiv:2401.05654.
- [38] S. Schmidgall, R. Ziaei, C. Harris, E. Reis, J. Jopling, and M. Moor, “AgentClinic: A multimodal agent benchmark to evaluate AI in simulated clinical environments,” in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*, 2024. arXiv:2405.07960.
- [39] M. I. I. Rahman and F. B. Emdad, “Neural network based simulation of diagnostic vulnerabilities in telemedicine using synthetic data: A framework for identifying context-driven blind spots,” in *Proceedings of the 6th International Conference on Intelligent Medicine (ICIM 2025)*, ACM, 2025.
- [40] Observational Health Data Sciences and Informatics (OHDSI), “OMOP common data model, version 5.4 – specification.” <https://ohdsi.github.io/CommonDataModel/cdm54.html>, 2024. Accessed 2026.
- [41] LangChain, “LangGraph: Building stateful, multi-actor applications with LLMs.” <https://langchain-ai.github.io/langgraph/>, 2024. Accessed 2026.
- [42] Qdrant, “Qdrant: Open-source vector search engine.” <https://qdrant.tech/documentation/>, 2024. Accessed 2026.
- [43] National Institute for Health and Care Excellence (NICE), “NICE guidance and advice.” <https://www.nice.org.uk/guidance>, 2024. Accessed 2026.
- [44] LangChain, “LangSmith: An all-in-one developer platform for every step of the LLM-powered application lifecycle.” <https://docs.smith.langchain.com/>, 2024. Accessed 2026.
- [45] European Parliament and Council of the European Union, “Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act).” Official Journal of the European Union, L series, 2024. Accessed 2026, <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>.

- [46] U.S. Food and Drug Administration, “Software as a Medical Device (SaMD): Clinical Evaluation – Guidance for Industry and Food and Drug Administration Staff.” FDA-2016-D-2483, 2017. Accessed 2026, <https://www.fda.gov/regulatory-information/search-fda-guidance-documents/software-medical-device-samd-clinical-evaluation>.
- [47] International Medical Device Regulators Forum, ““Software as a Medical Device”: Possible Framework for Risk Categorization and Corresponding Considerations.” IMDRF/SaMD WG/N12FINAL:2014, 2014. Accessed 2026, <https://www.imdrf.org/documents/software-medical-device-possible-framework-risk-categorization-and-corresponding-considerations>.